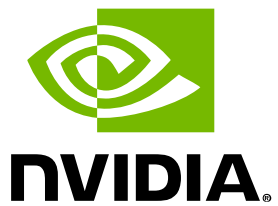


MODEL PARALLELISM: BUILDING AND DEPLOYING LARGE NEURAL NETWORKS

ADVANCED CONCEPTS OF LARGE DISTRIBUTED TRAINING



DEEP
LEARNING
INSTITUTE

PART 2

Advanced concepts of Large Distributed Training

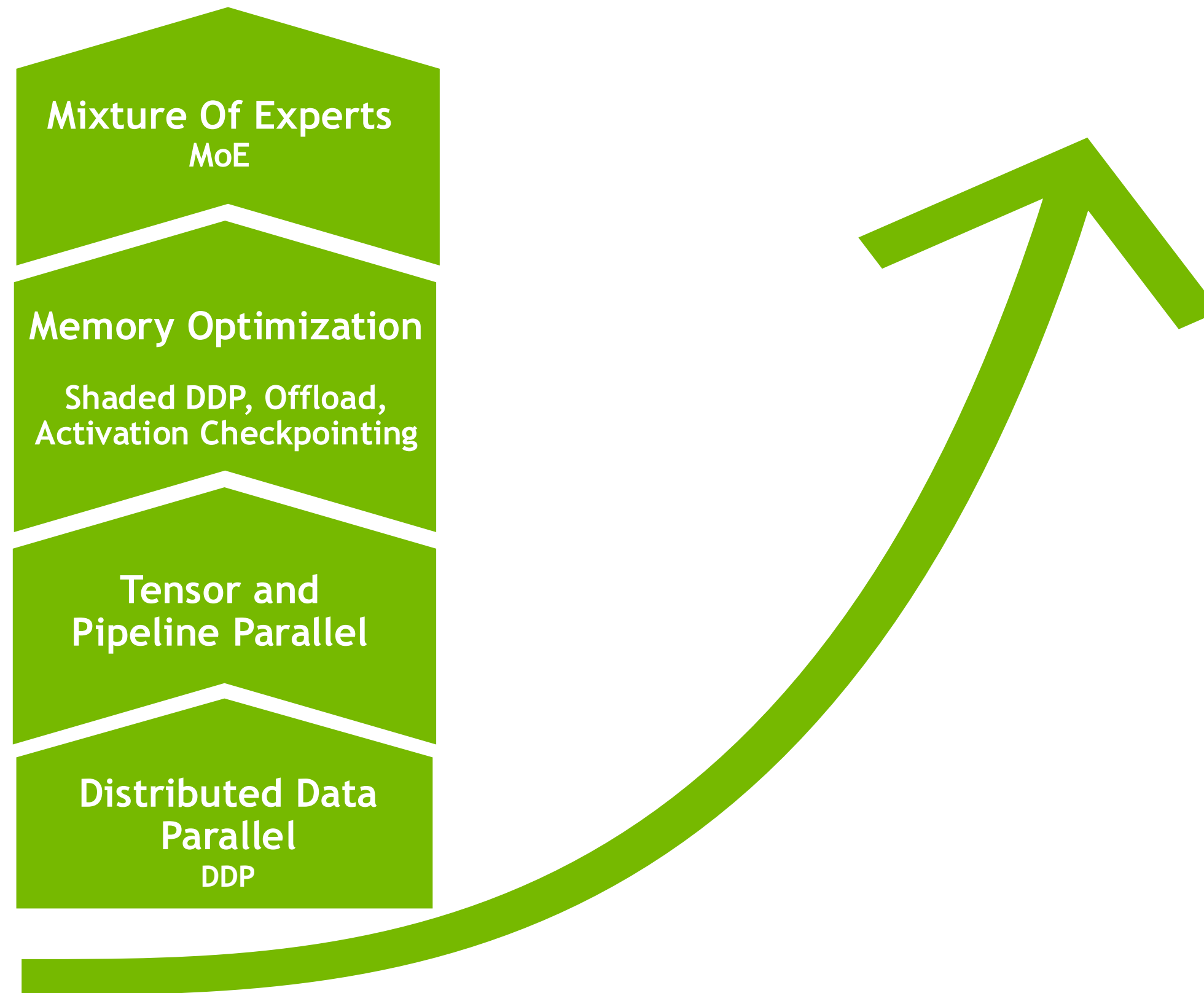
- Lecture
 - Distributed Data Parallel - DDP
 - Sharded data
 - Tensor/Pipeline Parallel
 - Sequence parallelism
 - Activations checkpointing
 - Offload
 - Mixture of Experts
 - Collective Communication
 - Hardware design
- Lab
 - Multi-Nodes Distributed Training for Computer Vision
 - Mixture of Experts

SCALING CHALLENGES

The background of the slide features a smooth gradient transitioning from a vibrant green on the left to a deep blue on the right. Overlaid on this gradient is a complex, abstract network of thin white lines connecting numerous small white dots, creating a mesh-like pattern that resembles a molecular structure or a data network. The density of these connections increases towards the right side of the image.

SCALING CHALLENGES

Distributed Training

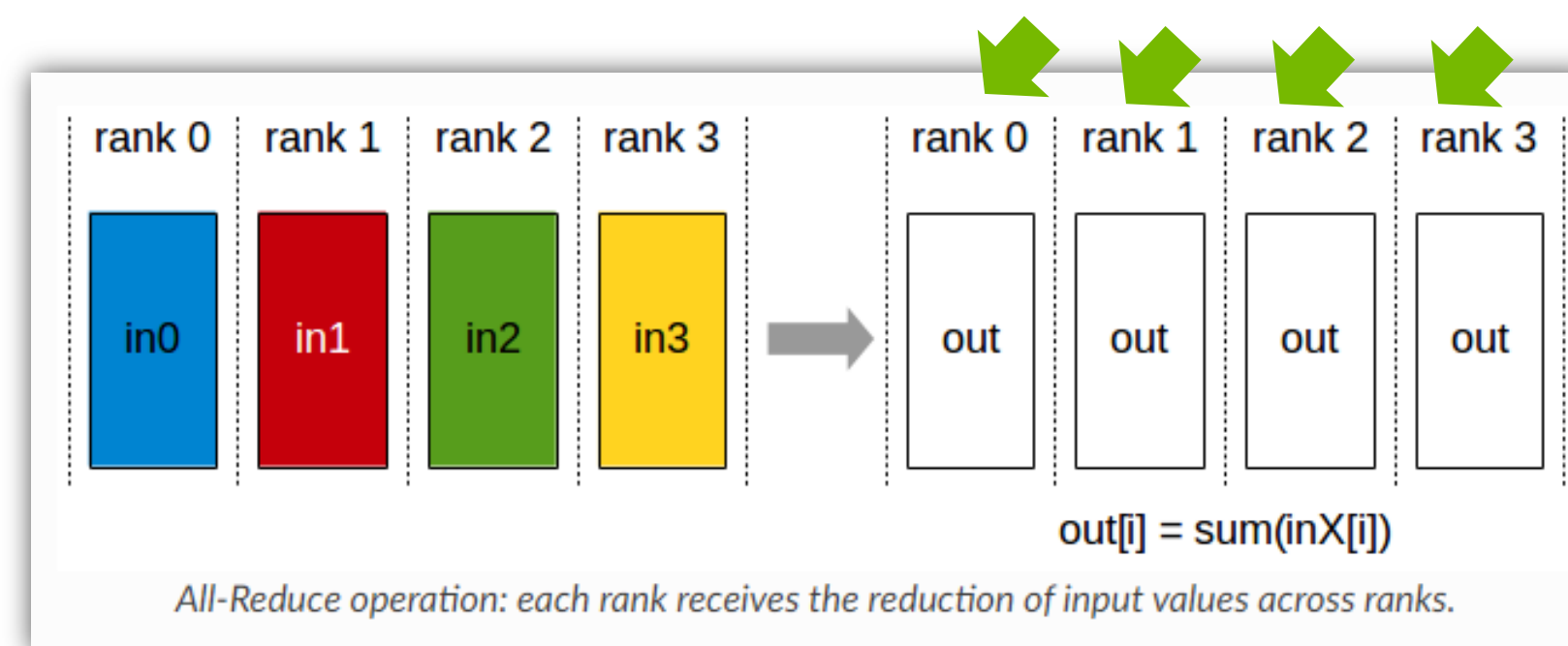
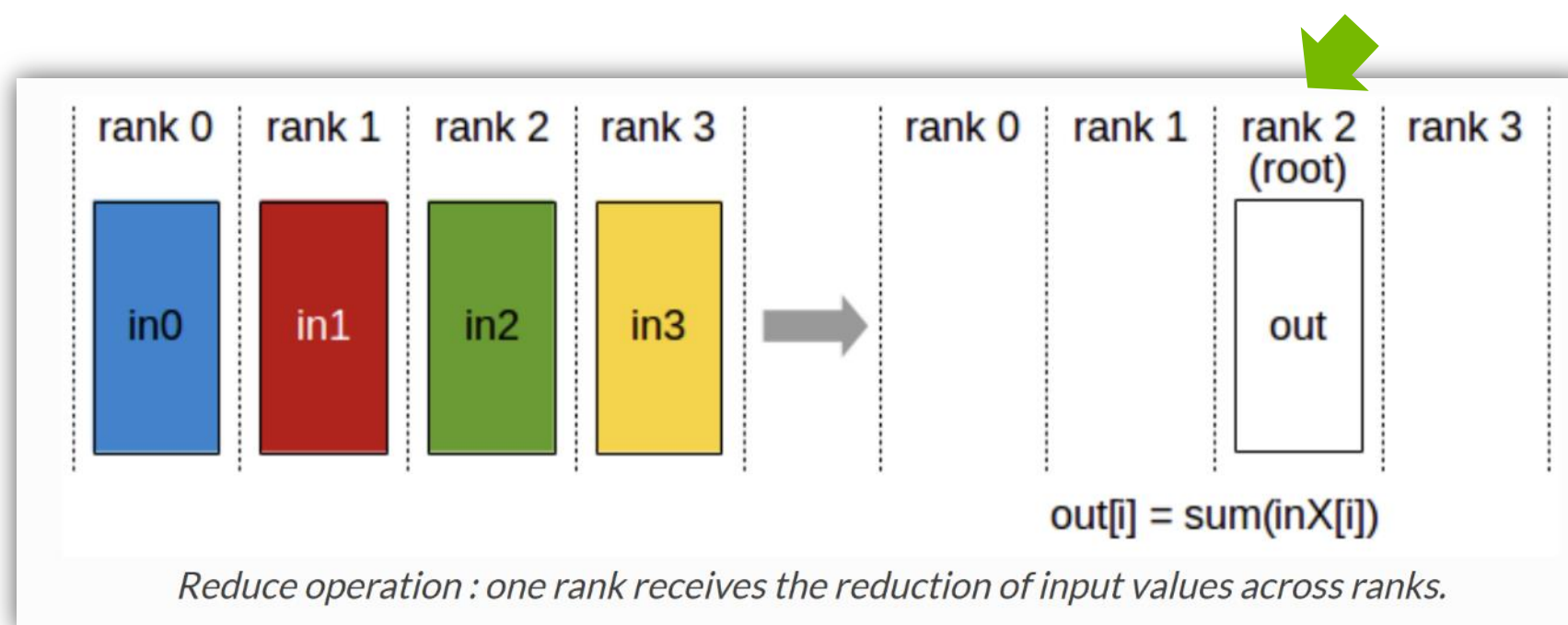




BASIC COLLECTIVE COMMUNICATIONS FOR DISTRIBUTED TRAINING

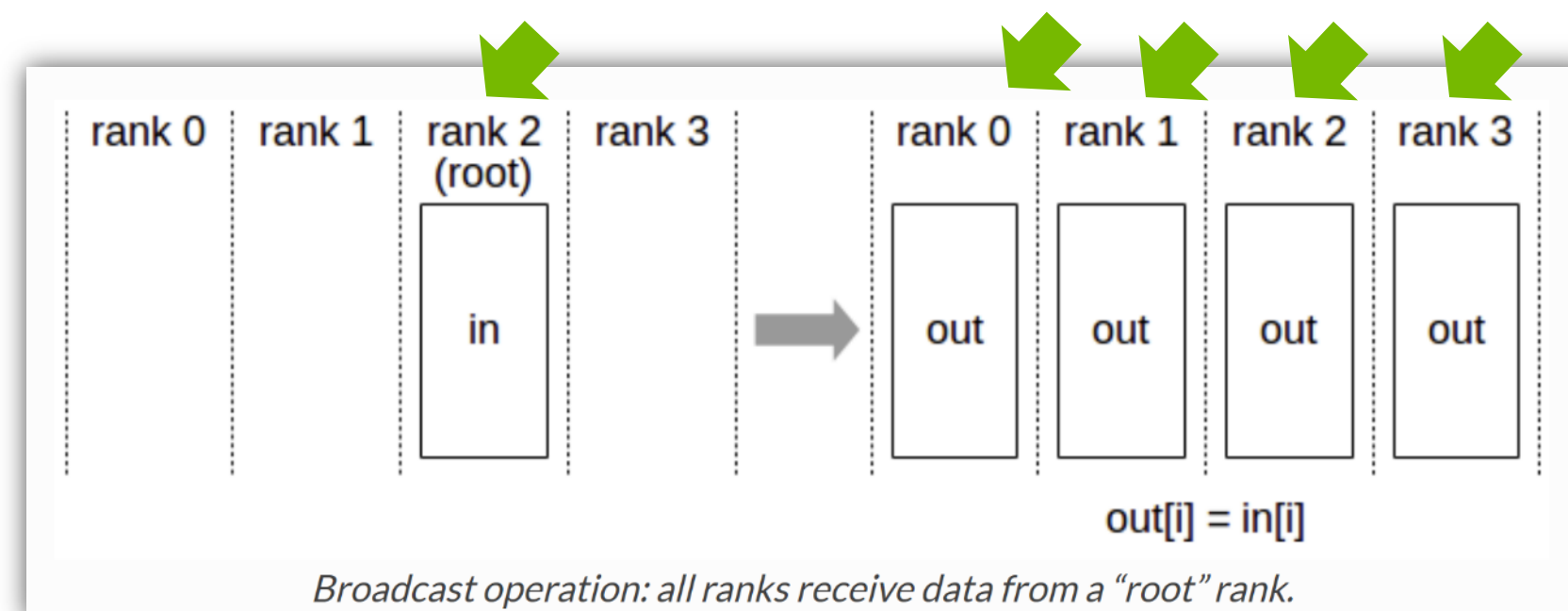
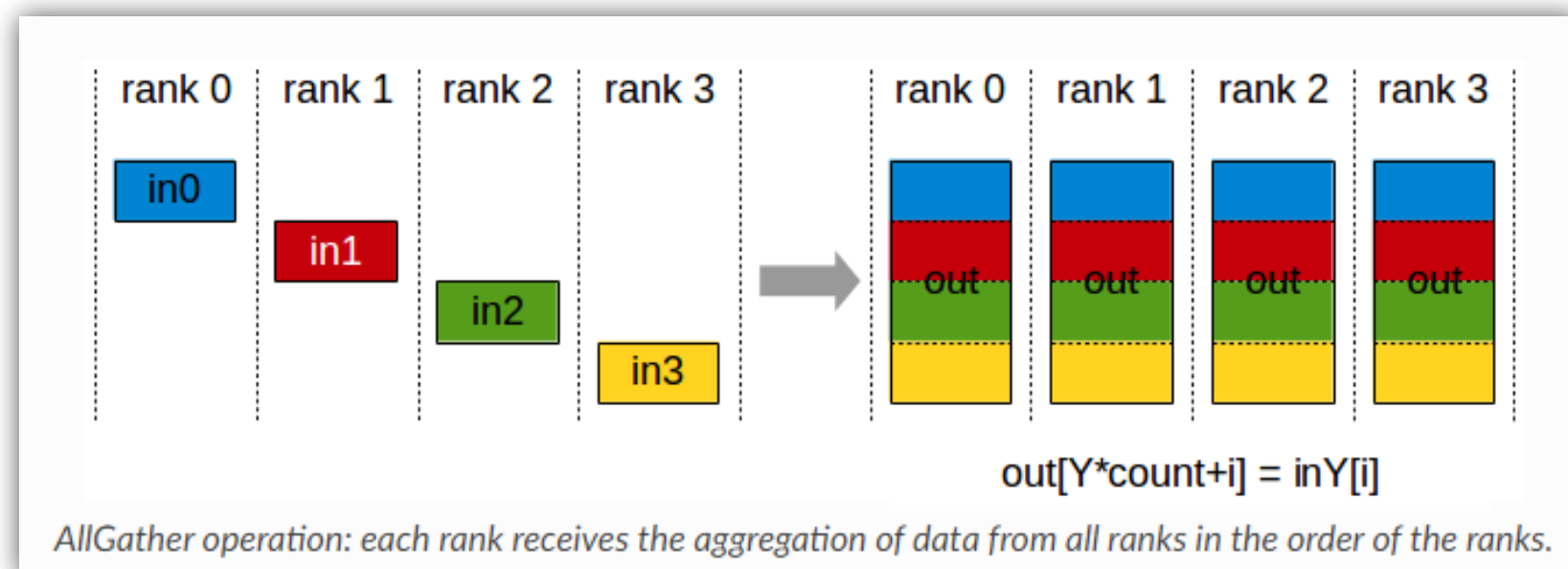
SOME CONCEPTS

Collective Communications



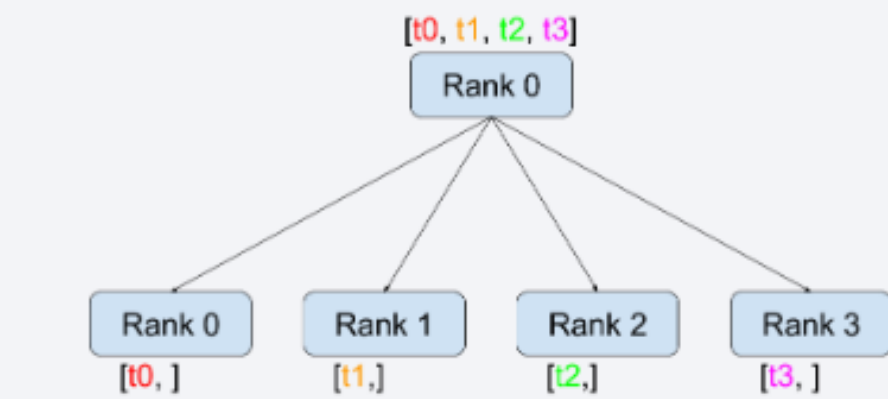
SOME CONCEPTS

Collective Communications

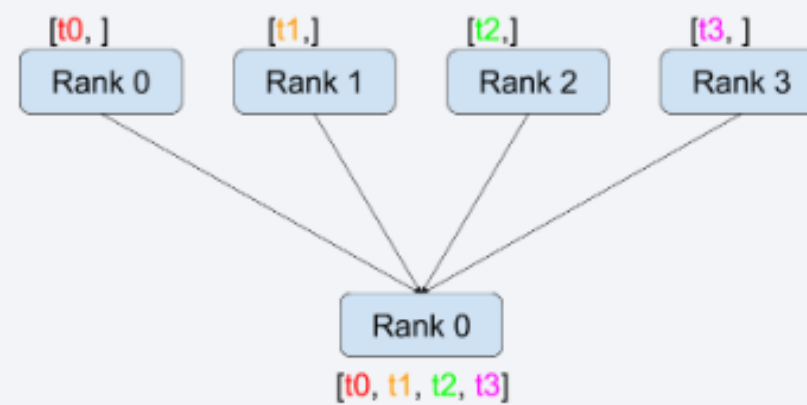


SOME CONCEPTS

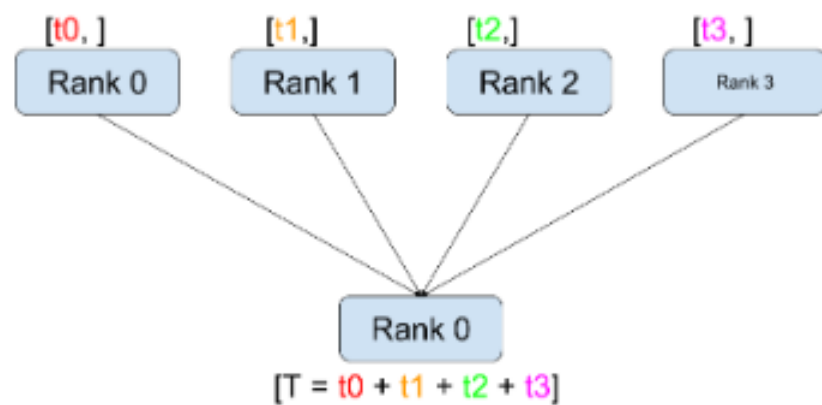
Collective Communications



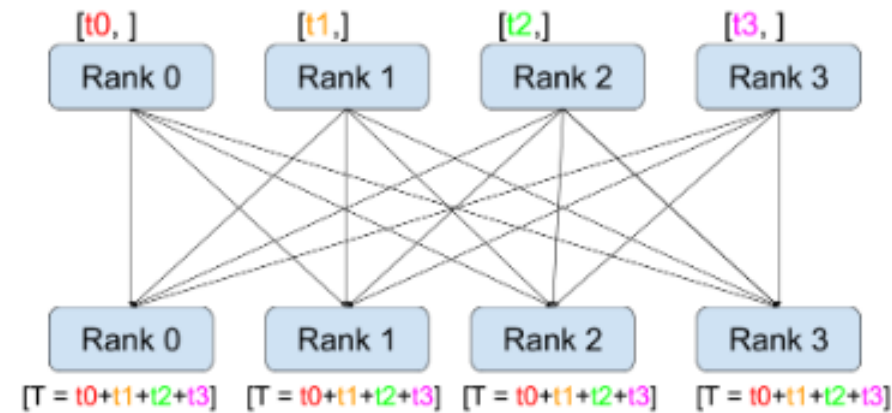
Scatter



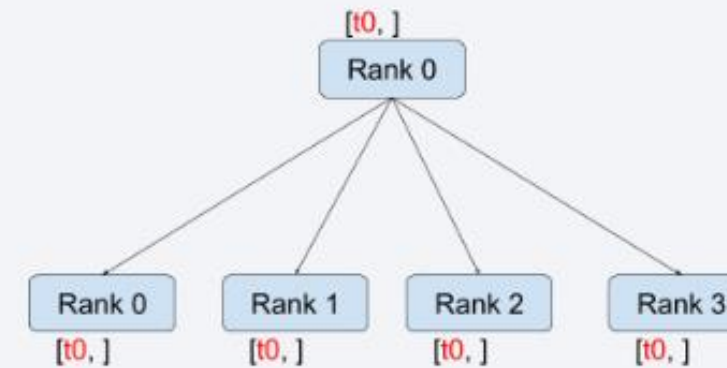
Gather



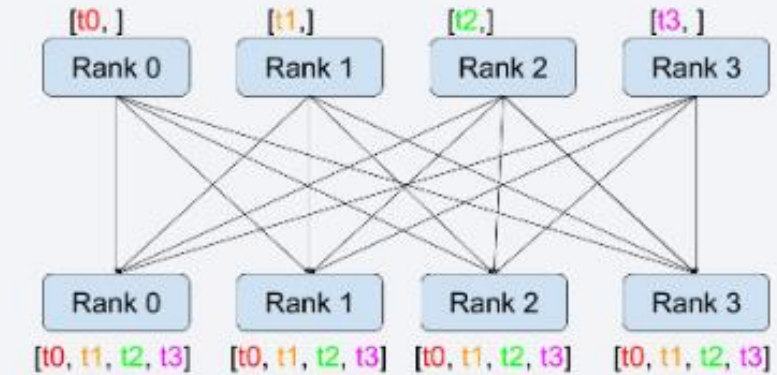
Reduce



All-Reduce

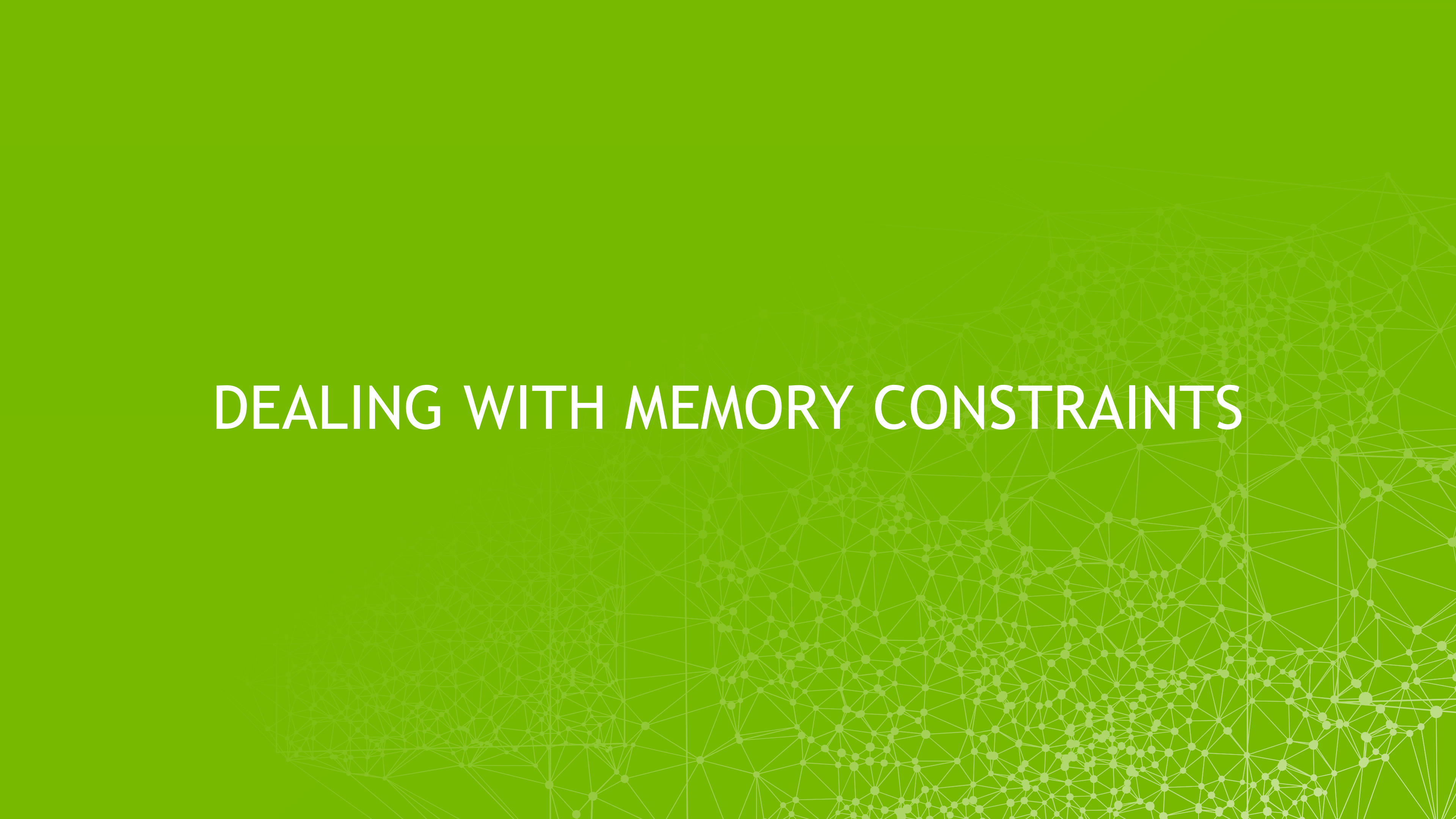


Broadcast



All-Gather

DEALING WITH MEMORY CONSTRAINTS

The background of the slide features a smooth gradient transitioning from a deep green on the left to a bright yellow on the right. Overlaid on this gradient is a complex, abstract network of small white dots connected by thin white lines, creating a mesh-like pattern that is denser on the right side of the image.

MEMORY CONSUMPTION IN LARGE MODEL TRAINING

Model States

Model Parameters

Gradients

Optimizer states (momentum and variances in Adam)

Mixed precision keeps two precision copies of parameters (FP16 and FP32)

Gradients are stored in FP16

Optimizer states are stored in FP32

Total memory = 16 x (#parameters) bytes

Residual States

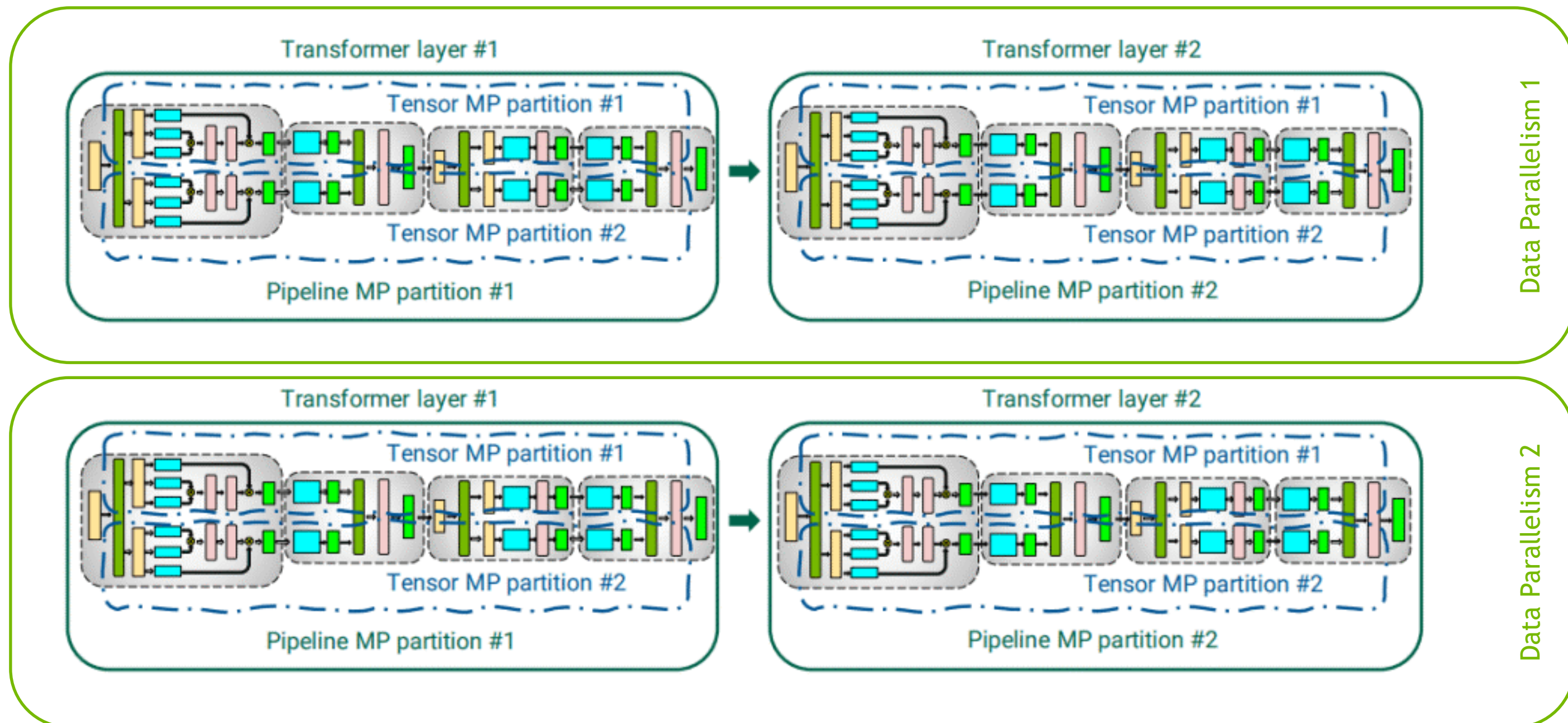
Activations

Temporary Buffers

Unusable Fragmented Memory

DEALING WITH MEMORY CONSTRAINTS

Various forms of parallelism



4 main types of parallelism are data, tensor, pipeline, and sequence

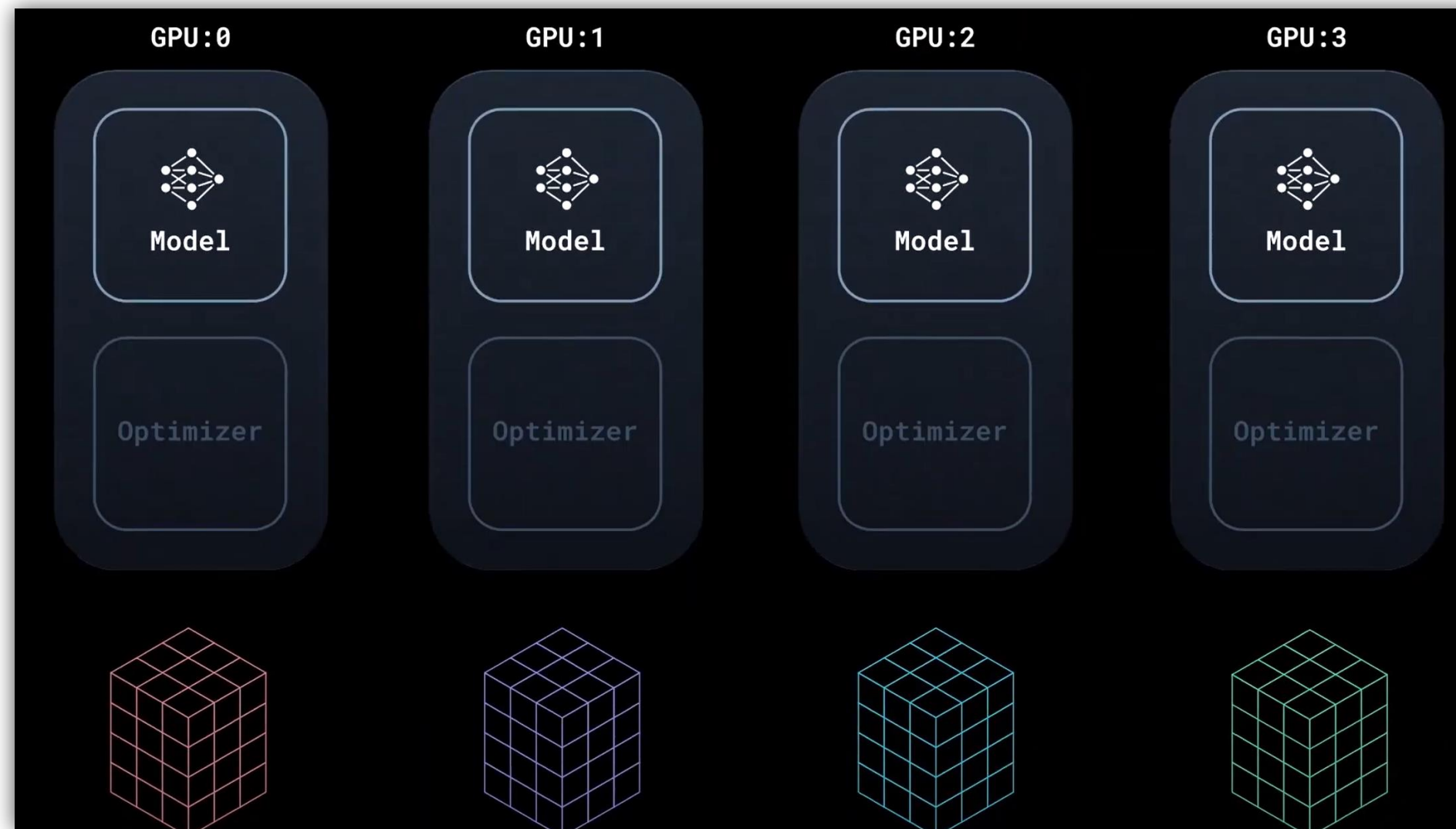


DISTRIBUTED DATA PARALLEL - DDP

PROCESS MORE DATA IN THE SAME TIME PERIOD

DATA PARALLELISM

PyTorch Distributed Data Parallel

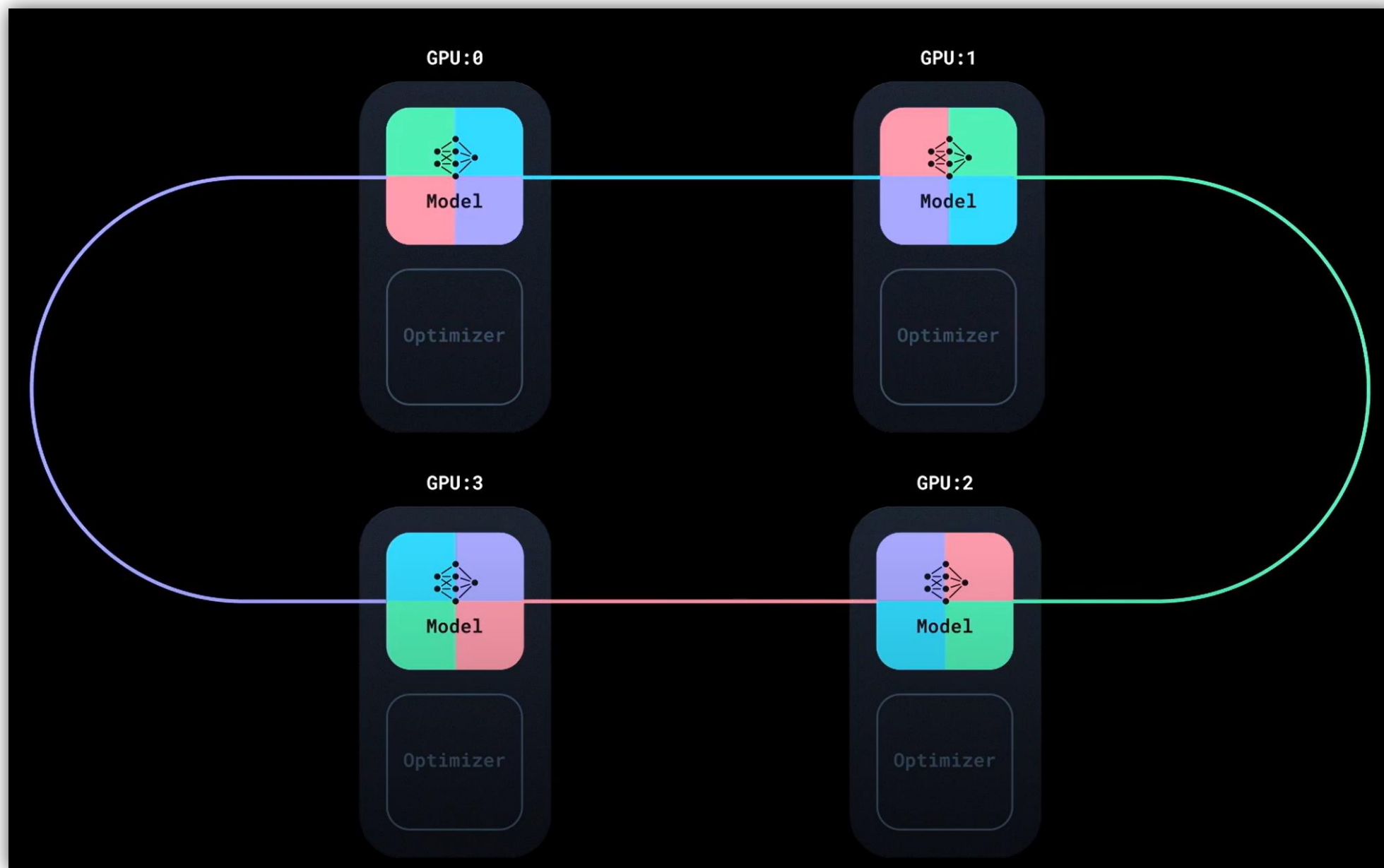


- Process more data by scaling model replicas
- Each GPU has a copy of the model
- Optimizer states and random seeds are the same
- Each GPU gets a different batch of data
- Concurrently processing 4x the data of a single GPU
- Data sampling is handled by a Distributed Sampler

Data Parallel training at large scale may affect model quality

DATA PARALLEL

PyTorch Data Parallel Training



Each model is different because it trained on different batch of data

Gradients from each GPU need to be synchronized, represented by the curved lines

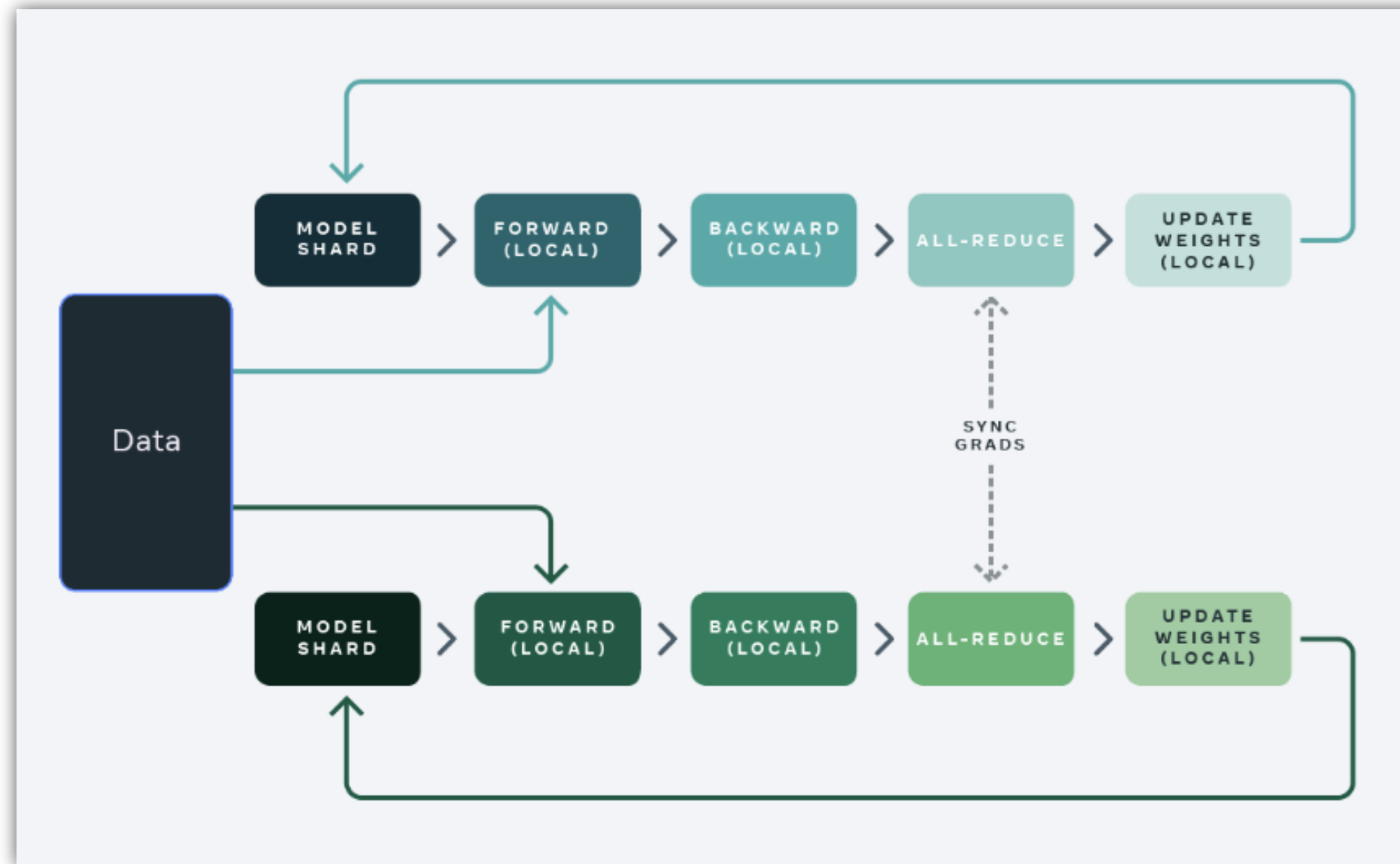
Synchronization is done with bucketed Ring-AllReduce algorithm

Algorithm overlaps gradient computation with communication so GPU is always utilized

Scaling with data parallel introduces communication overhead when syncing gradients

STANDARD DISTRIBUTED DATA PARALLEL - DDP

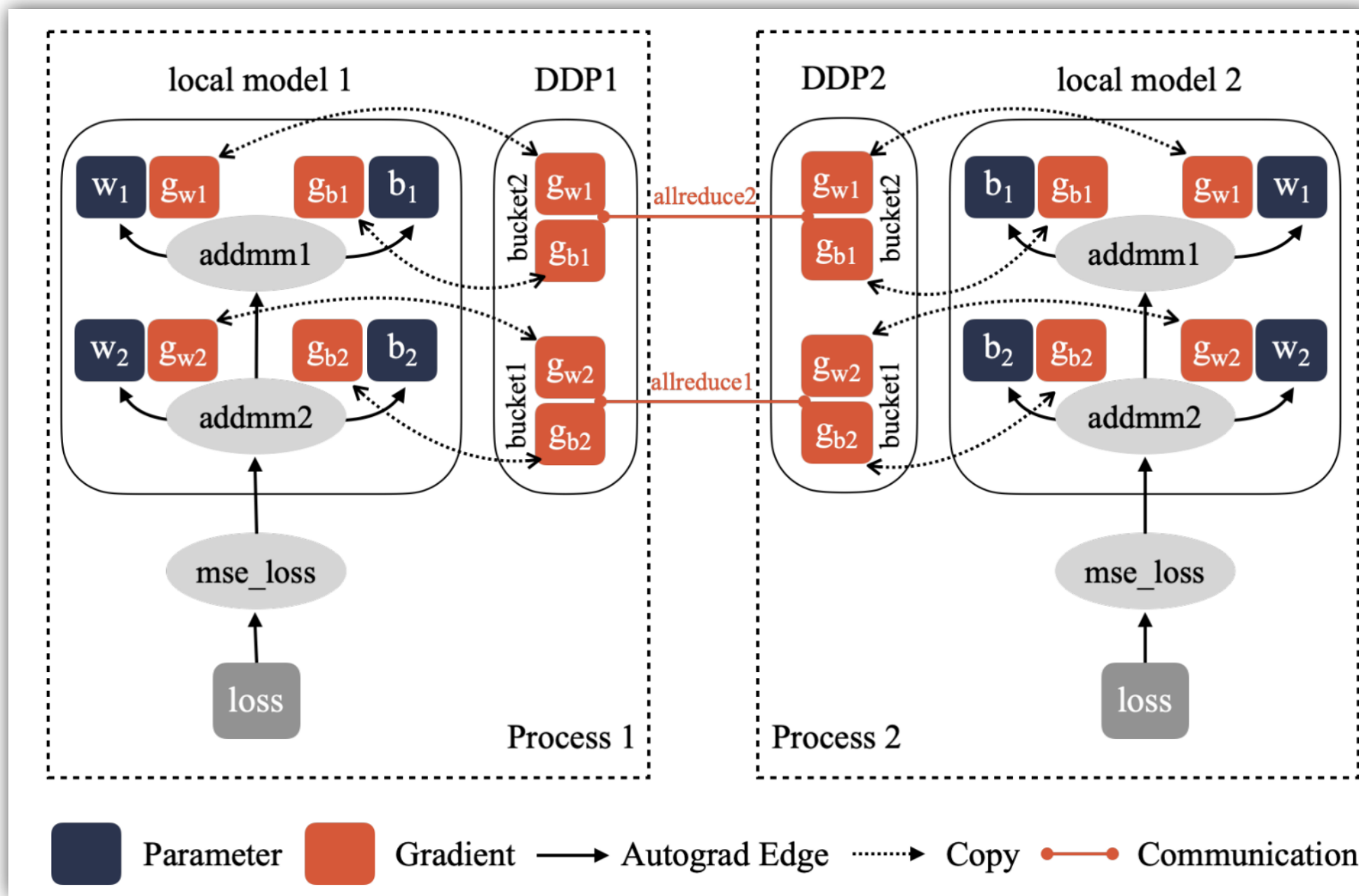
PyTorch Data Parallel Training



Gradients are synchronized before the update step, ensured model replicas stay consistent across iterations

DISTRIBUTED DATA PARALLEL - DDP

PyTorch Distributed Data Parallel



Two models exist as two gpus (processes)

Model parameters are shown in dark blue

Gradients (obtained after forward pass) are shown in orange

Gradients are copied/pooled together in buckets to increase communication efficiencies

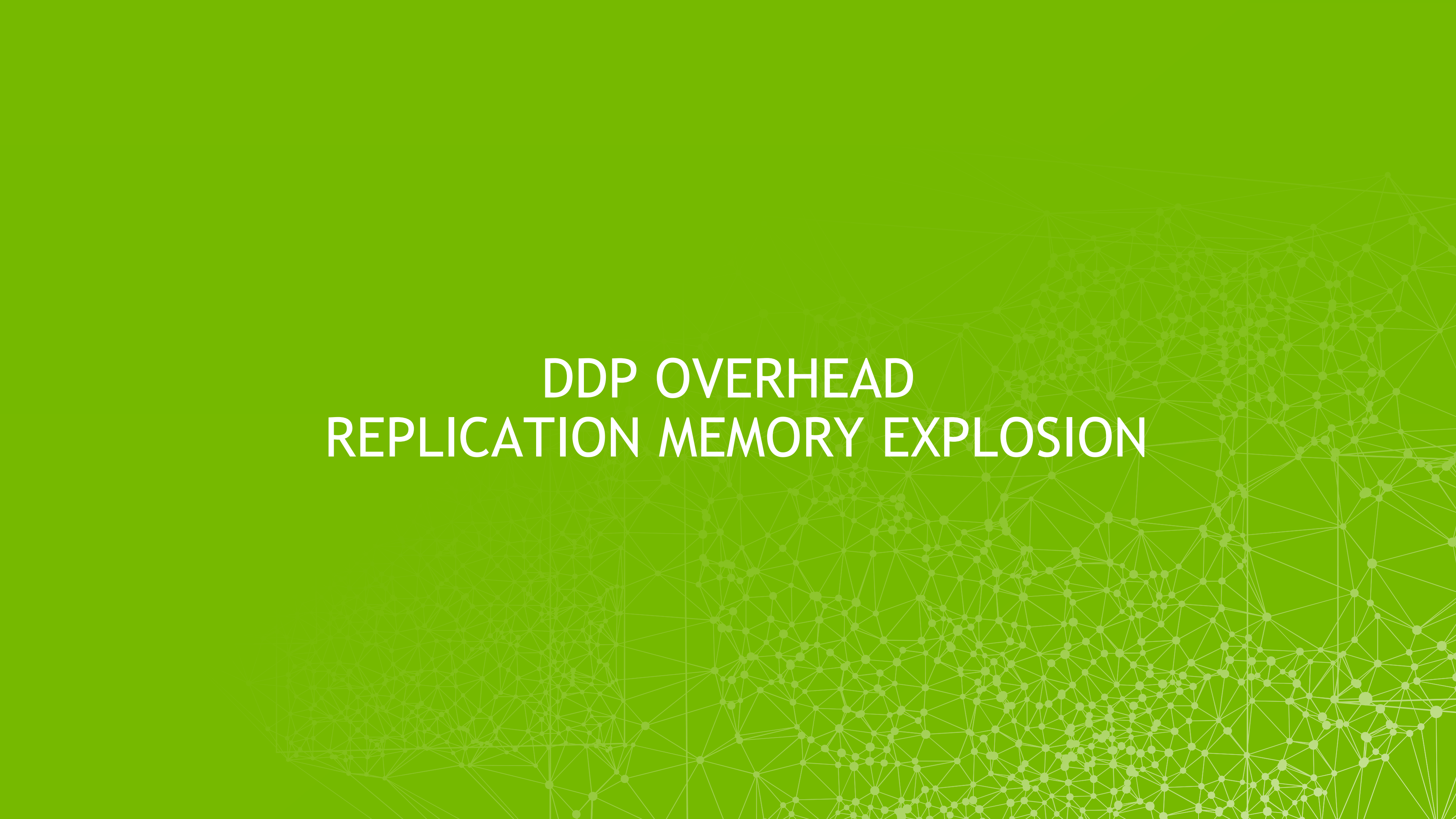
Gradients are synchronized as backpropagation progresses

DISTRIBUTED DATA PARALLEL - DDP

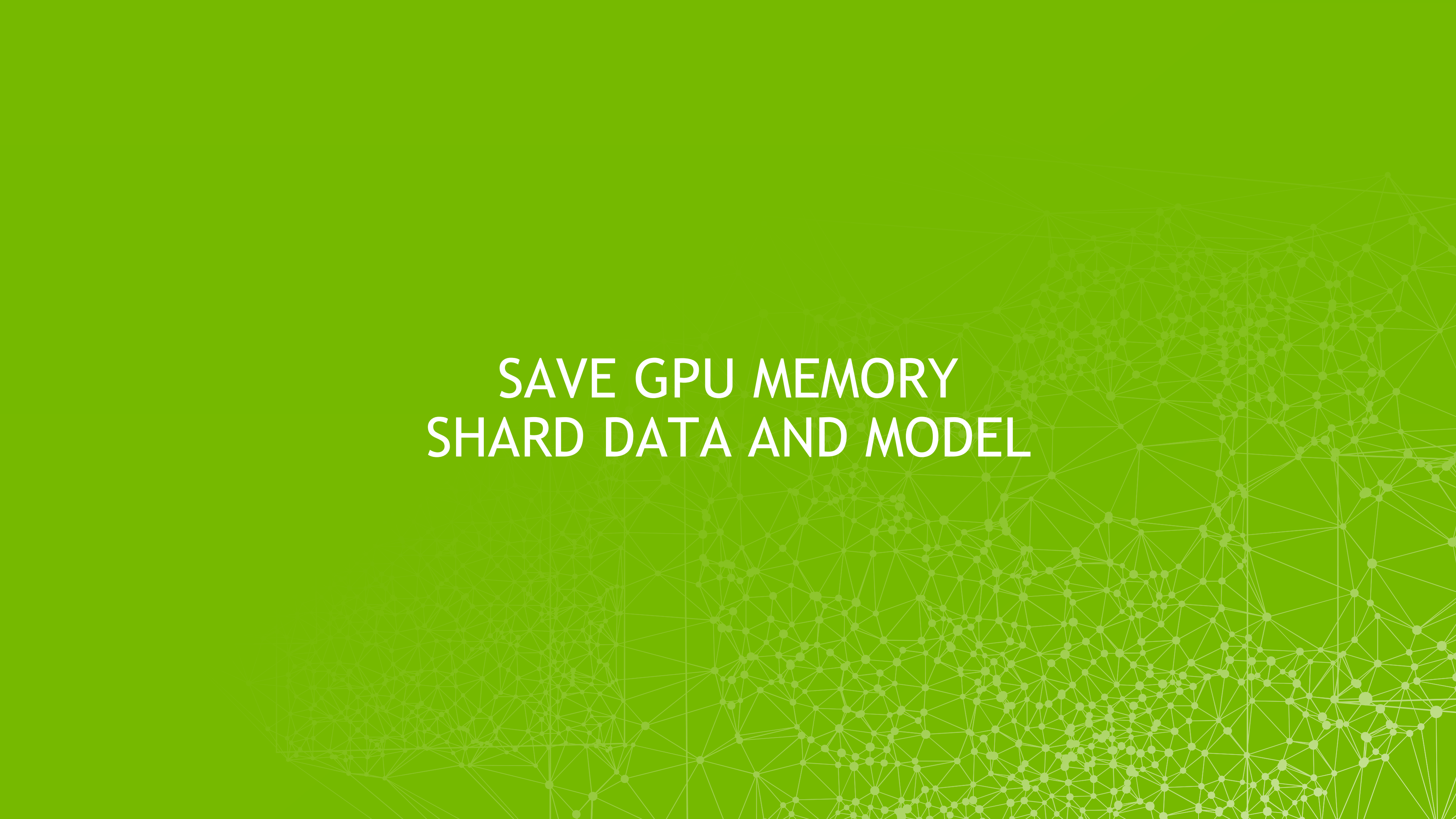
PyTorch Distributed Data Parallel - Communication operations across hardware types

Backend	gloo		mpi		nccl	
Device	CPU	GPU	CPU	GPU	CPU	GPU
send	✓	✗	✓	?	✗	✓
recv	✓	✗	✓	?	✗	✓
broadcast	✓	✓	✓	?	✗	✓
all_reduce	✓	✓	✓	?	✗	✓
reduce	✓	✗	✓	?	✗	✓
all_gather	✓	✗	✓	?	✗	✓
gather	✓	✗	✓	?	✗	✓
scatter	✓	✗	✓	?	✗	✗
reduce_scatter	✗	✗	✗	✗	✗	✓
all_to_all	✗	✗	✓	?	✗	✓
barrier	✓	✗	✓	?	✗	✓

NCCL has the best communication performance across NVIDIA hardware



DDP OVERHEAD REPLICATION MEMORY EXPLOSION



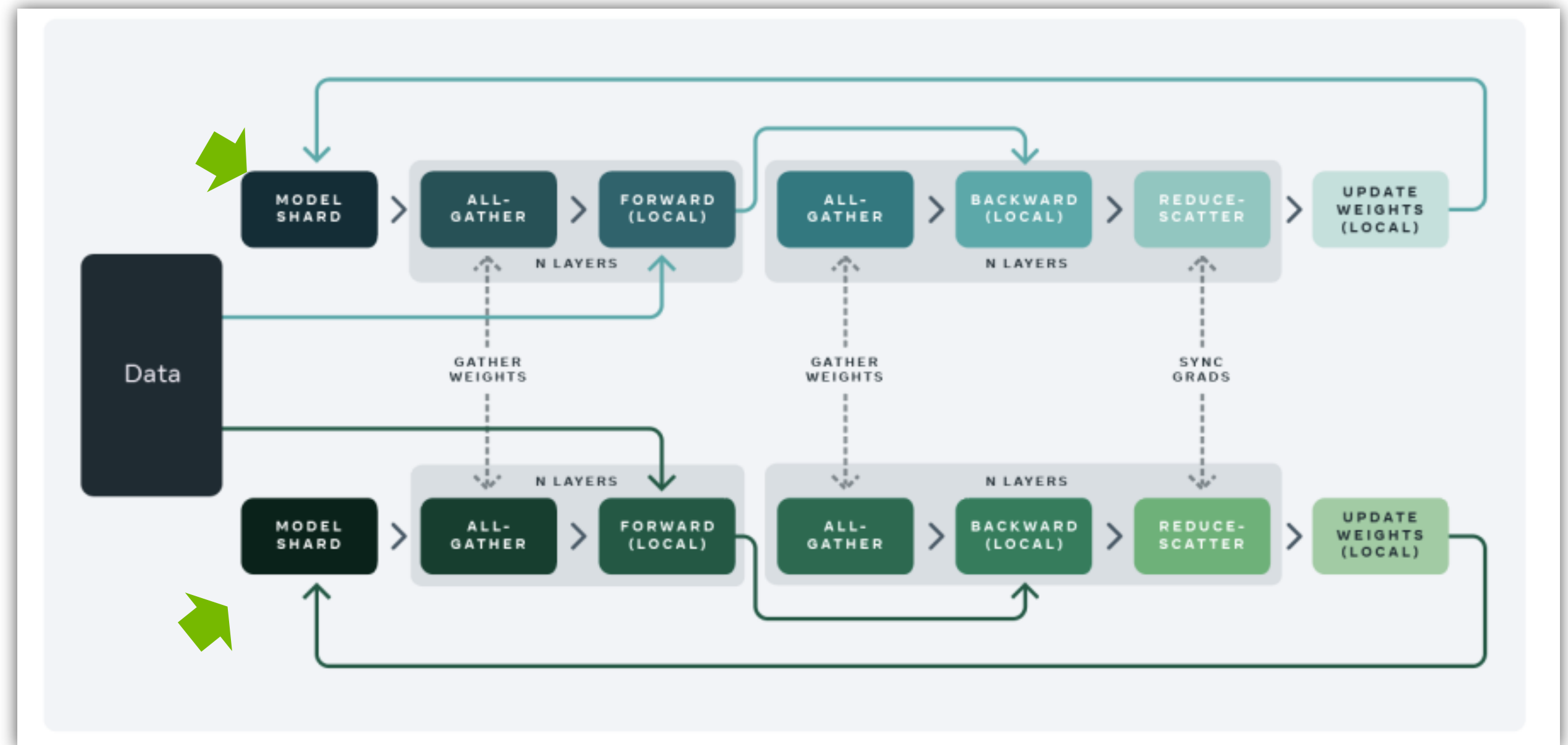
**SAVE GPU MEMORY
SHARD DATA AND MODEL**

DISTRIBUTED DATA PARALLEL - DDP

FairScale: Fully Sharded Data Parallel - FSDP

For each GPU:

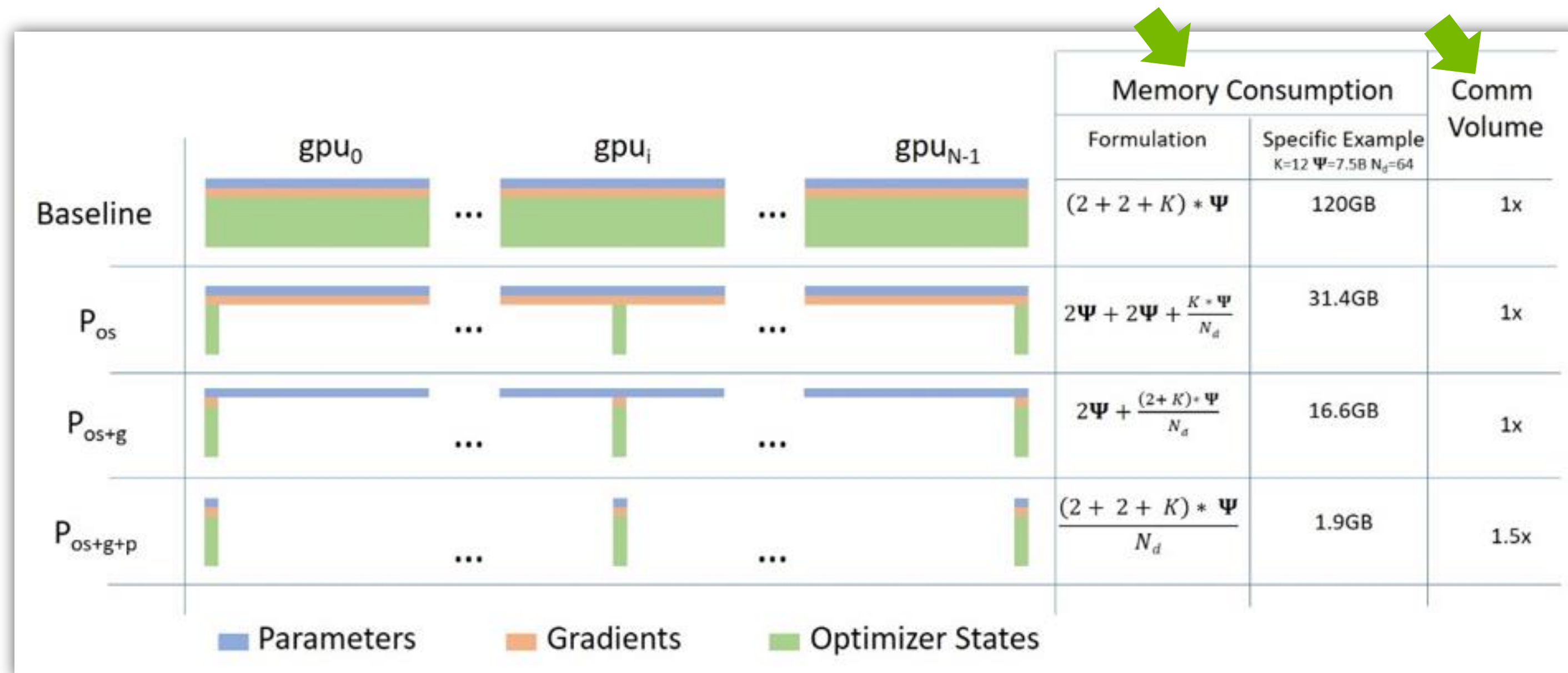
1. Get the shard of the model
2. Get the shard of the data
3. Local forward pass: **Gather weights** from the others
4. Local backward pass: **Gather again** weights from the others
5. Local weights shard update: Synchronize Gradients



To maximize memory efficiency, we can discard the full weights after each layer's forward pass, saving memory for subsequent layers.

SHARDED DATA PARALLELISM

Communication overheads



Increases in sharding lead to increases in communications, reducing memory but increasing training times

SHARDED DATA PARALLELISM

GPT2 example: Enable ZeRO optimization

```
# Construct FP16, distributed, GPT2 model
model = GPT2Model(num_layers=args.num_layers, ...)
model = FP16_Module(model)
model = DistributedDataParallel(model, ...)

...

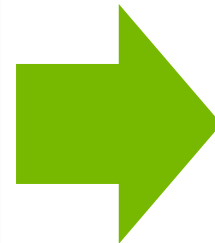
# Construct FP16 Adam optimizer
optimizer = Adam(param_groups, ...)
optimizer = FP16_Optimizer(optimizer, ...)
```

```
# Forward pass
output = model(tokens, ...)

# Backward pass
optimizer.backward(loss)

# Parameter update
optimizer.step()
```

Python train.py <args>



```
# Construct GPT2 model
model = GPT2Model(num_layers=args.num_layers, ...)

# Construct Adam optimizer
optimizer = Adam(param_groups, ...)

# Wrap model, optimizer, and lr scheduler
model, optimizer, lr_scheduler, _ = deepspeed.initialize(
    args=args,
    model=model,
    optimizer=optimizer,
    lr_scheduler=lr_scheduler,
    mpu=mpu
)
```

```
# Forward pass
output = model(tokens, ...)

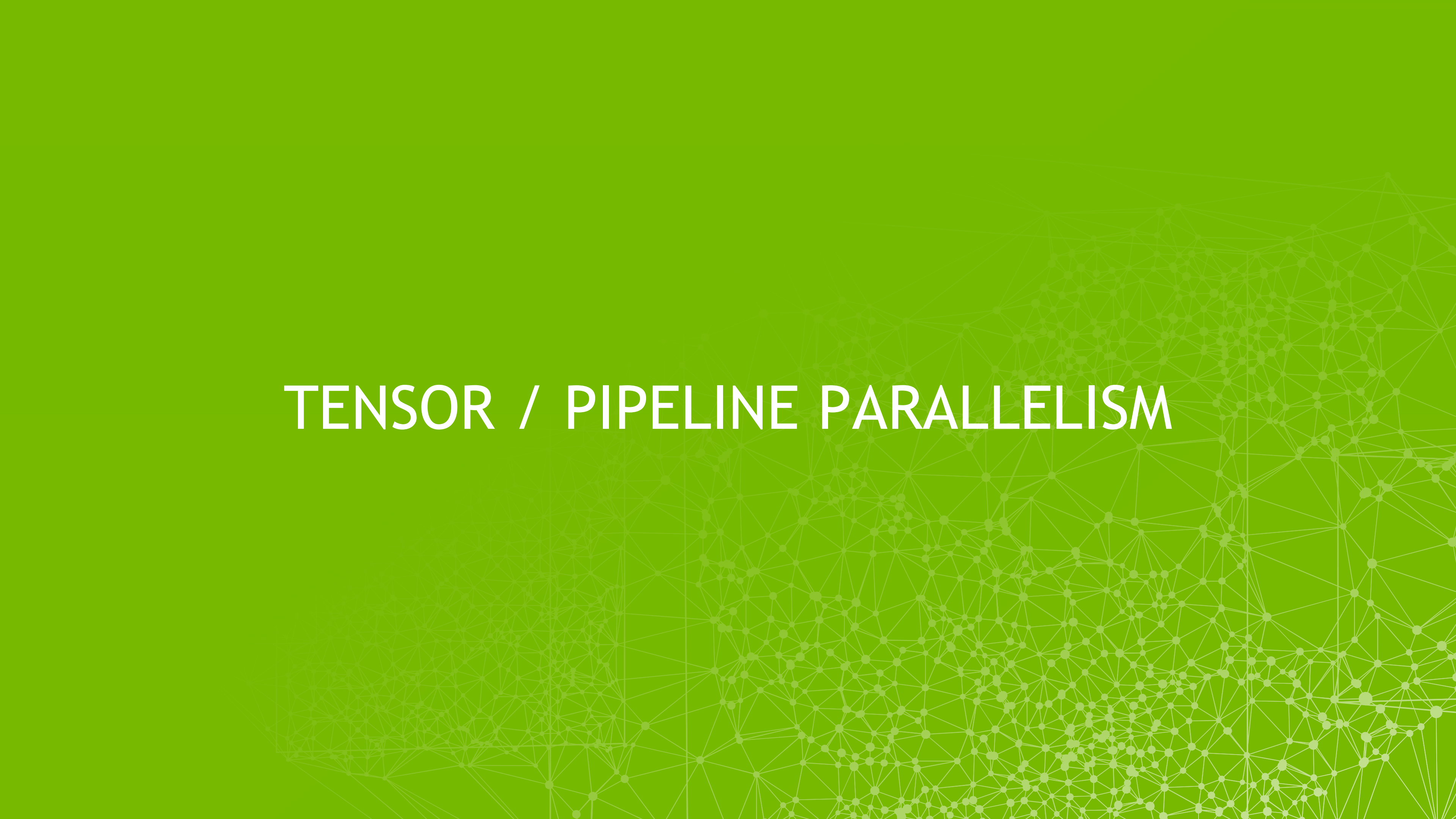
# Backward pass
model.backward(loss)

# Parameter update
model.step()
```

deepspeed train.py <args> ¥
--deepspeed_config ds_config.json

```
{
  "train_batch_size": 64,
  "gradient_accumulation_steps": 1,
  "steps_per_print": 1,
  "zero_optimization": {
    "stage": 3,
    "stage3_max_live_parameters": 1e9,
    "stage3_max_reuse_distance": 1e9,
    "stage3_prefetch_bucket_size": 1e7,
    "stage3_param_persistence_threshold": 1e5,
    "reduce_bucket_size": 1e7,
    "contiguous_gradients": true
  },
  "gradient_clipping": 1.0,
  "fp16": {
    "enabled": true,
    "loss_scale": 0,
    "loss_scale_window": 1000,
    "hysteresis": 2,
    "min_loss_scale": 1
  },
  "wall_clock_breakdown": true,
  "zero_allow_untested_optimizer": false
}
```

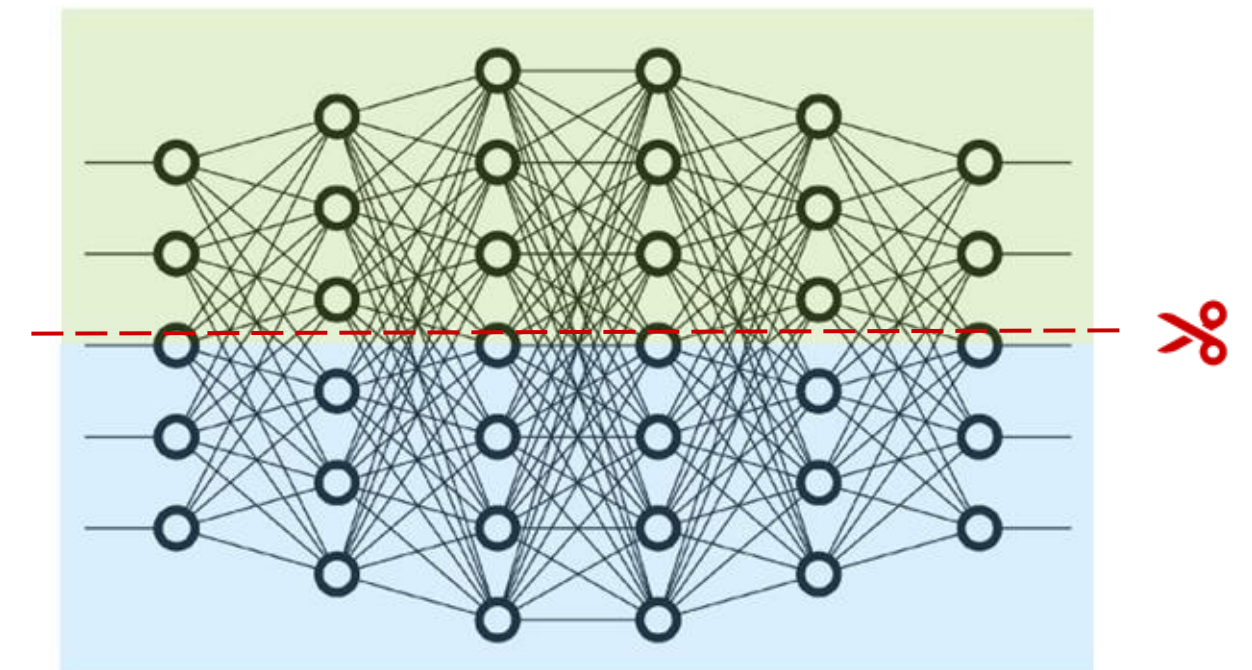
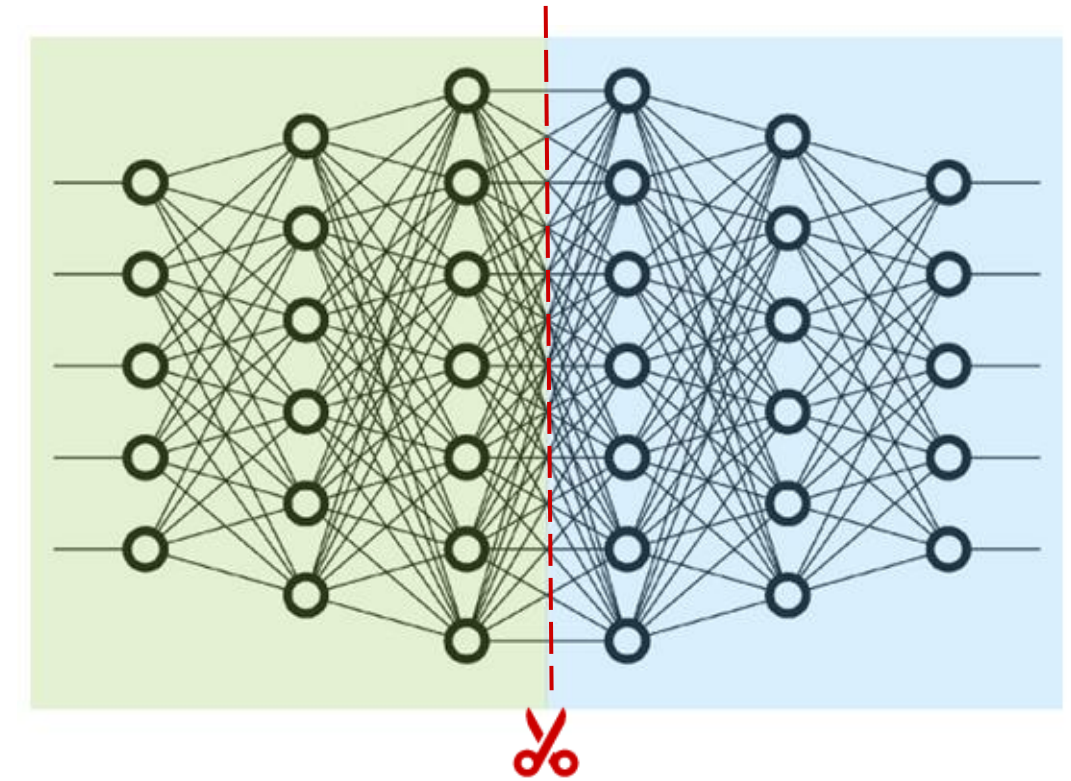
TENSOR / PIPELINE PARALLELISM

The background of the slide features a smooth gradient from a vibrant green on the left to a deep blue on the right. Overlaid on this gradient is a complex, abstract network of white dots and thin white lines, resembling a neural network or a data mesh. The dots are of varying sizes and are interconnected by lines of different thicknesses, creating a sense of depth and connectivity. The overall aesthetic is modern and technological.

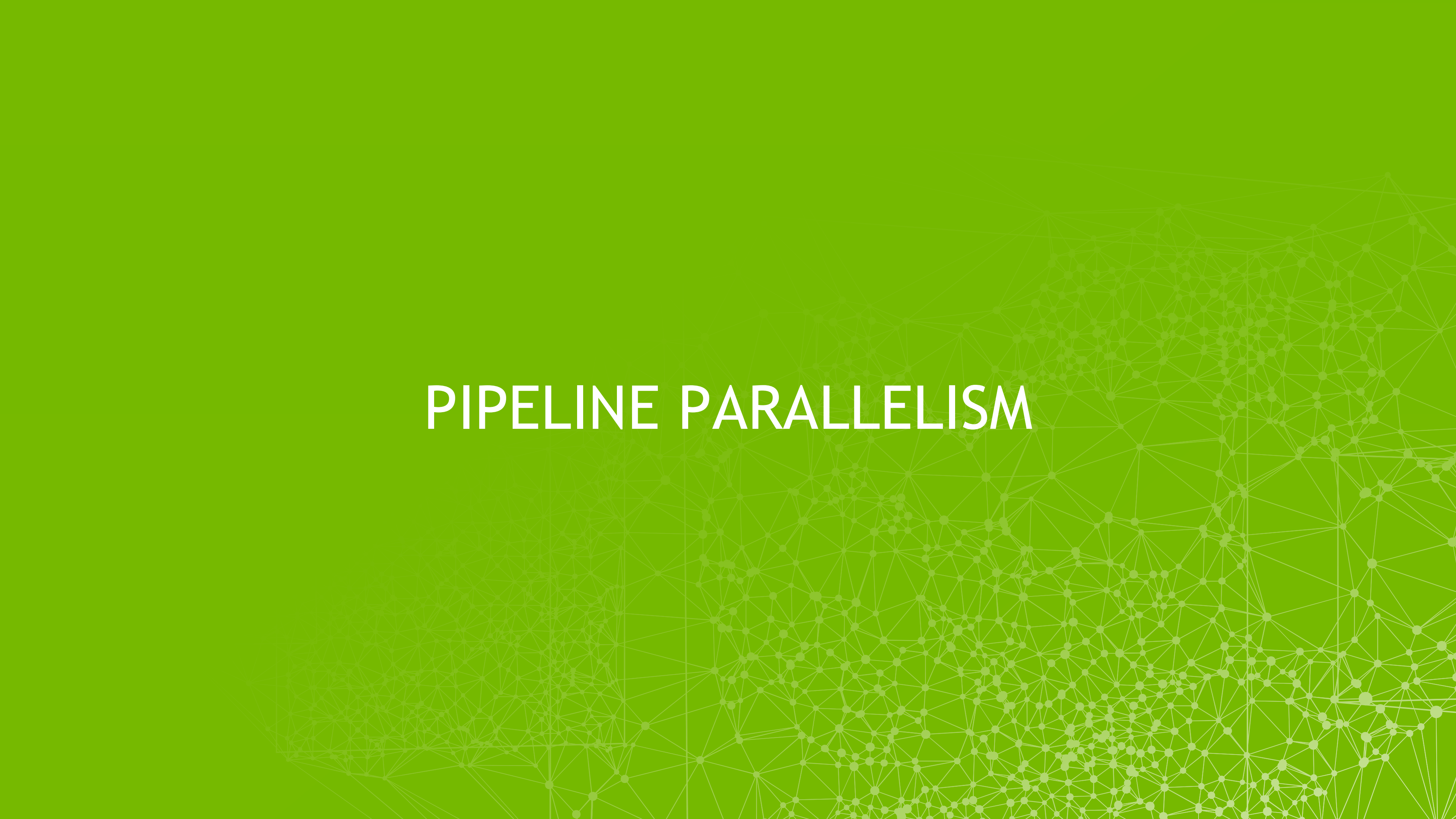
TECHNOLOGIES THAT ENABLE SCALING LARGE MODELS

Complementary Types of Parallelism

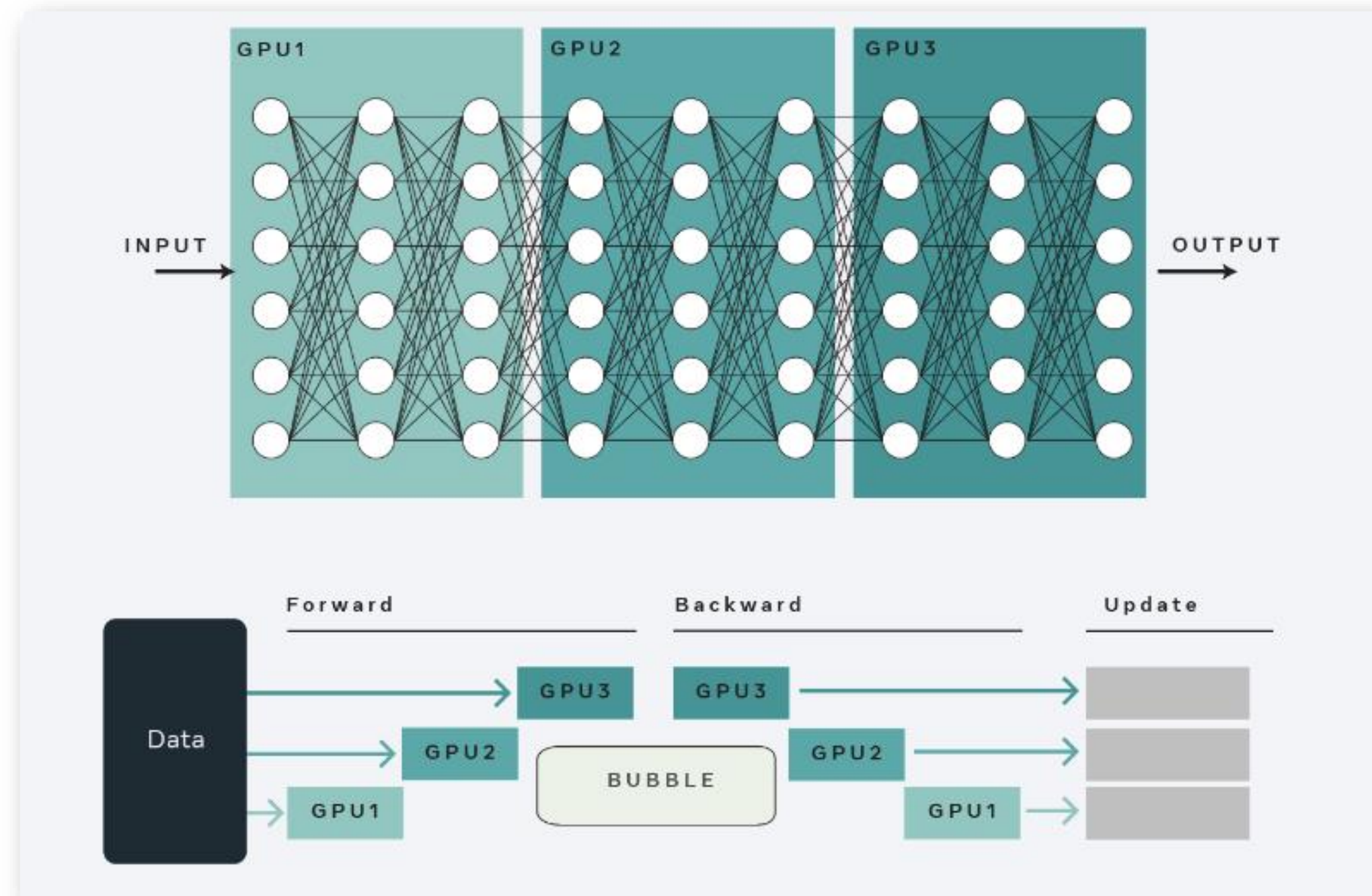
- Pipeline (Inter-Layer) Parallelism
 - Split contiguous sets of layers across multiple GPUs
 - Layers 0,1,2 and layers 3,4,5 are on different GPUs
 - *Maximizes GPU utilization in single-node*
- Tensor (Intra-Layer) Parallelism
 - Split individual layers across multiple GPUs
 - Both devices compute different parts of Layers 0,1,2,3,4,5
 - *Minimizes Latency in single-node*



PIPELINE PARALLELISM

The background of the slide features a smooth gradient from a deep green at the top to a bright yellow at the bottom. Overlaid on this gradient is a complex, abstract network of thin white lines connecting numerous small white dots, creating a mesh-like or molecular structure that spans the entire width of the image.

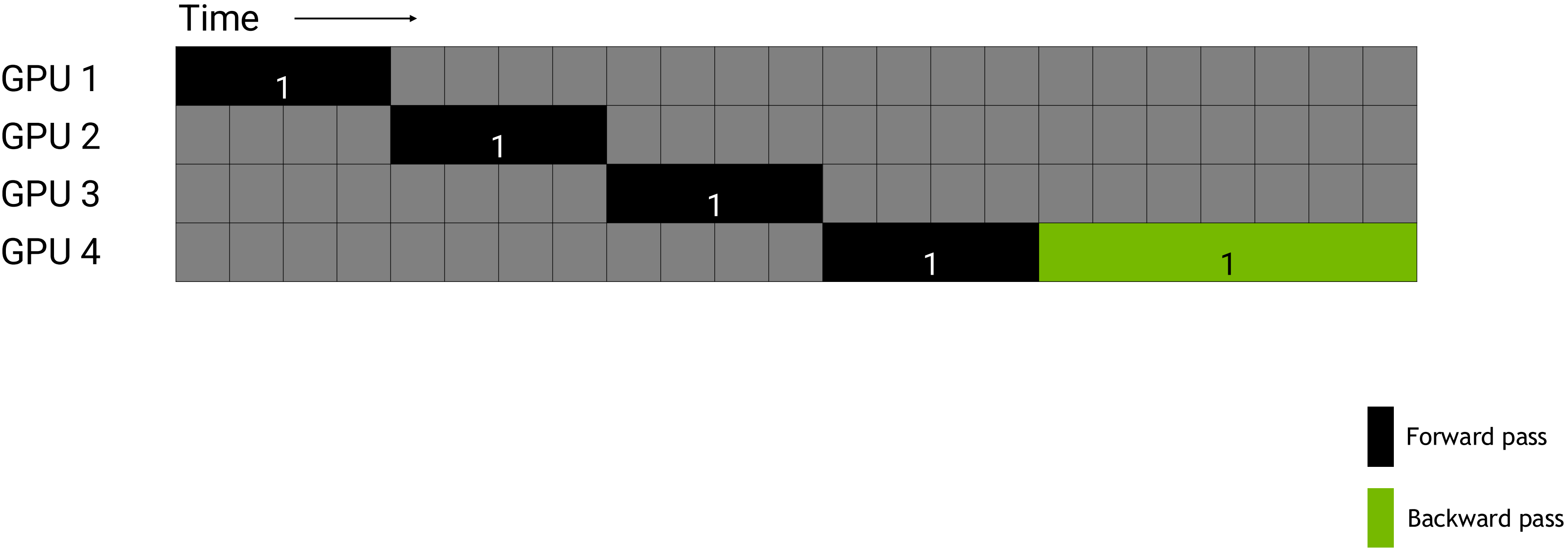
PIPELINE PARALLELISM



Pipeline parallel is sequentially processed as groups of continuous model layers, leading to GPU underutilization (bubbles)

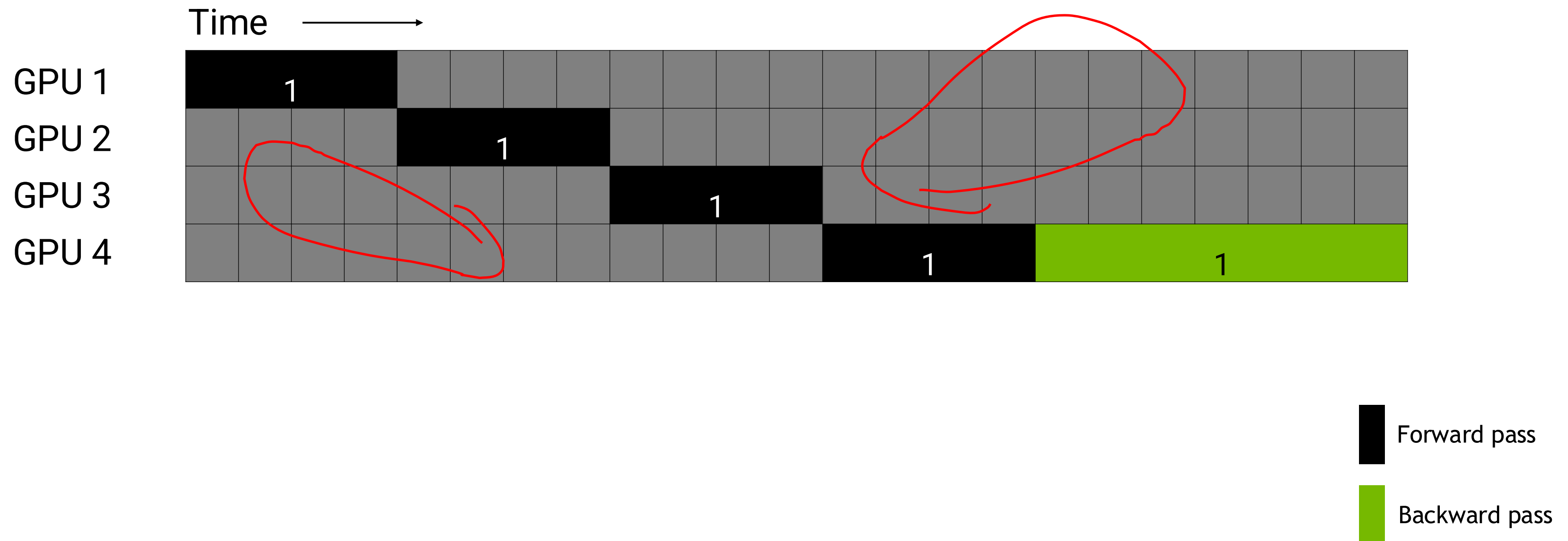
PIPELINE PARALLELISM

Challenges



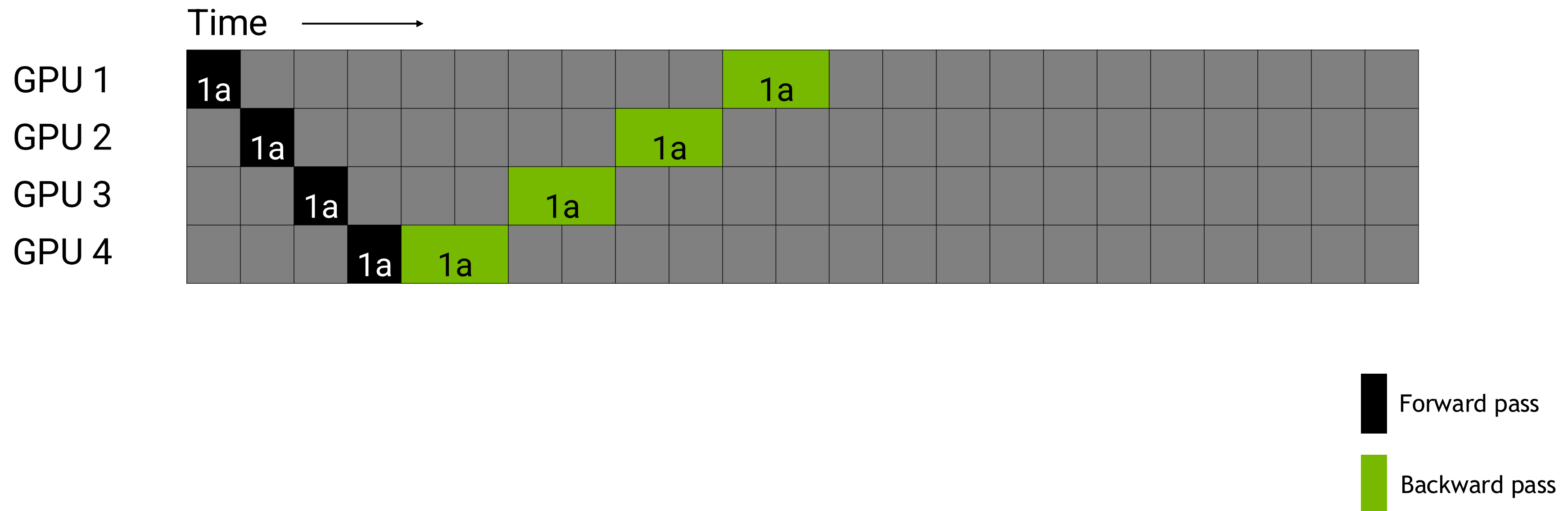
PIPELINE PARALLELISM

Challenges - Idle Workers



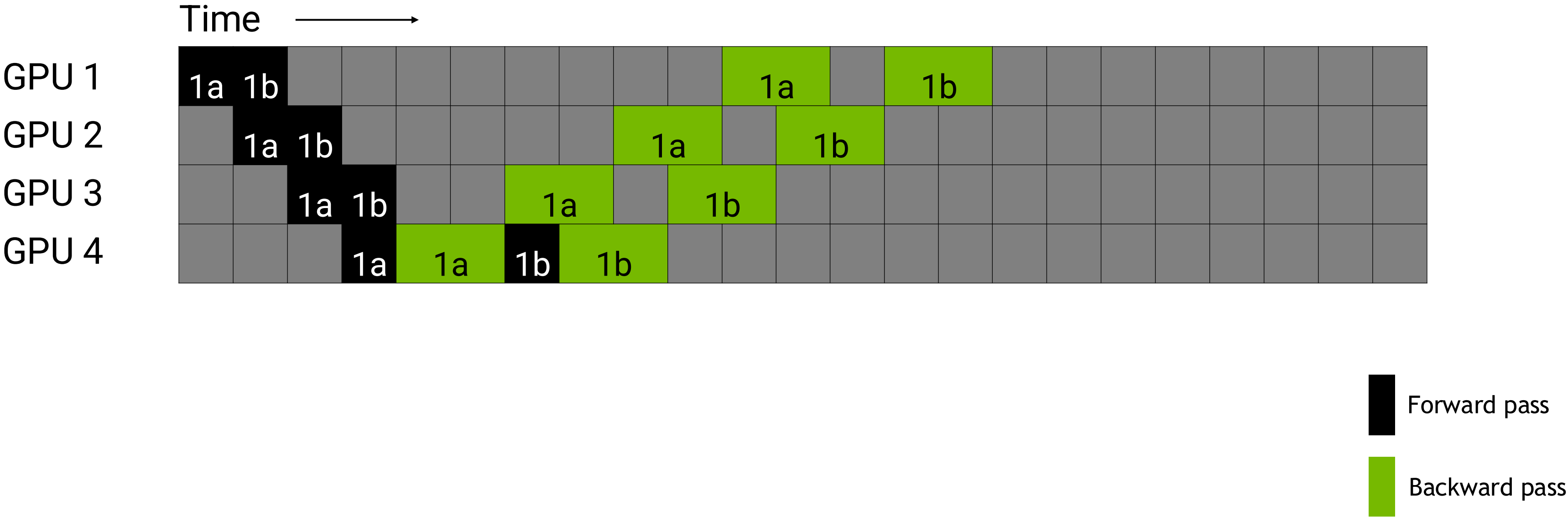
PIPELINE PARALLELISM

Split batch into micro batches and pipeline execution



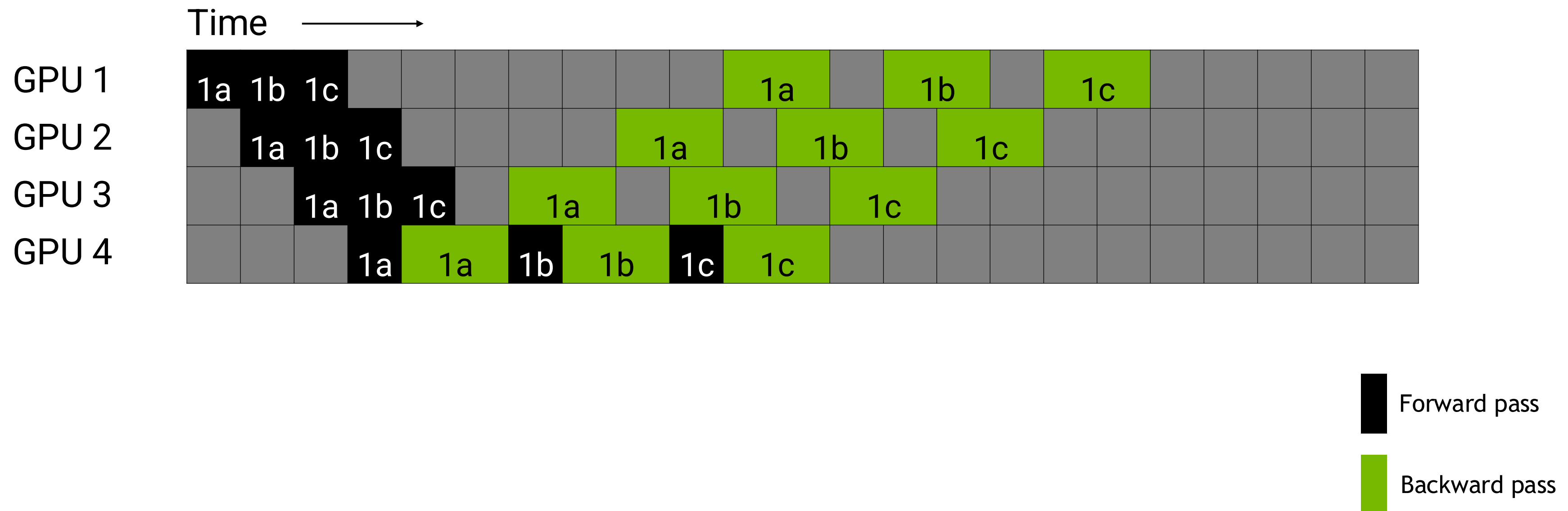
PIPELINE PARALLELISM

Split batch into micro batches and pipeline execution



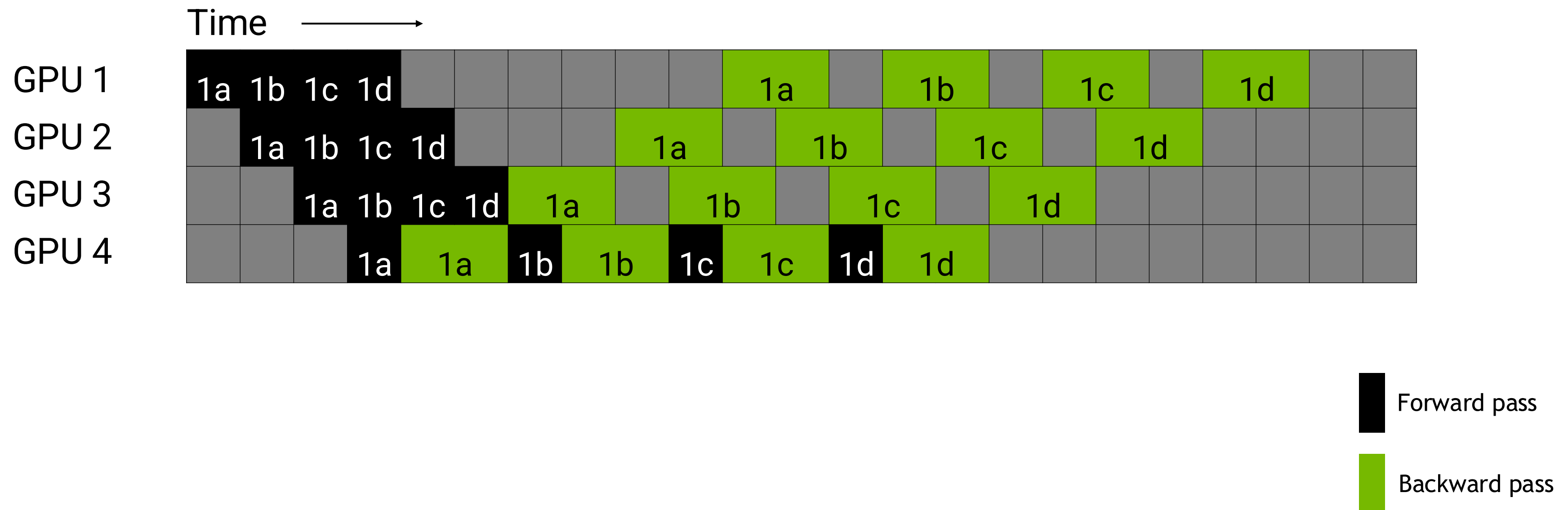
PIPELINE PARALLELISM

Split batch into micro batches and pipeline execution



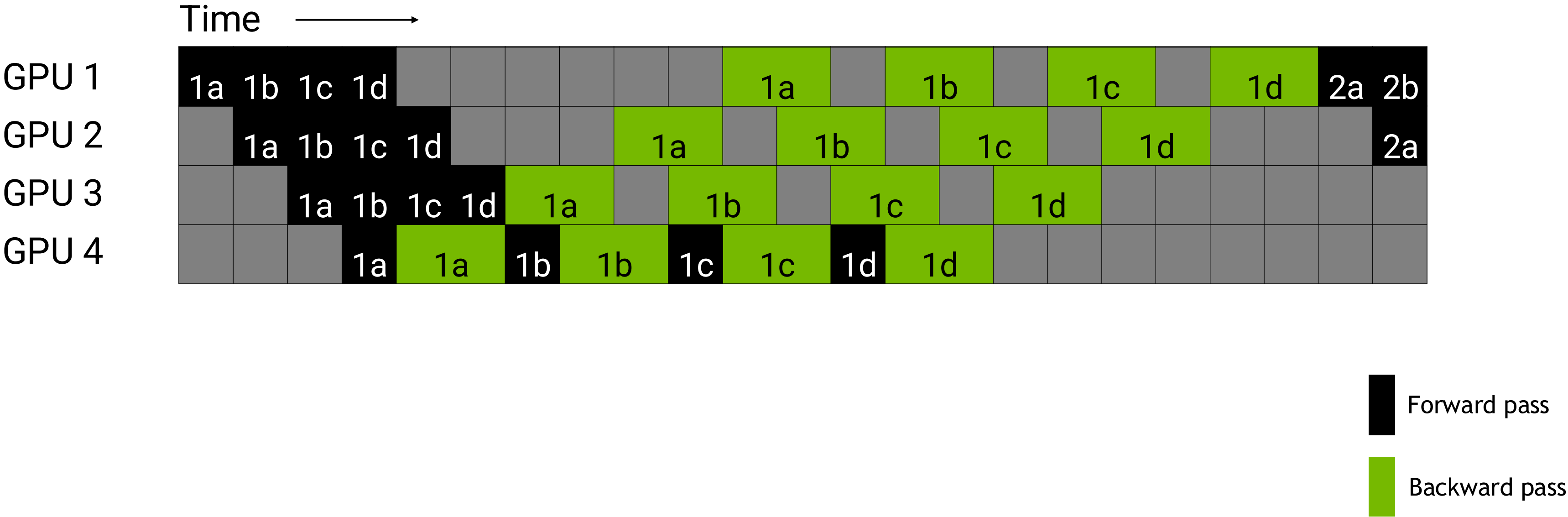
PIPELINE PARALLELISM

Split batch into micro batches and pipeline execution



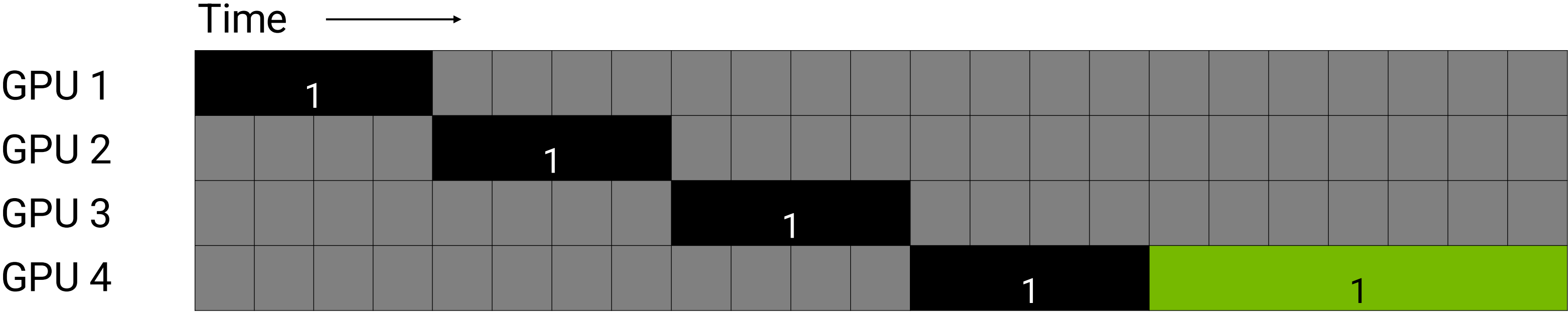
PIPELINE PARALLELISM

Split batch into micro batches and pipeline execution

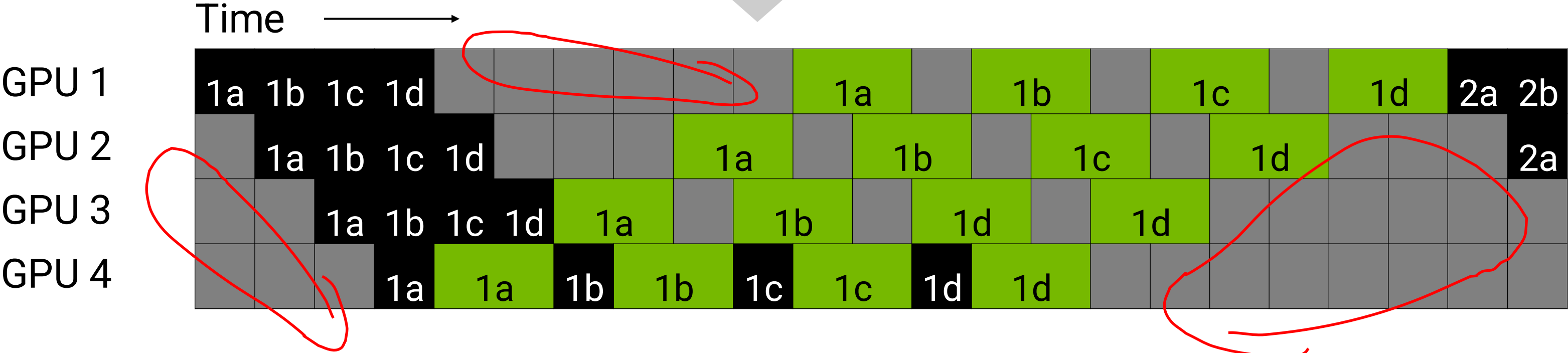


PIPELINE PARALLELISM

Split batch into micro batches and pipeline execution



Split batch into micro batches and pipeline execution

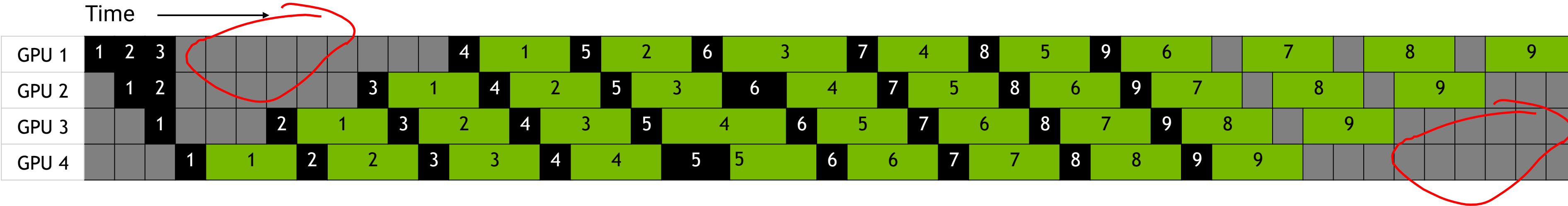


Forward pass

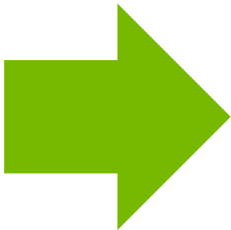
Backward pass

PIPELINE PARALLELISM

Split batch into micro batches and pipeline execution



total time = $(m + p - 1) \times (t_f + t_b)$
ideal time = $m \times (t_f + t_b)$
bubble time = $(p - 1) \times (t_f + t_b)$



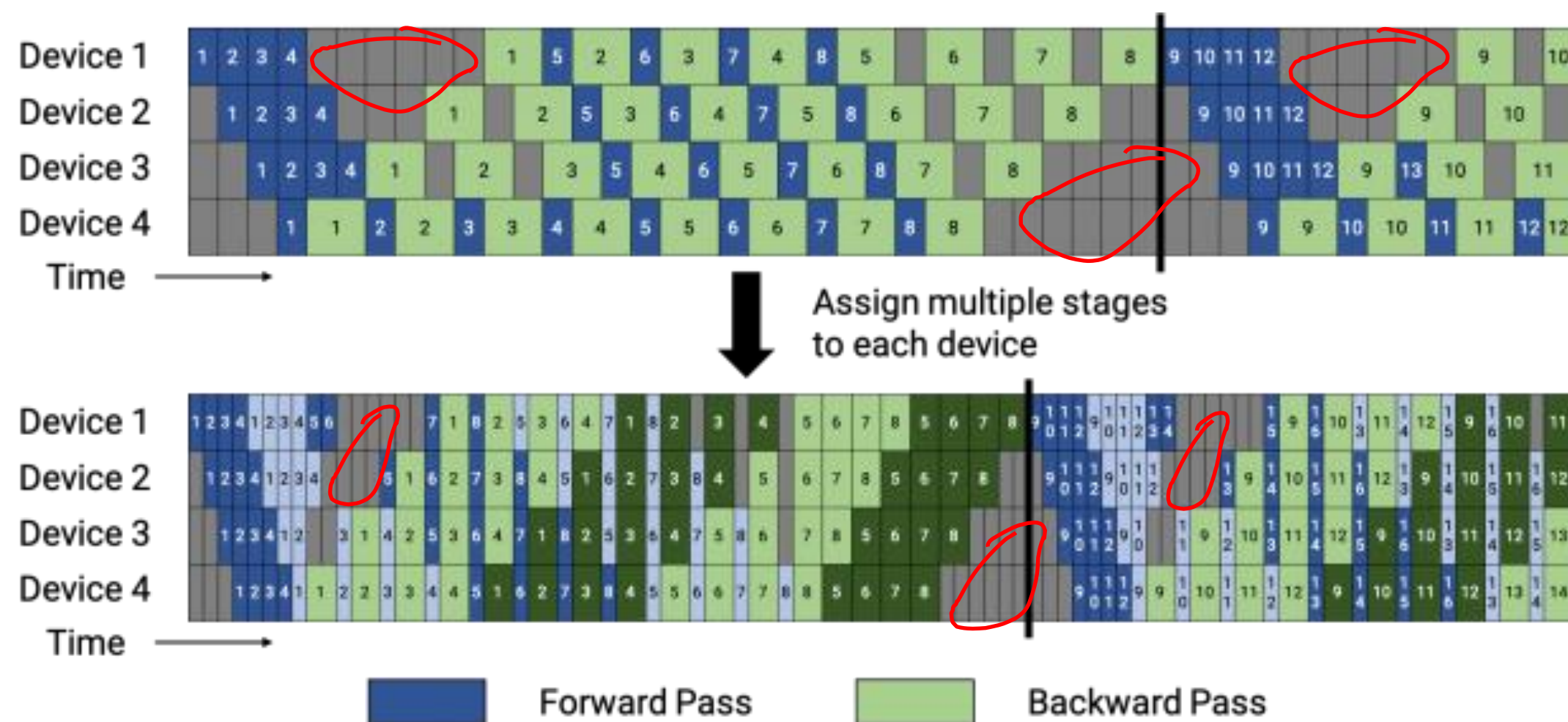
bubble time overhead = $\frac{\text{bubble time}}{\text{ideal time}} = \frac{p - 1}{m}$

p : number of pipeline stages
 m : number of micro batches
 t_f : forward step time
 t_b : backward step time

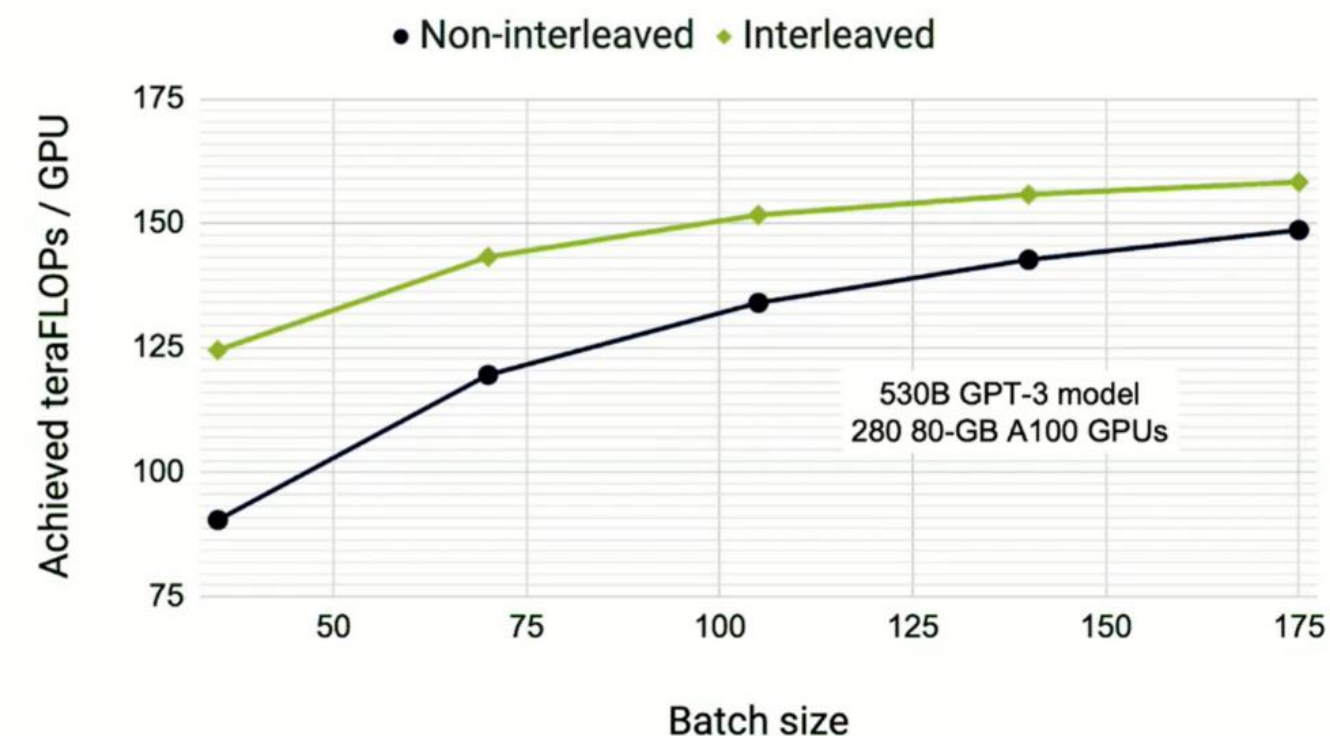
Forward pass
Backward pass

PIPELINE PARALLELISM

Interleaved Pipeline

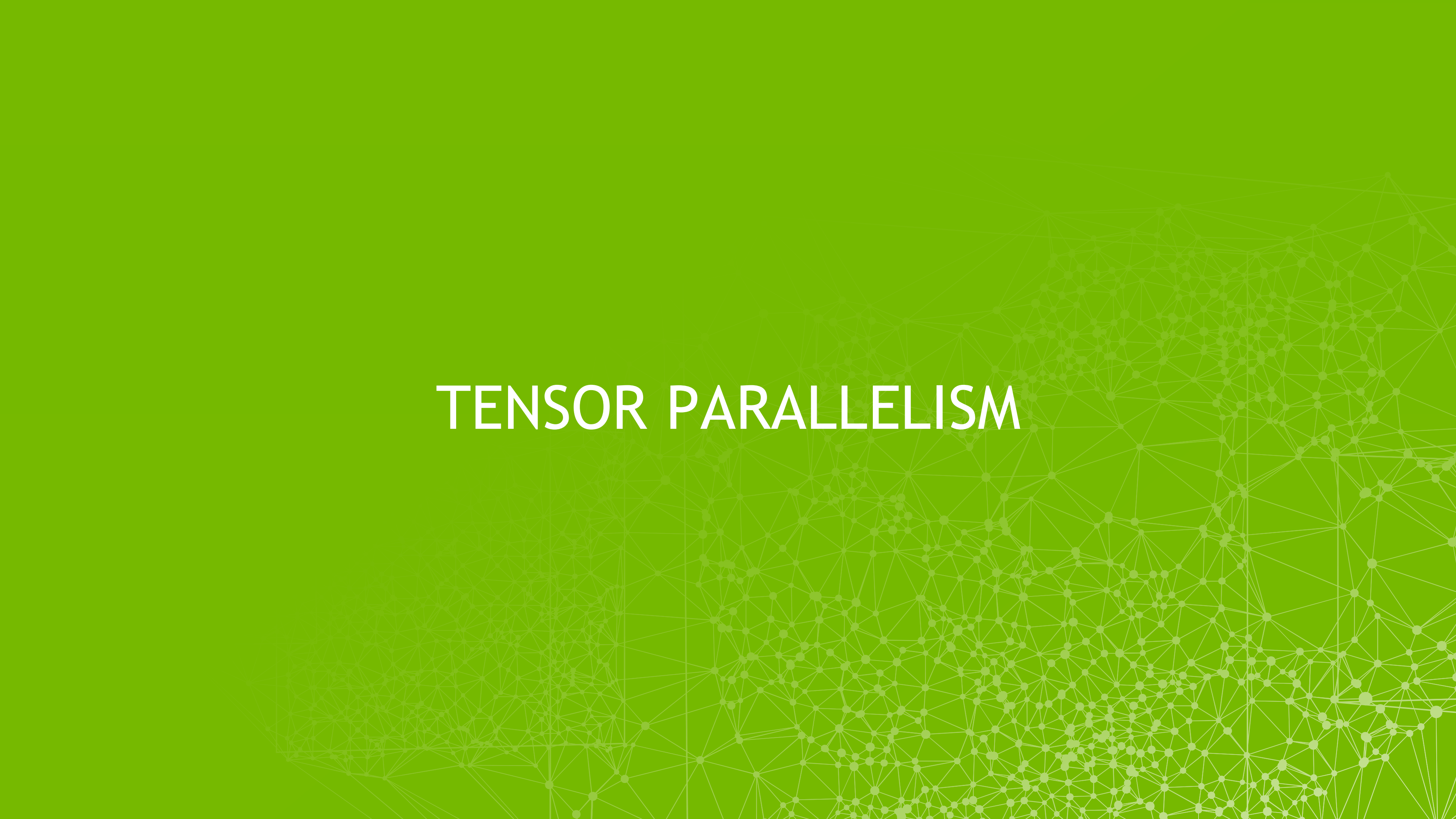


Each gpu has groups of layers that are not continuous across groups, reducing bubbles



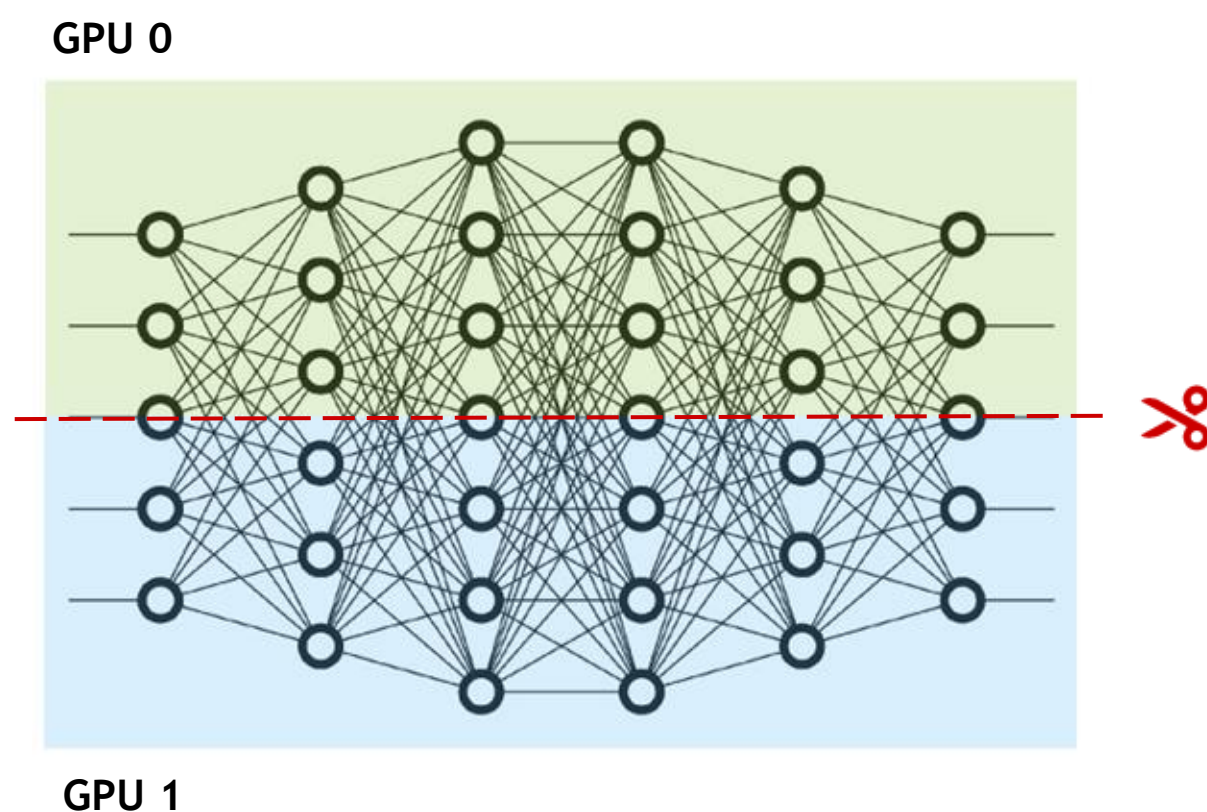
ous set of layers. For example, if each device had 4 layers before (i.e., device 1 had layers 1 – 4, device 2 had layers 5 – 8, and so on), we could have each device perform computation for two model chunks (each with 2 layers), i.e., device 1 has layers 1, 2, 9, 10; device 2 has layers 3, 4, 11, 12; and so on. With this scheme, each device in the pipeline is assigned multiple pipeline stages (each pipeline stage has less computation compared to before).

TENSOR PARALLELISM



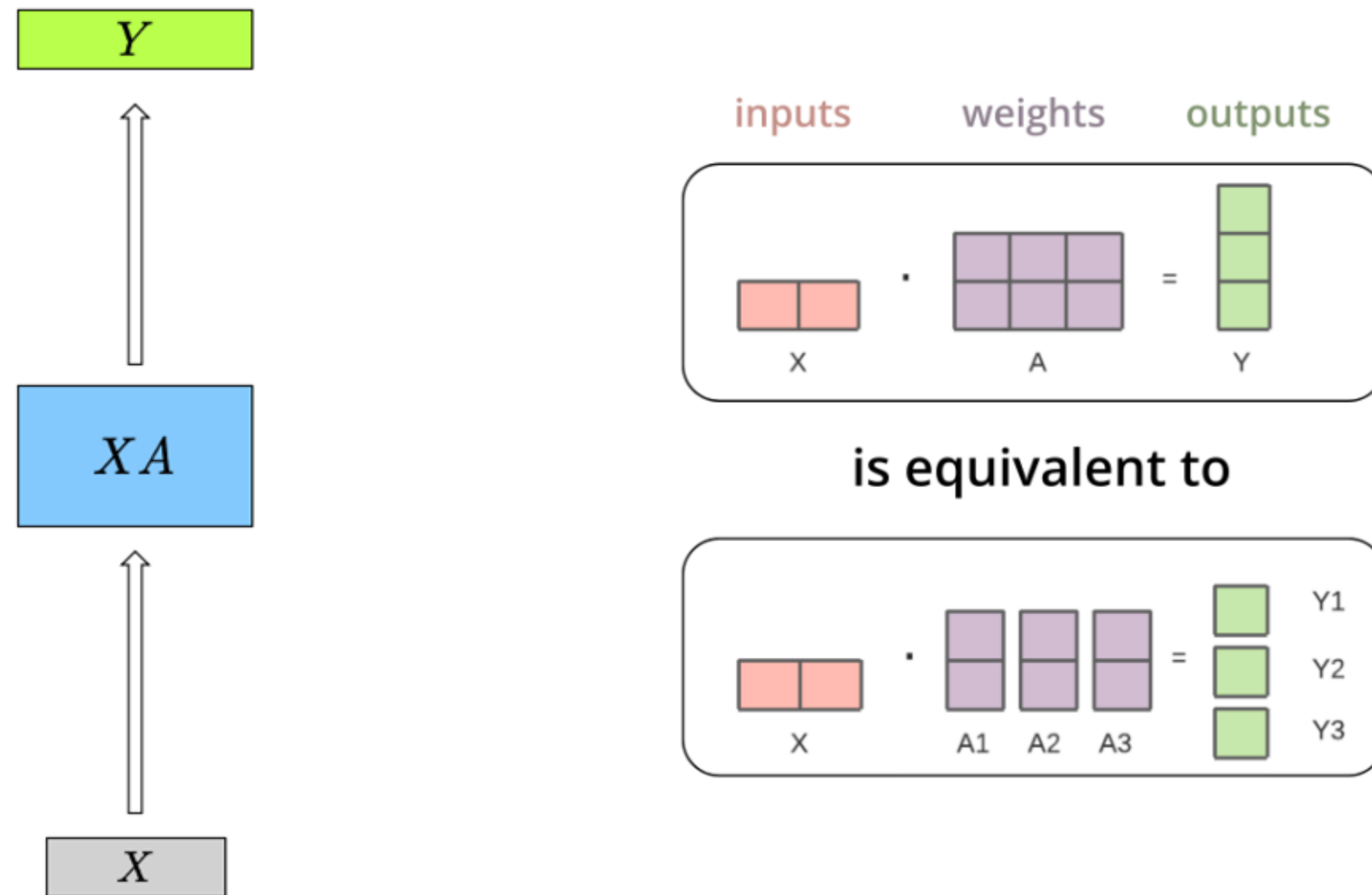
TENSOR PARALLELISM

Why?



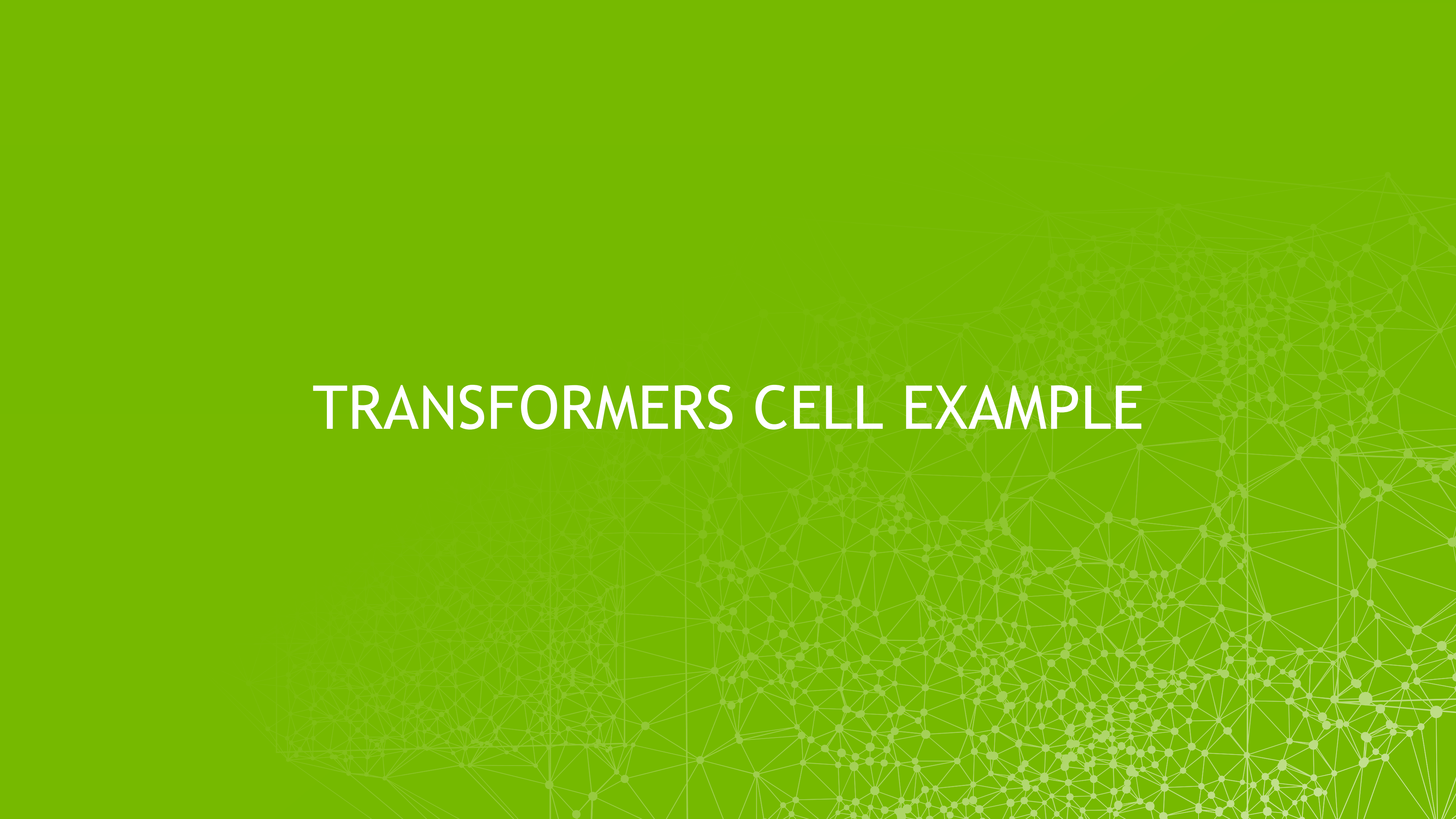
- Use to scale beyond data parallelism
- Typically used when model can't fit on a single GPU
- Reduces memory proportional to the number of workers (model dependent)
- Requires high compute-bandwidth to overcome communication overhead
 - NVIDIA DGX servers with NVSwitch (600GB/sec on DGX A100)
- Easier to load-balance (used to reduce latency for inference)
- Less restrictive on the batch-size (avoids bubble issue in pipelining)
- Tensor parallelism works well for large matrices (e.g. Transformers)
- Doesn't scale well beyond node boundary

SIMPLE EXAMPLE OF TENSOR PARALLELISM

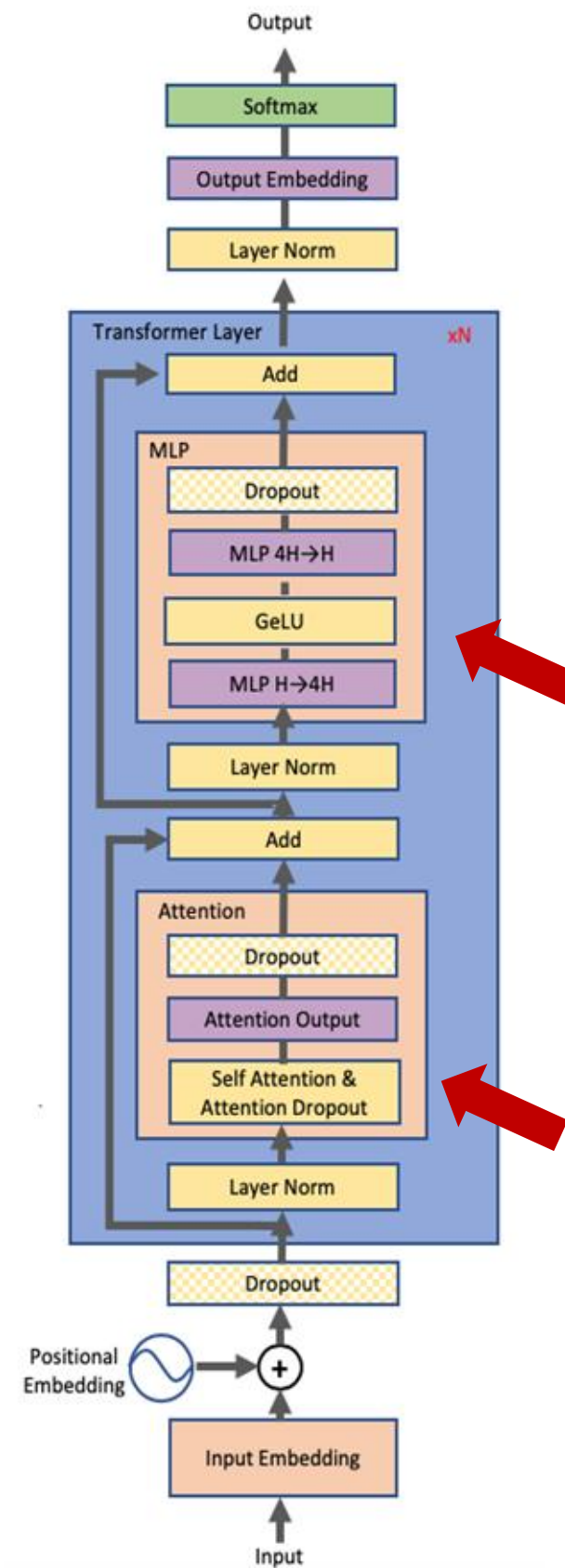


Split the XA matrix multiplications along the columns of A , column parallel split

TRANSFORMERS CELL EXAMPLE

The background of the slide features a smooth gradient from a deep green on the left to a bright yellow on the right. Overlaid on this gradient is a complex, abstract network of small white dots connected by thin white lines, resembling a neural network or a data visualization. The density of the network increases towards the right side of the slide.

TRANSFORMERS CELL



MLP TENSOR PARTITIONING

Focus on the GeLU operation:

- Approach 1: Split X column-wise and A row-wise:

$$X = [X_1, X_2] \quad A = \begin{bmatrix} A_1 \\ A_2 \end{bmatrix} \quad \longrightarrow \quad Y = \text{GeLU}(X_1 A_1 + X_2 A_2)$$

- Before GeLU we will need a communication point

- Approach 2: Split A column-wise:

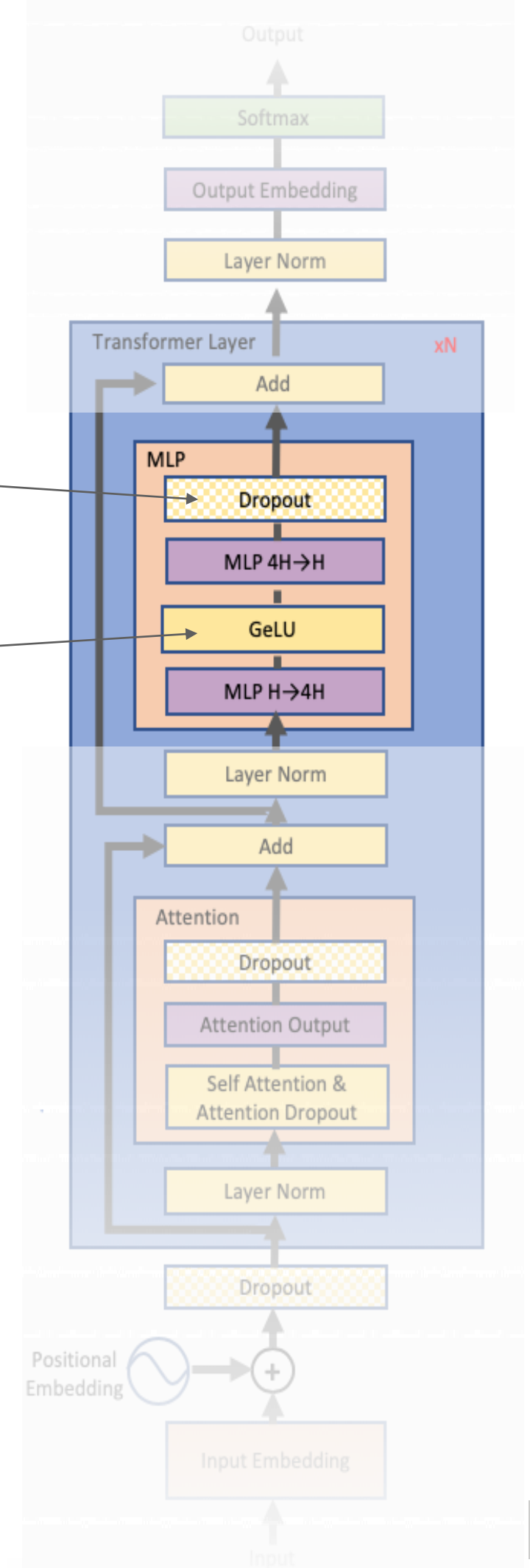
$$A = [A_1, A_2] \quad \longrightarrow \quad [Y_1, Y_2] = [\text{GeLU}(X A_1), \text{GeLU}(X A_2)]$$

- No communication is required

$$Z = \text{Dropout}(Y B)$$

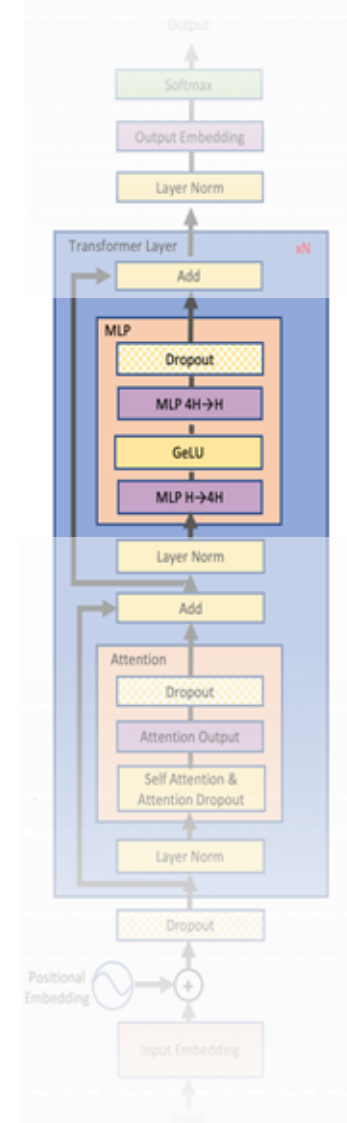
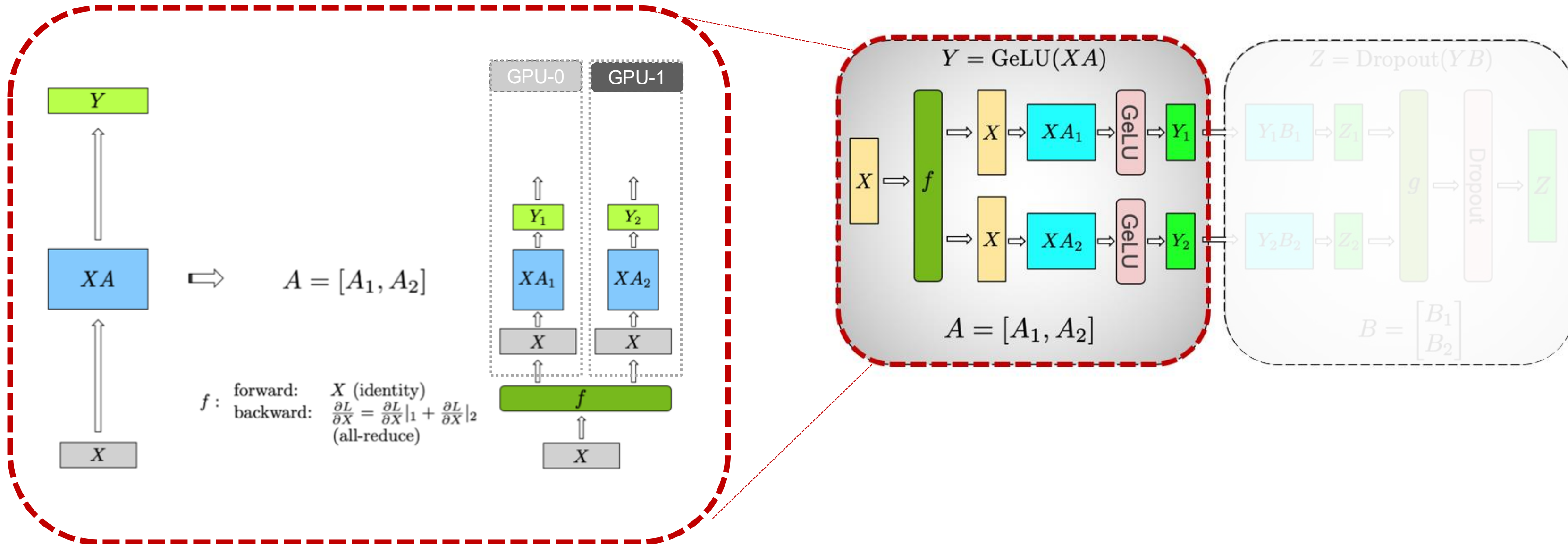
$$Y = \text{GeLU}(X A)$$

Chosen approach



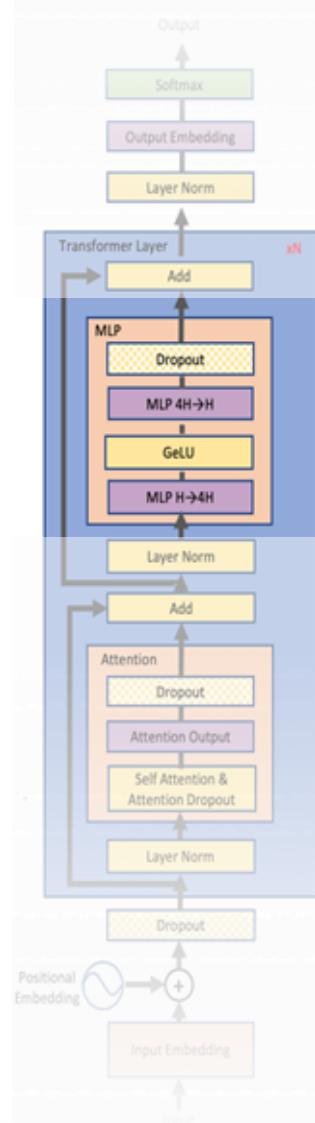
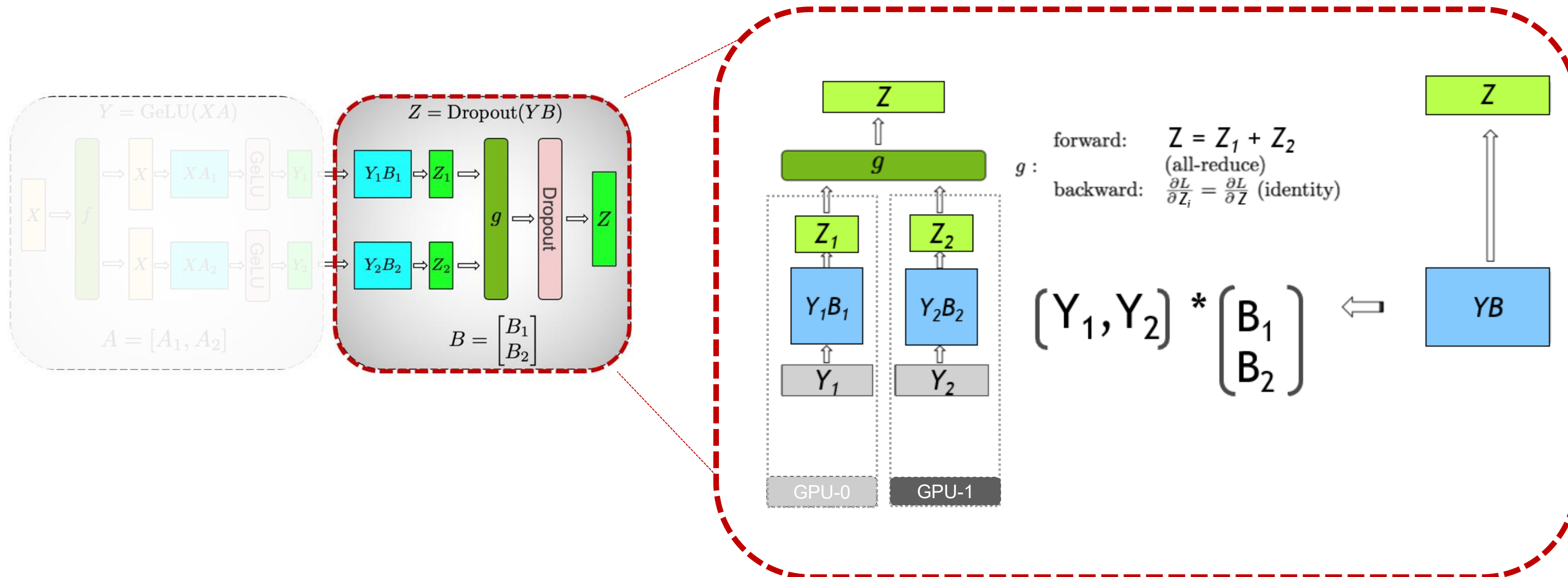
MLP TENSOR PARTITIONING

GeLU Column Parallel



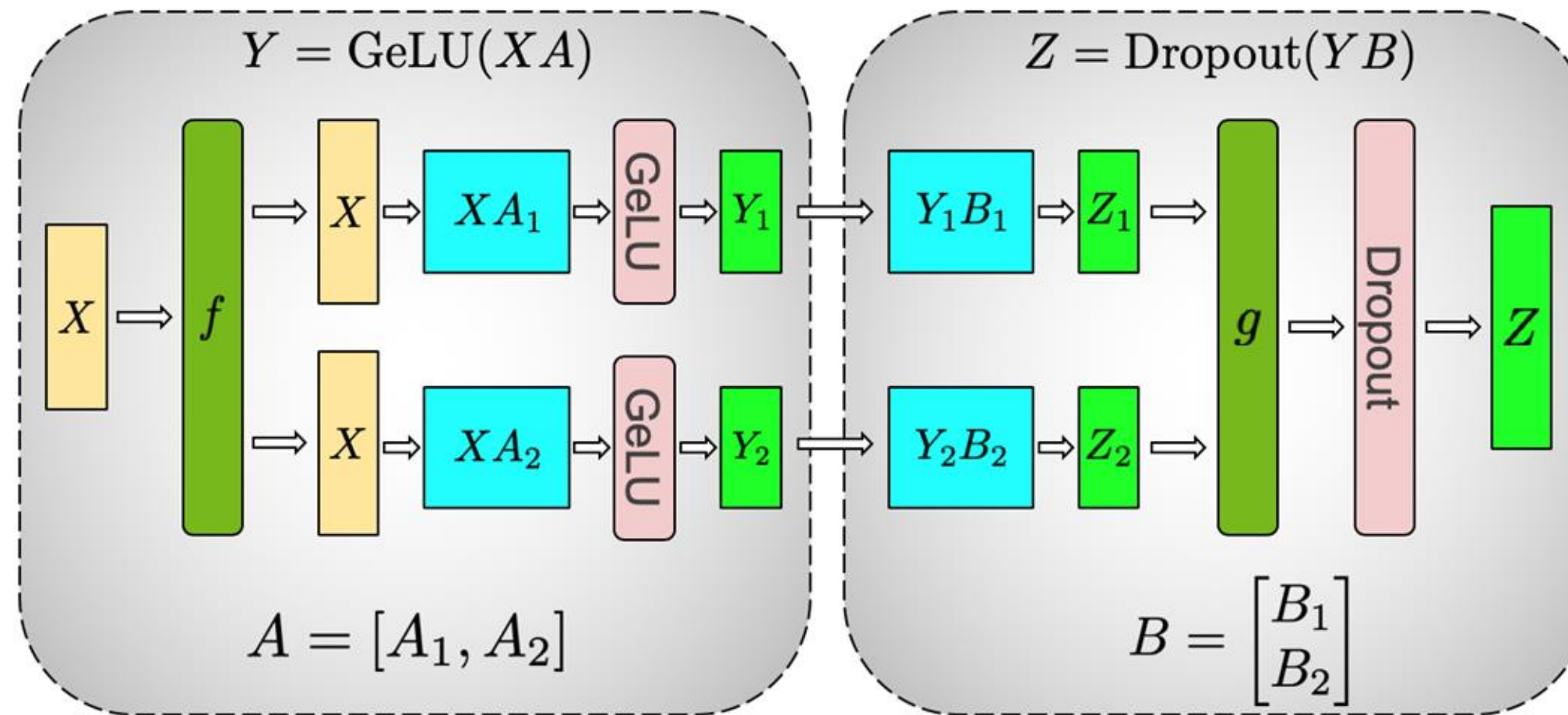
MLP TENSOR PARTITIONING

Dropout Row Parallel

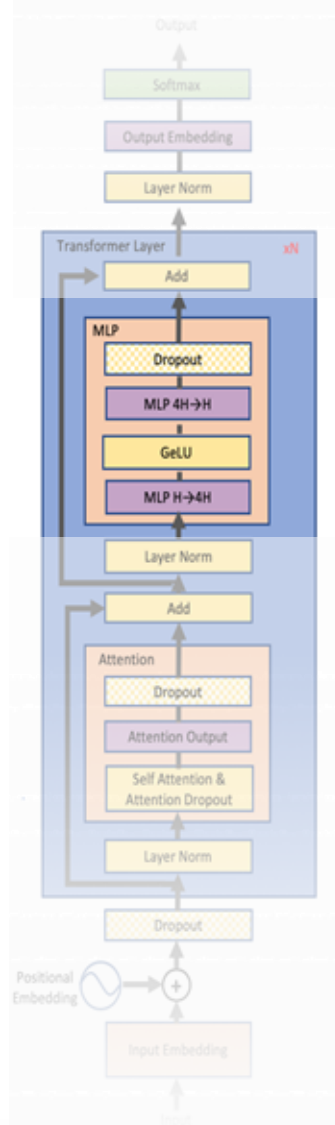


MLP TENSOR PARTITIONING

GeLU Column Parallel

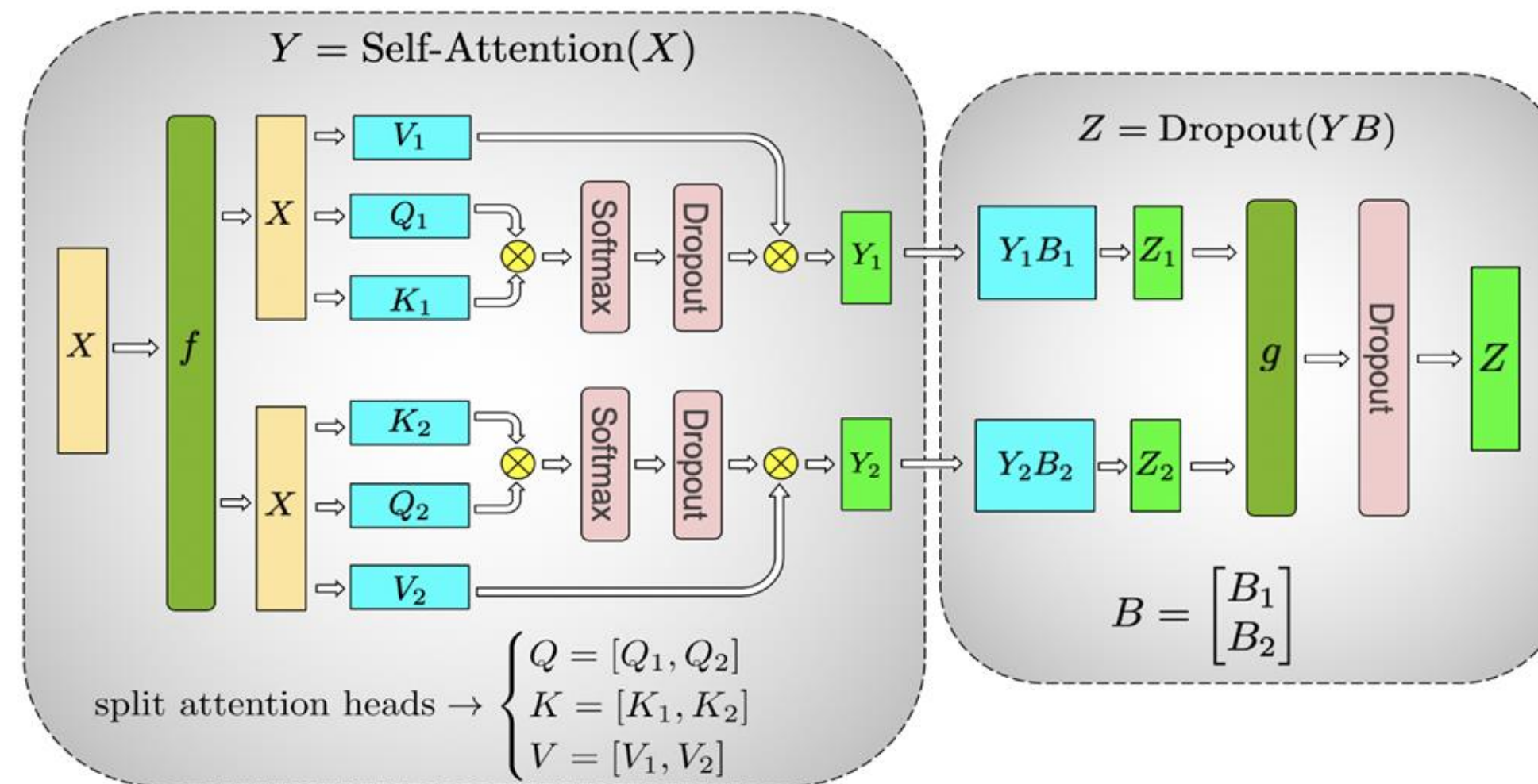


f and g are conjugate, f is identity operator in the forward pass and all-reduce in the backward pass while g is all-reduce in forward and identity in backward.

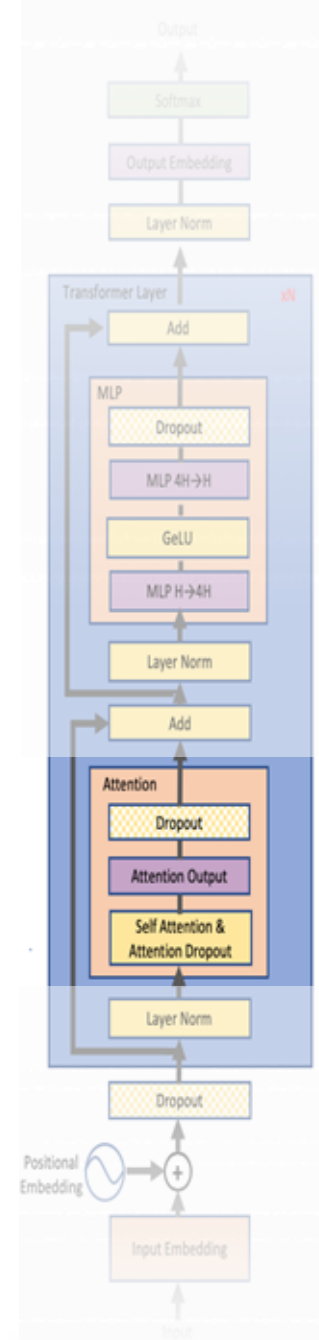


SELF-ATTENTION TENSOR PARTITIONING

Same mechanism applies

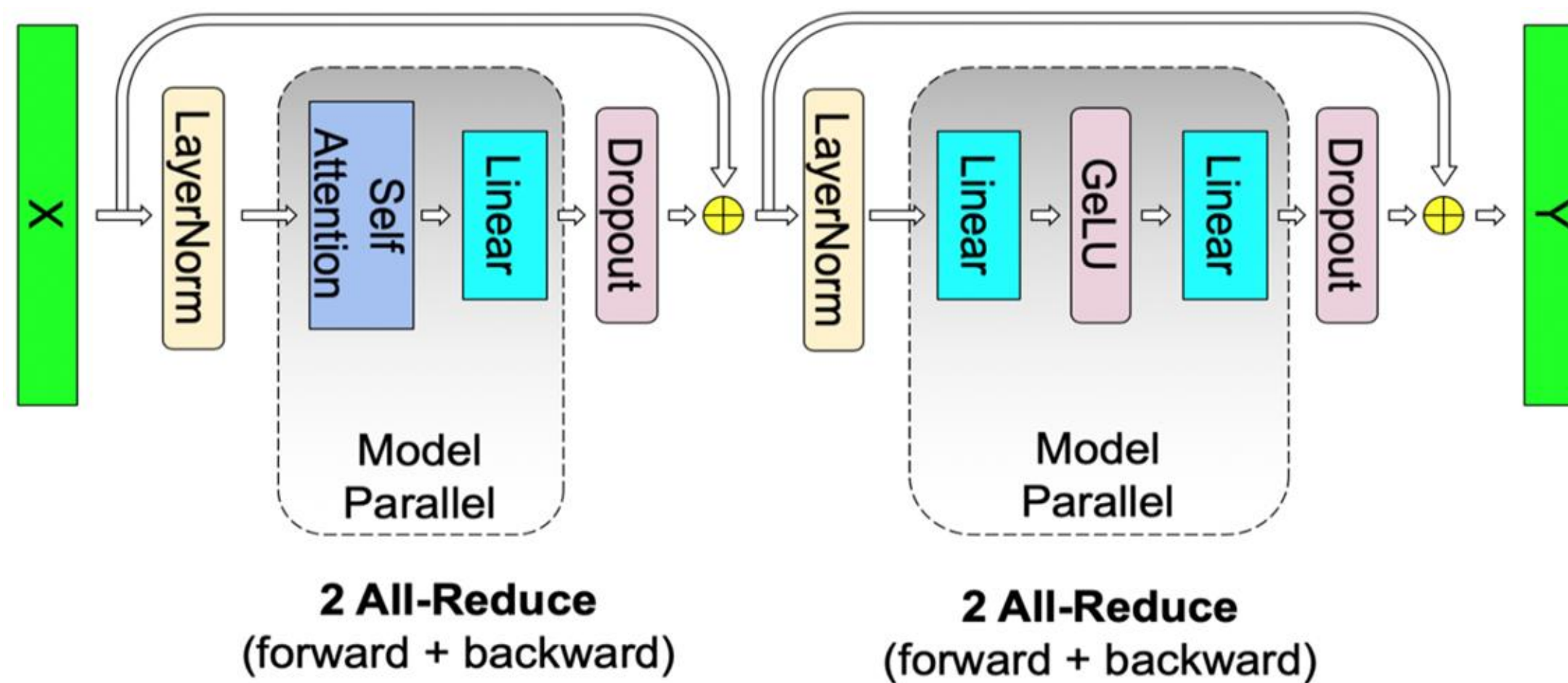


f and g are **conjugate**, f is **identity** operator in the forward pass and **all-reduce** in the backward pass while g is **all-reduce** in forward and **identity** in backward.



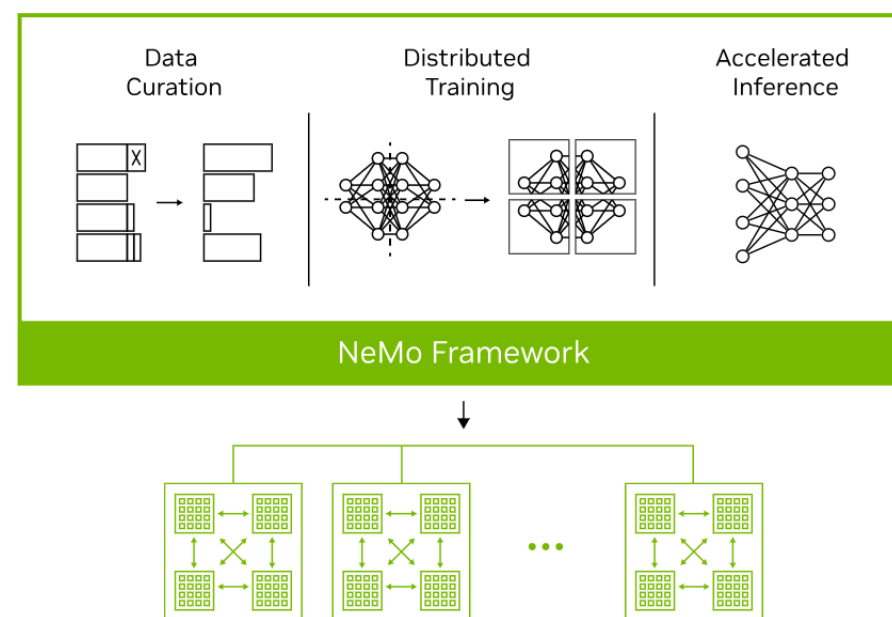
TENSOR PARALLEL TRANSFORMER LAYER

All Together



TENSOR PARALLEL IMPLEMENTATIONS

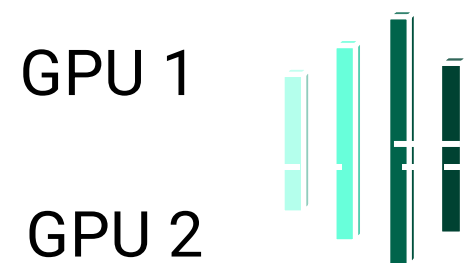
Libraries examples



PIPELINE AND TENSOR PARALLELISM

Recap

Tensor Parallelism



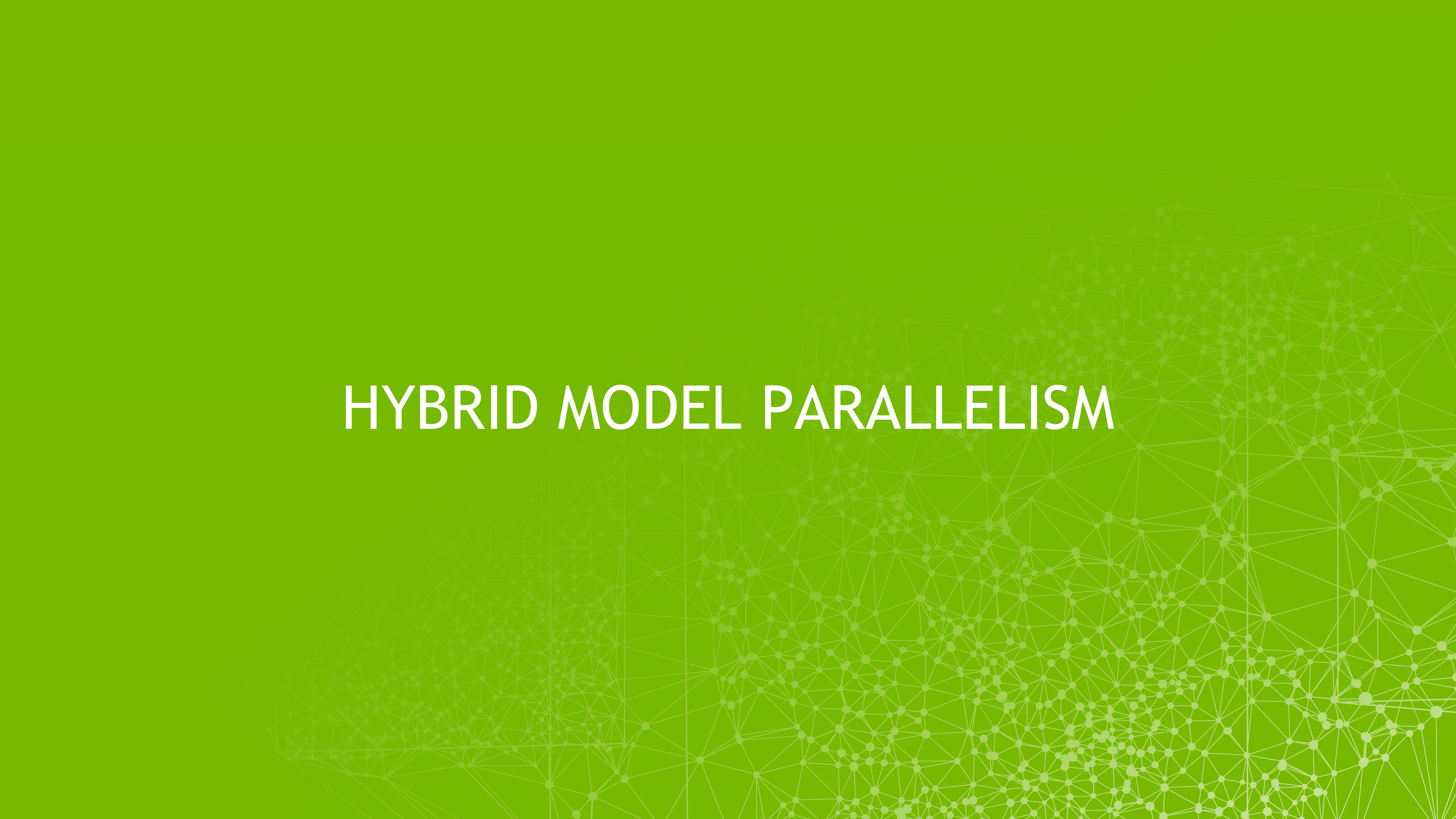
- Split individual layers across multiple GPUs where all devices compute different parts of Layers
- Challenge: Communication expensive
- Great performance within a server using NVSwitch
- Limitations: Limited number of Model Architectures | GPT-3 & T5

Pipeline Parallelism



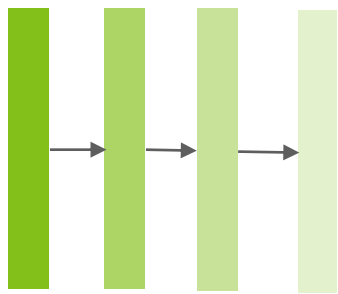
- Split contiguous groups of layers across multiple GPUs so that Layers 0,1,2 and layers 3,4,5 are on different GPUs ...
- Communication cheap, maximizes GPU utilization over InfiniBand
- Good performance at larger batch sizes (pipeline stall amortized)
- Exceptions/Limitations: No Interleave Scheduling for Pipeline parallelism

HYBRID MODEL PARALLELISM

The background of the slide features a smooth gradient from a deep blue on the left to a vibrant green on the right. Overlaid on this gradient is a complex, abstract network of white dots and thin white lines, resembling a neural network or a data mesh. The dots are of varying sizes and are interconnected by a web of lines that create a sense of depth and connectivity across the entire frame.

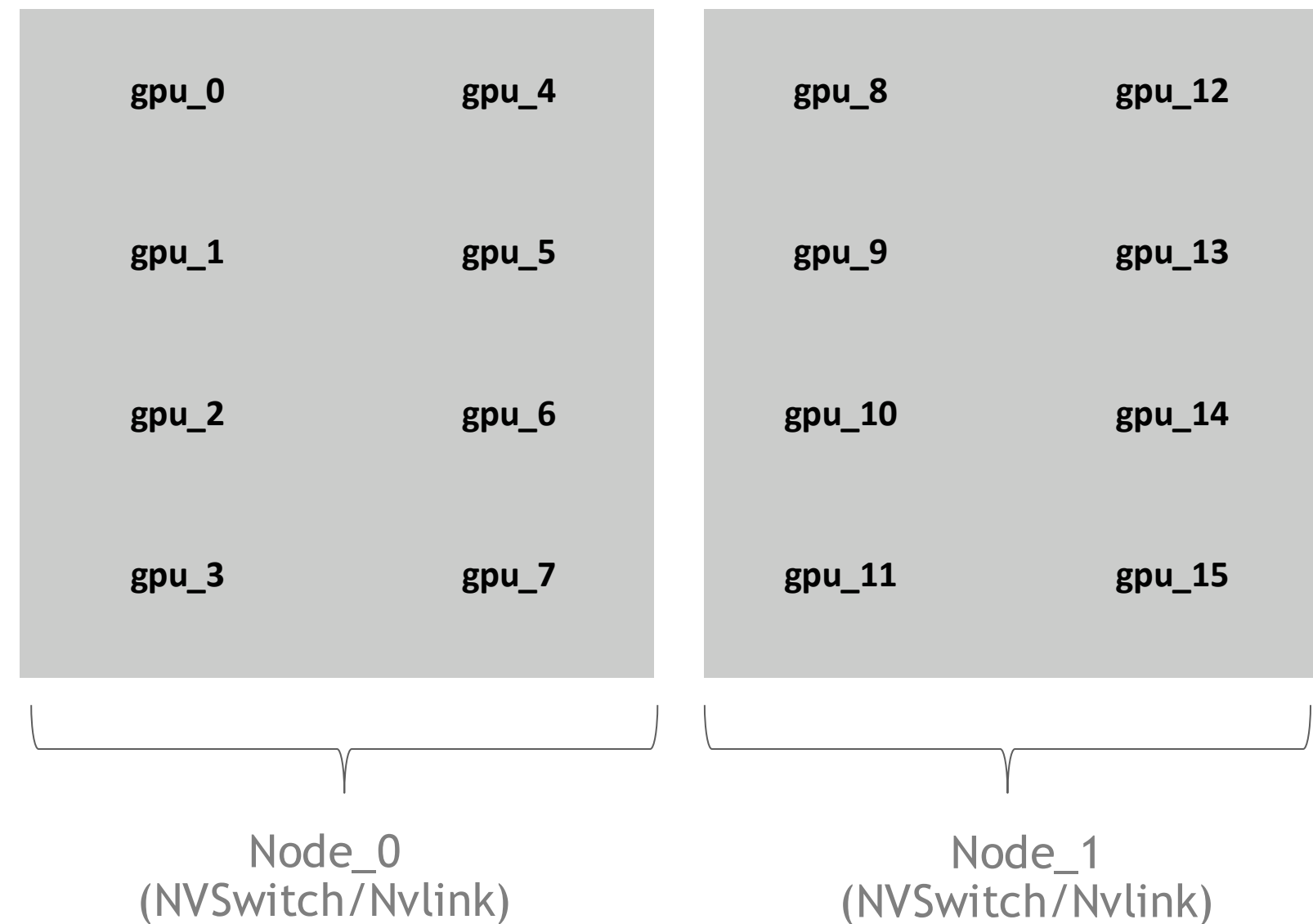
NEMO MODEL PARALLELISM

GPU Affinity Grouping Example



Neural Network: 4 layers
Hardware: 2 nodes , 8 GPUs per node

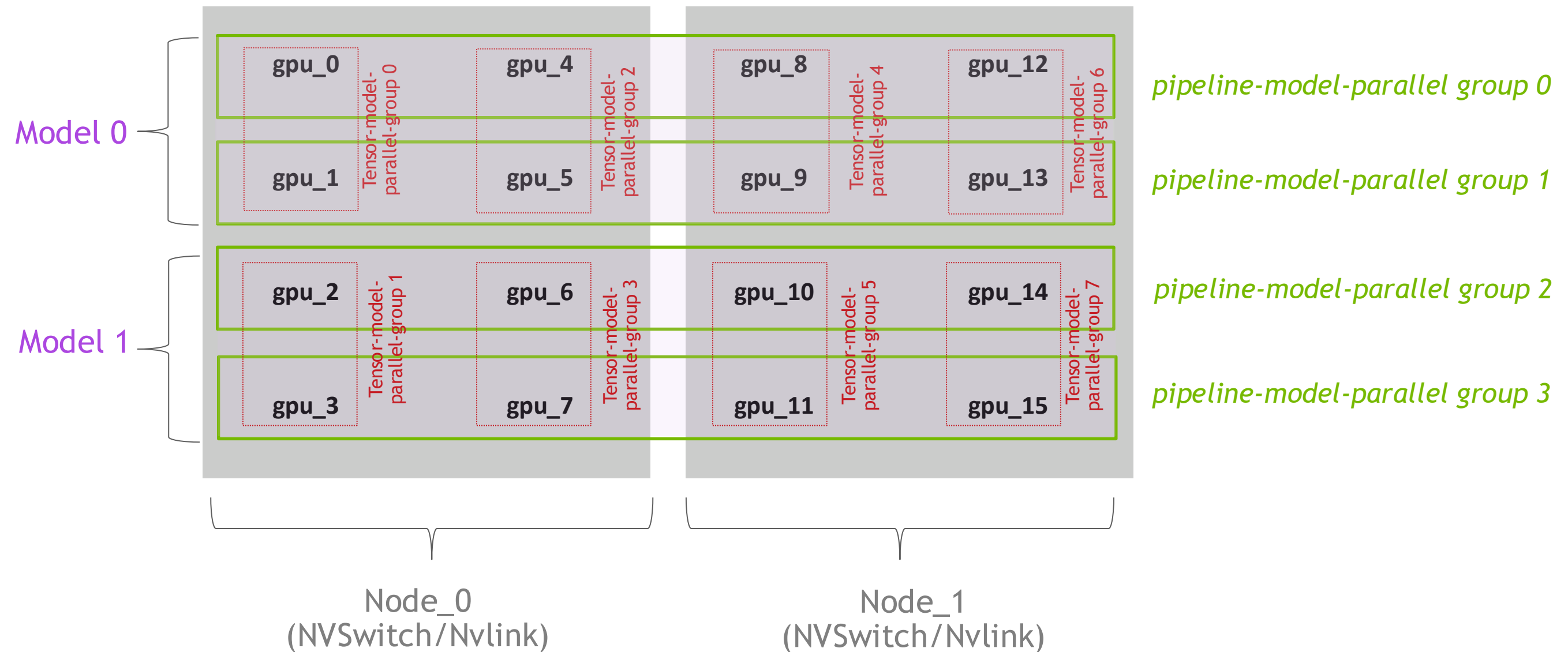
- Tensor parallel = 2
- Pipeline parallel = 4
- Data parallel = 2



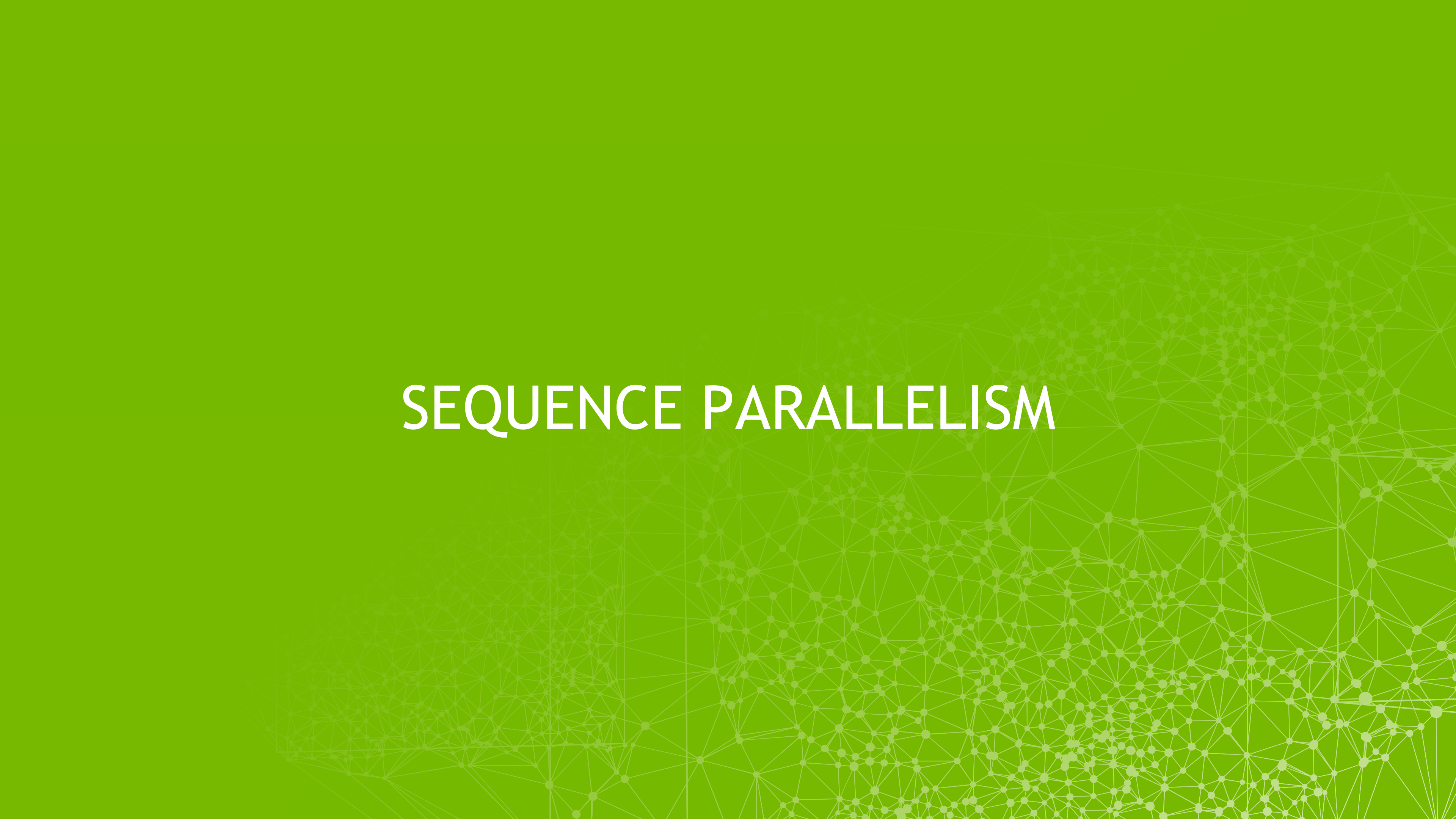
NEMO MODEL PARALLELISM

GPU Affinity Grouping Example

- 2 nodes , 8 GPUs per node
- Tensor parallel = 2
 - Pipeline parallel = 4
 - Data parallel = 2

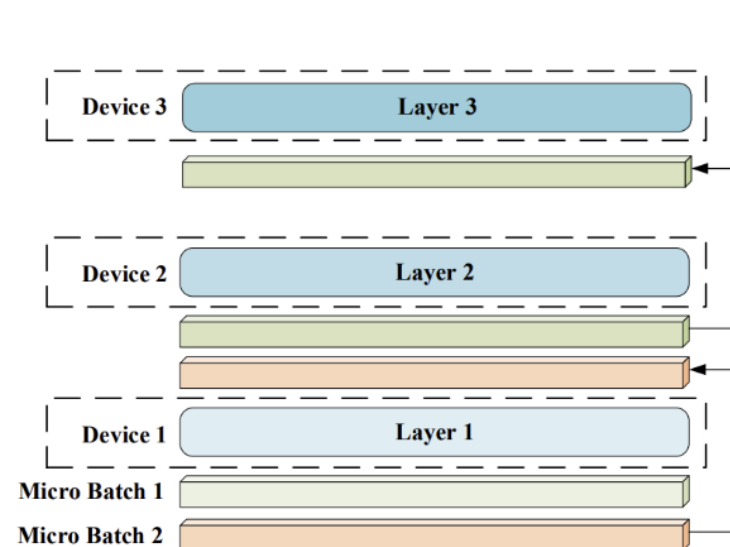


SEQUENCE PARALLELISM

The background of the slide features a smooth gradient from a deep green at the top to a bright yellow at the bottom. Overlaid on this gradient is a complex, abstract network of thin white lines connecting numerous small white dots, creating a mesh-like or molecular structure that spans the entire width of the image.

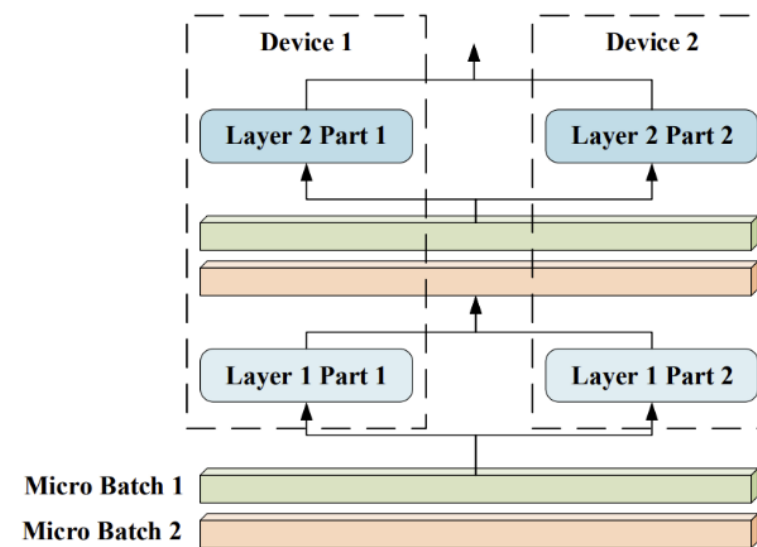
DEALING WITH MEMORY CONSTRAINTS

Sequence Parallelism - Train with longer sequences



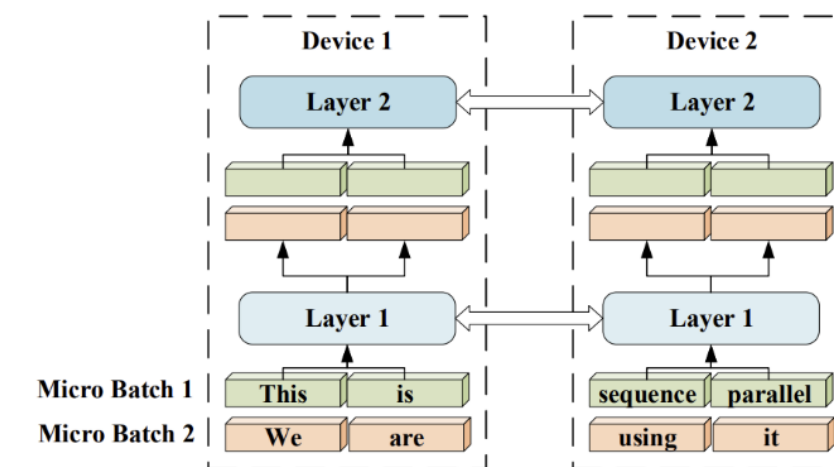
(a) Pipeline parallelism

- In Pipeline parallelism, groups of layers are placed on GPUs (sequential execution)
- Whole sequence is needed for training



(b) Tensor parallelism

- In Tensor Parallelism, individual layers are split amongst GPUs
- Whole sequence is needed for training

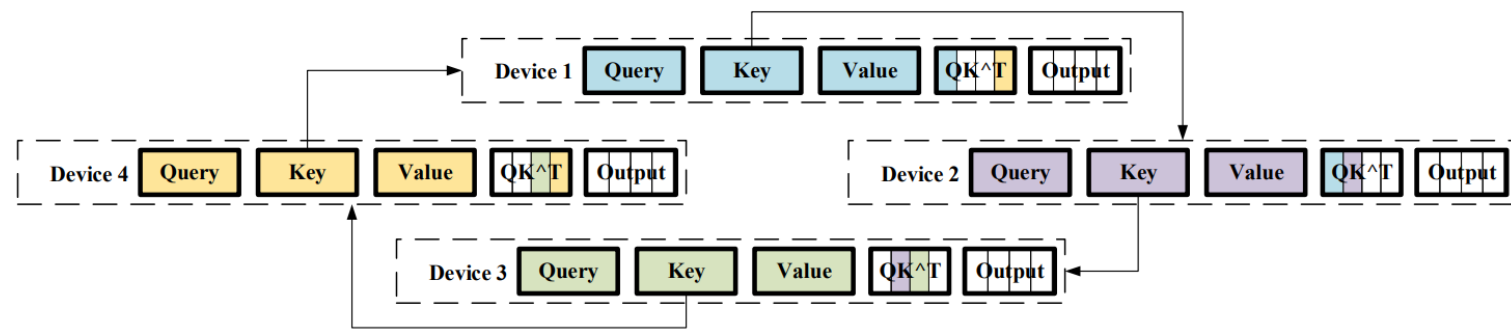


(c) Sequence parallelism (Ours)

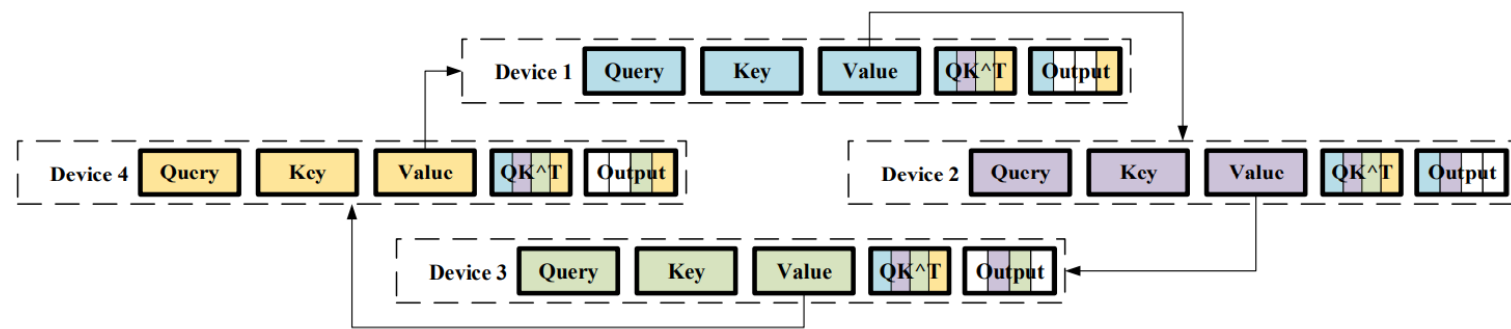
- In Sequence Parallelism we distribute the sequence across gpus
- Devices share the same parameters

RING SELF ATTENTION

Sequence Parallelism - Train with longer sequences



(a) Transmitting key embeddings among devices to calculate attention scores



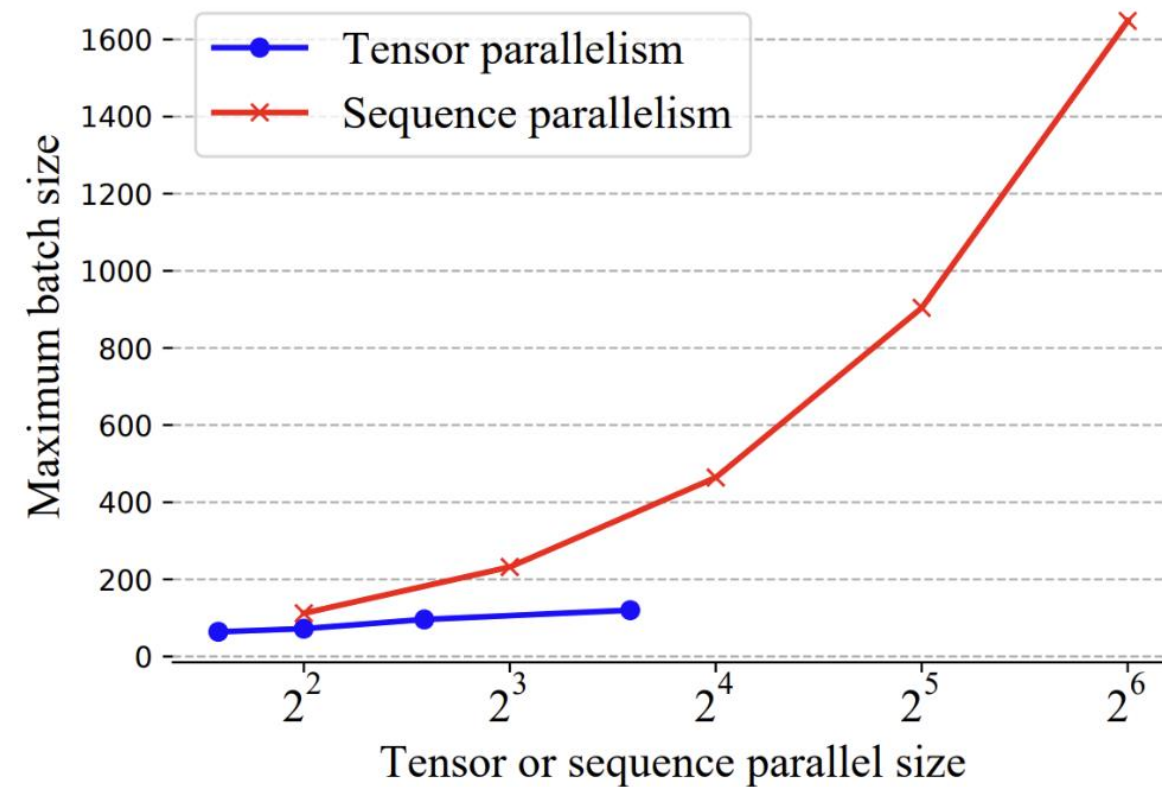
(b) Transmitting value embeddings among devices to calculate the output of attention layers

Figure 2: Ring Self-Attention

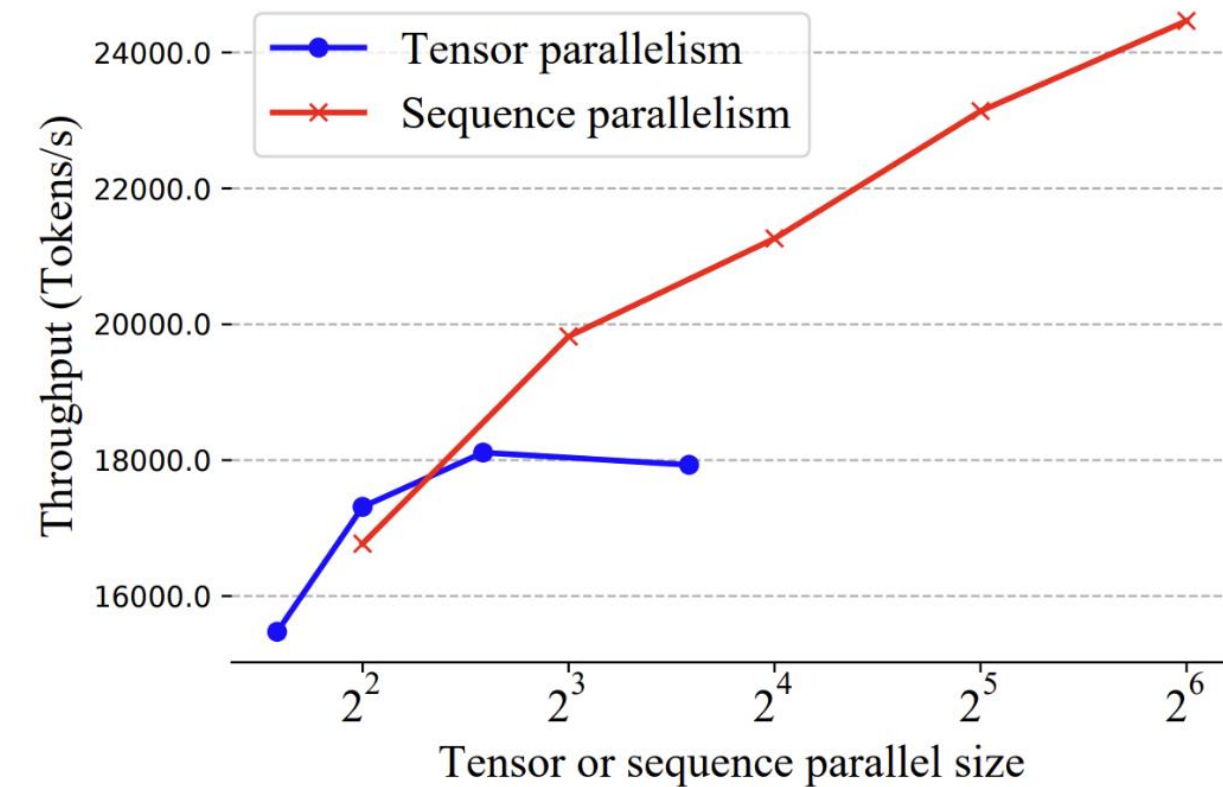
- Main challenge in sequence parallelism is calculating attention scores across devices
- Query, Key and Value (Q,K,V) matrices are split amongst the gpus
- Key embeddings are transferred among the GPUs
 - Query embeddings of each sub-sequence can multiply all key embeddings
- All query embeddings collect their attention scores on each device
- After attention scoring, Value embeddings are transferred among the GPUs to compute-self attention output.
- No communication in the MLP layer just self-attention module

DEALING WITH MEMORY CONSTRAINTS

Sequence Parallelism



(a) Maximum batch size of BERT Base scaling along tensor or sequence parallel size



(b) Throughput of BERT Base scaling along tensor or sequence parallel size

Figure 3: Scaling with sequence/tensor parallelism

Sequence parallelism allows you to get higher batch sizes and throughput during training by increasing the parallel size and same communication overhead as tensor parallelism

DEALING WITH MEMORY CONSTRAINTS

Sequence parallelism

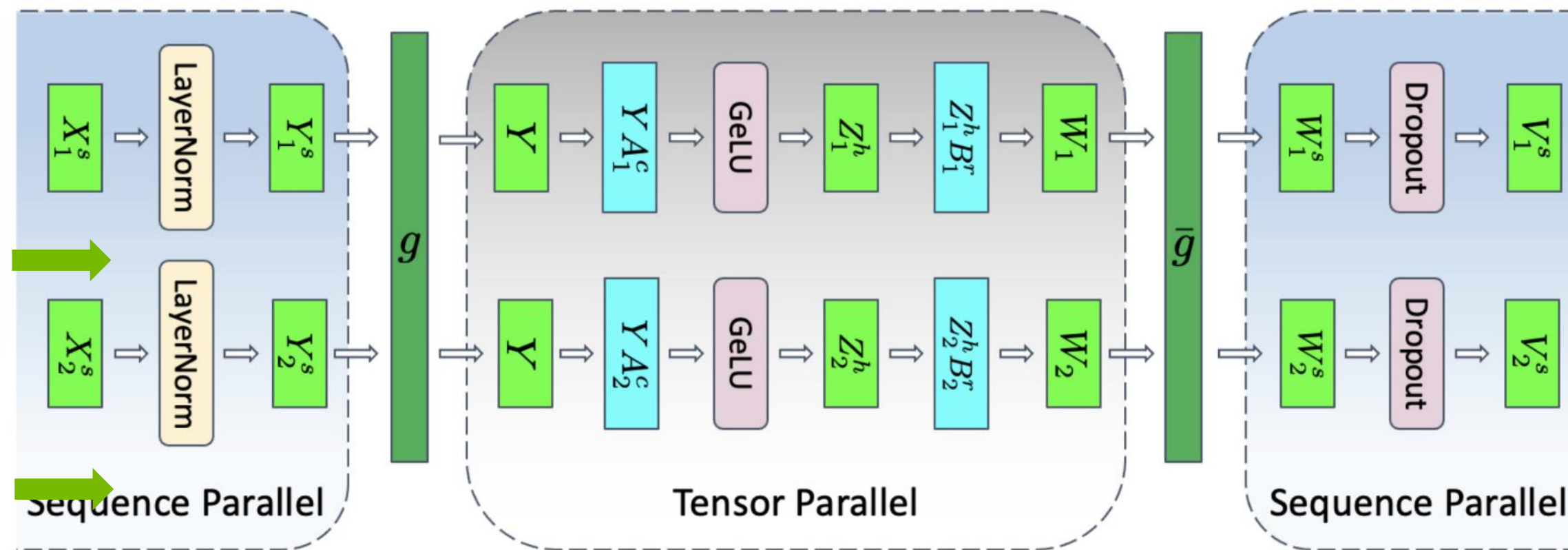


Figure 6: MLP layer with tensor and sequence parallelism. g and $\bar{g}$ are conjugate. g is all-gather in forward pass and reduce-scatter in backward pass. $\bar{g}$ is reduce-scatter in forward pass and all-gather in backward pass.

Tensor + Sequence Parallel has the same communication overhead as Tensor Parallel

ACTIVATIONS CHECKPOINTING

The background of the slide features a smooth gradient from a deep green on the left to a bright yellow on the right. Overlaid on this gradient is a complex, abstract network of thin white lines connecting numerous small, semi-transparent dots. This network pattern is denser on the right side and fades into the background on the left, creating a sense of depth and connectivity.

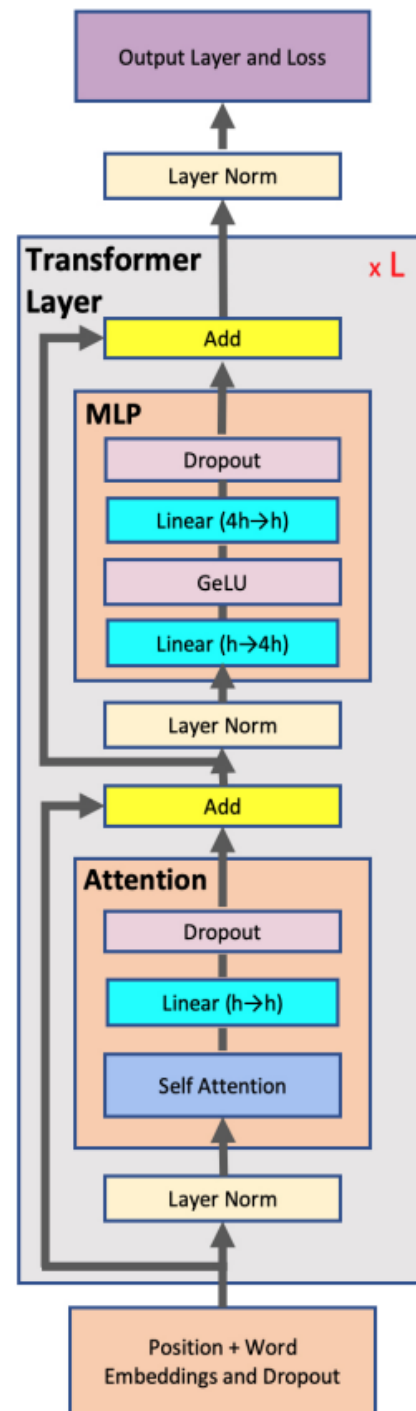
ACTIVATION RECOMPUTE CHALLENGES

Activation Definition

Activation: Any tensor that is created in the forward pass and is necessary for gradient computation in the backward pass

ACTIVATION RECOMPUTE CHALLENGES

Activation Memory Per Transformer Layer



4.1 Activations Memory Per Transformer Layer

As shown in Figure 2, each transformer layer consists of an attention and an MLP block connected with two layer-norms. Below, we derive the memory required to store activations for each of these elements:

Attention block: which includes self attention followed by a linear projection and an attention dropout. The linear projection stores its input activations with size $2sbh$ and the attention dropout requires a mask with size sbh . The self attention shown in Figure 3 consists of several elements:

- **Query (Q), Key (K), and Value (V) matrix multiplies:** We only need to store their shared input with size $2sbh$.
- **QK^T matrix multiply:** It requires storage of both Q and K with total size $4sbh$.
- **Softmax:** Softmax output with size $2as^2b$ is required for back-propagation.
- **Softmax dropout:** Only a mask with size as^2b is needed.
- **Attention over Values (V):** We need to store the dropout output ($2as^2b$) and the Values ($2sbh$) and therefore need $2as^2b + 2sbh$ of storage.

Summing the above values, in total, the attention block requires $11sbh + 5as^2b$ bytes of storage.

MLP: The two linear layers store their inputs with size $2sbh$ and $8sbh$. The GeLU non-linearity also needs its input with size $8sbh$ for back-propagation. Finally, dropout stores its mask with size sbh . In total, MLP block requires $19sbh$ bytes of storage.

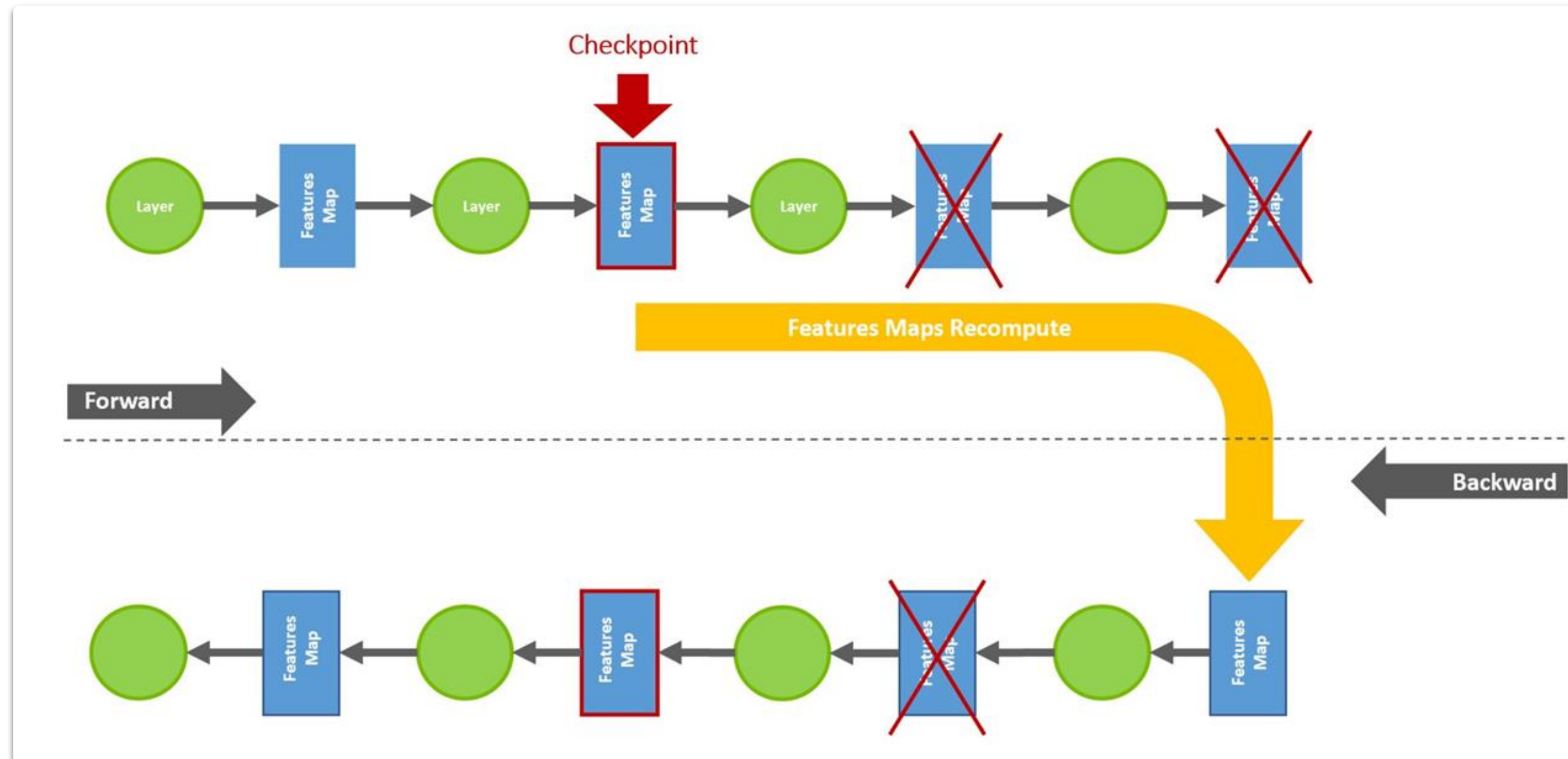
Layer norm: Each layer norm stores its input with size $2sbh$ and therefore in total, we will need $4sbh$ of storage.

Summing the memory required for attention, MLP, and the layer-norms, the memory required to store the activations for a single layer of a transformer network is:

$$\text{Activations memory per layer} = sbh \left(34 + 5 \frac{as}{h} \right). \quad (1)$$

ACTIVATION RECOMPUTE CHALLENGES

Activation Recompute



Balance the memory savings and computational overhead?

ACTIVATION RECOMPUTE CHALLENGES

Selective Activation Recompute with Megatron-LM

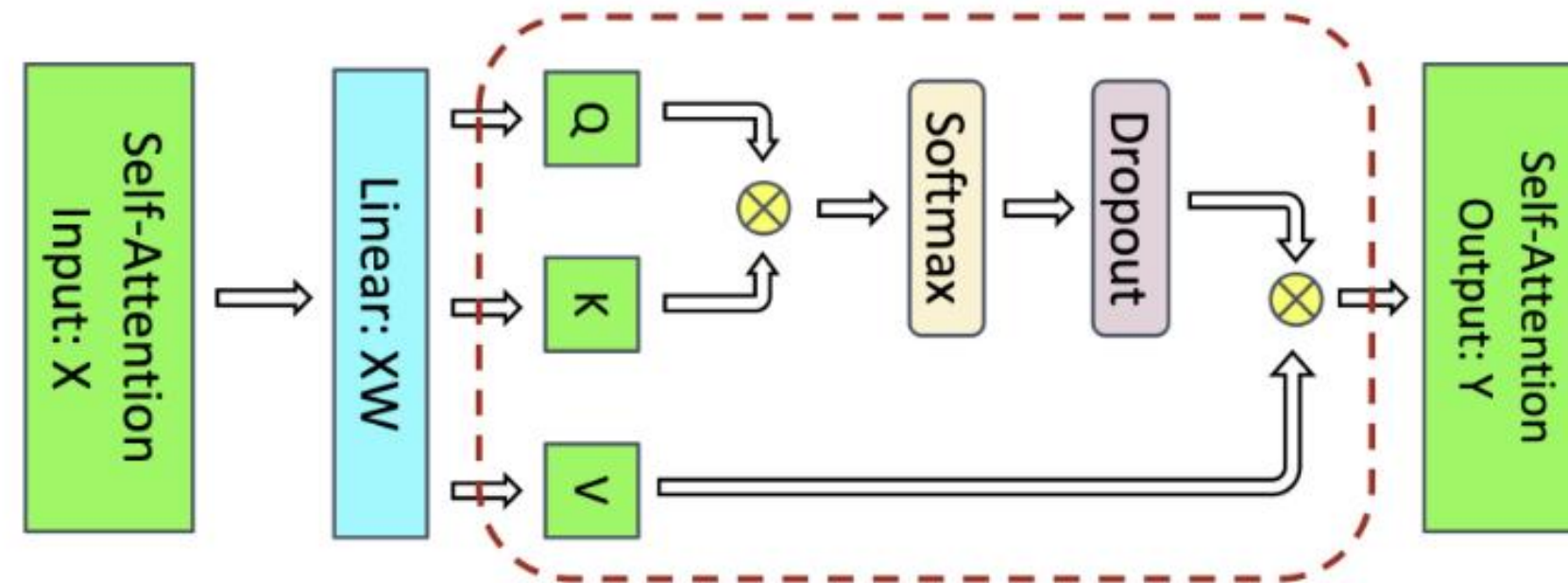


Figure 3: Self-attention block. The red dashed line shows the regions to which selective activation recompute is applied (see Section 5 for more details on selective activation recompute).

- Saves the activations that take less space and are expensive to recompute
- Recompute activations that take a lot of space but are relatively cheap to recompute.

DEALING WITH MEMORY CONSTRAINTS

Megatron Sequence parallelism + Selective Checkpointing

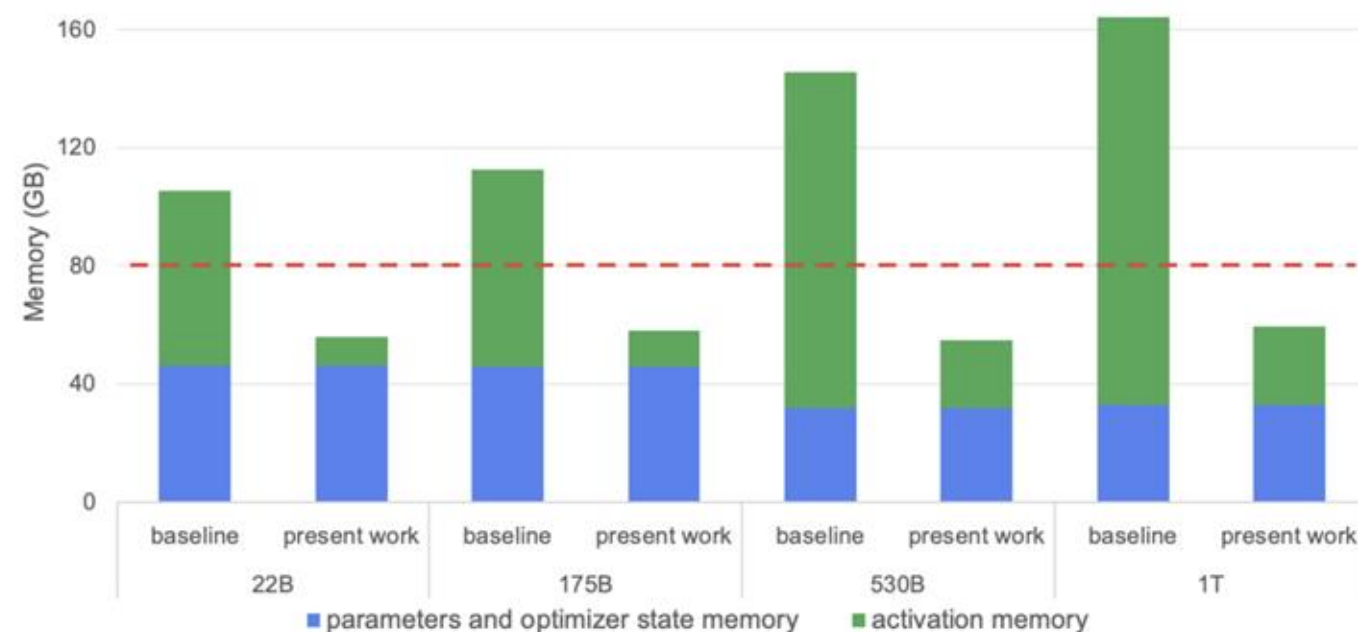


Figure 1: Parameters, optimizer state, and activations memory. The dashed red line represents the memory capacity of an NVIDIA A100 GPU. Present work reduces the activation memory required to fit the model. Details of the model configurations are provided in Table 3.

Model Size	Iteration Time (seconds)		Throughput Increase	Model FLOPs Utilization	Hardware FLOPs Utilization
	Full Recompute	Present Work			
22B	1.42	1.10	29.0%	41.5%	43.7%
175B	18.13	13.75	31.8%	51.4%	52.8%
530B	49.05	37.83	29.7%	56.0%	57.0%
1T	94.42	71.49	32.1%	56.3%	57.0%

Table 5: End-to-end iteration time. Our approach results in throughput increase of around 30%.

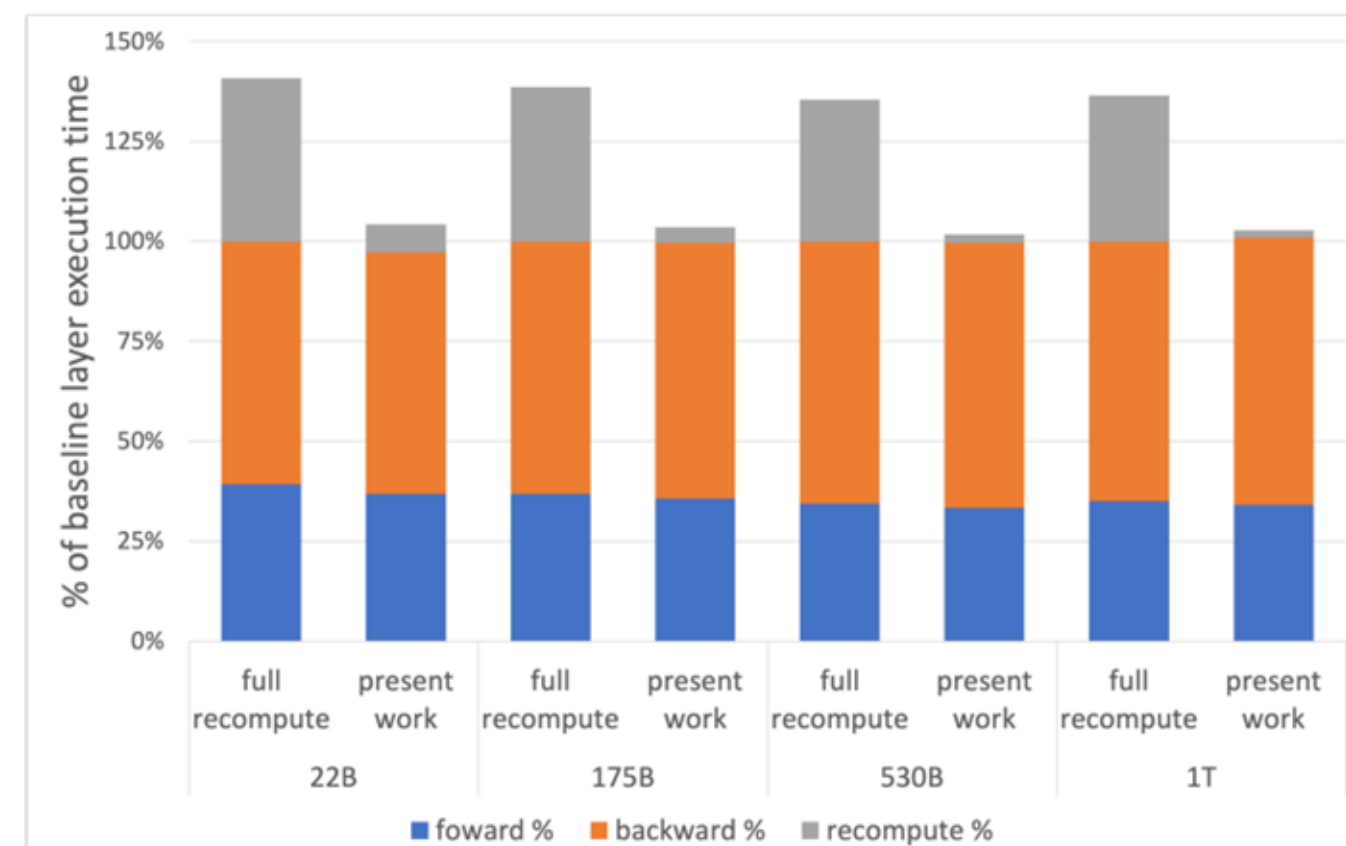


Figure 8: Per layer breakdown of forward, backward, and recompute times. Baseline is the case with no recomputation and no sequence parallelism. Present work includes both sequence parallelism and selective activation recomputation.



SAVE MEMORY BY OFFLOADING TO THE CPU

SCALING TRAINING

Scale out vs Scale up

Scale Out

Uses aggregate memory of multiple GPUs

Scale Up

Trade compute for memory from activations leveraging checkpoints

Compression techniques - low and mixed precisions

External memory - CPU

DEALING WITH MEMORY CONSTRAINTS

Offloading data to CPU memory (Heterogeneous DL Training) - Viewed as a computational graph

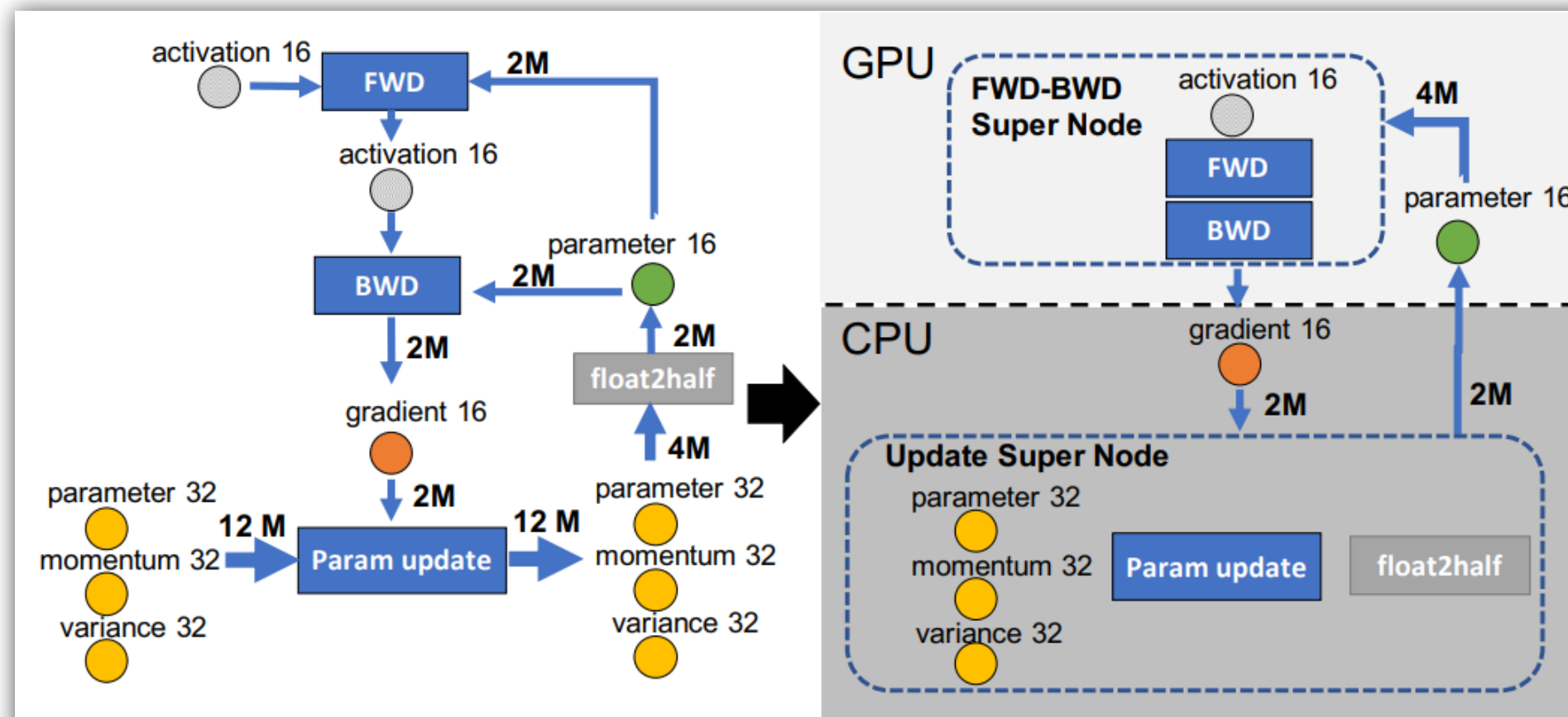
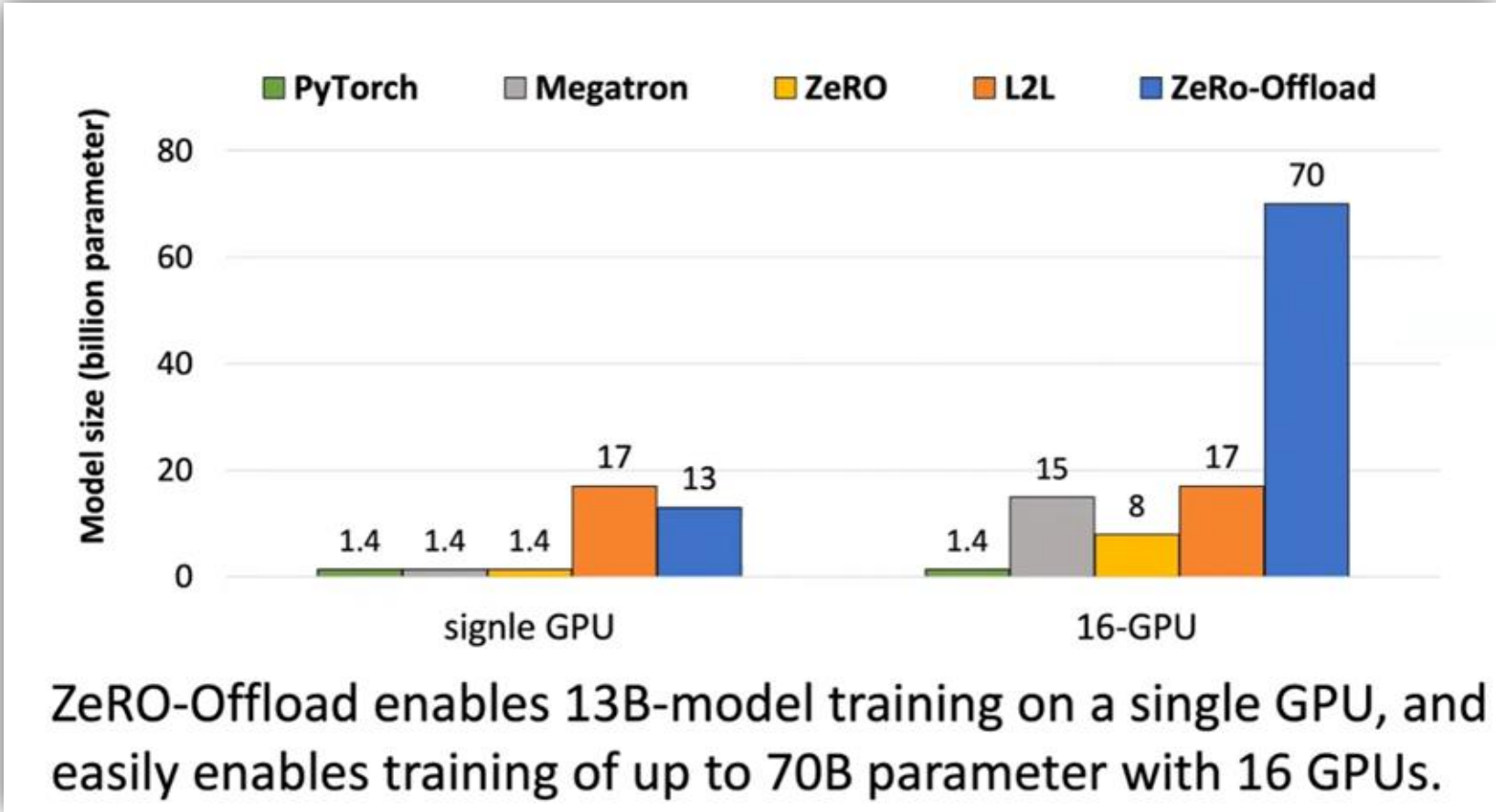
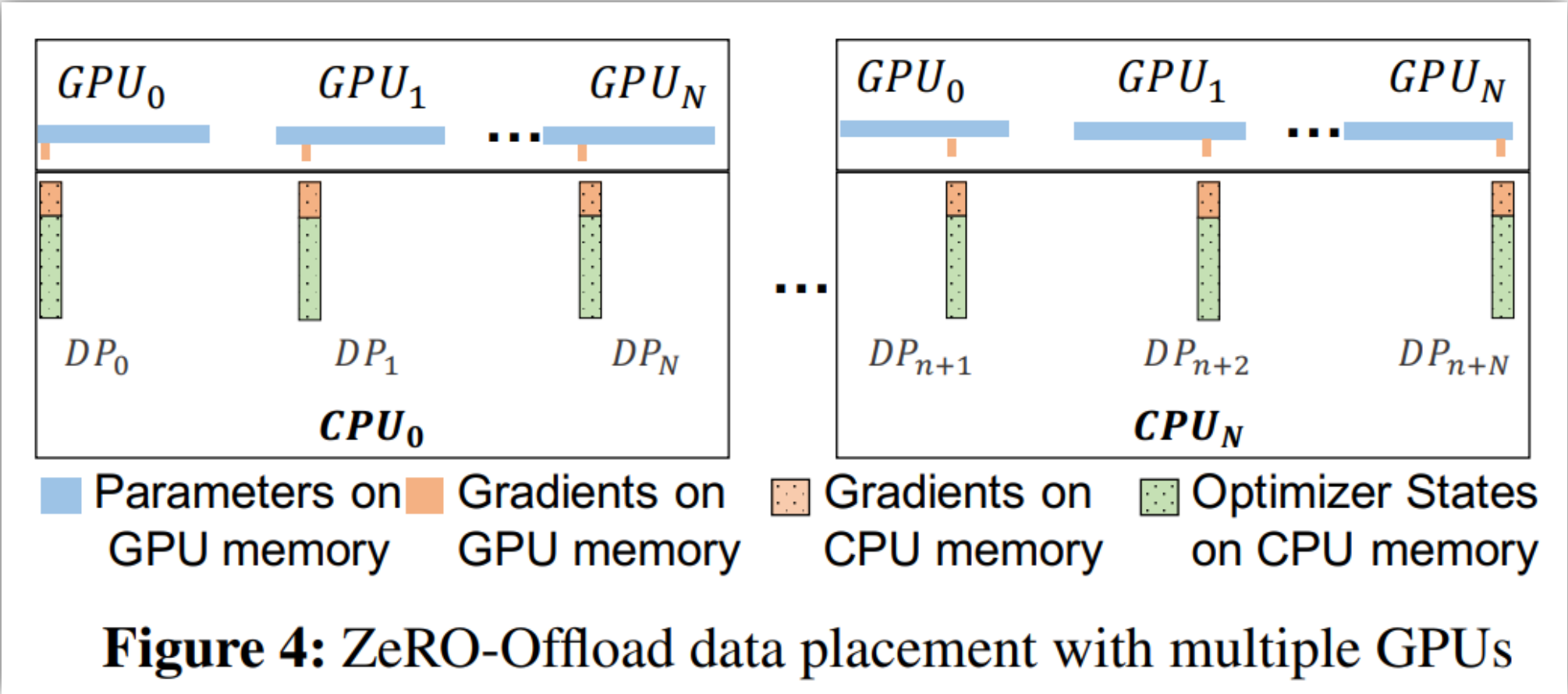


Figure 2: The dataflow of fully connected neural networks with M parameters. We use activation checkpoint to reduce activation memory to avoid activation migration between CPU and GPU.

- Circular nodes represent model states (number is precision)
 - Parameter16, gradient16, parameter32, momentum32, variance32
- Rectangular nodes represent computation
 - Forward, backward, param update
- Edges represent dataflow between nodes
 - Edge weight proportional to volume in bytes
- Offload gradients, optimizer states and optimizer computation to CPU
- Keep fp16 parameters and forward/backward pass on GPU
- Size of edges for M parameters: 2M-fp16 and 4M-fp32
 - Represents the volume of data movement from CPU to GPU

DEALING WITH MEMORY CONSTRAINTS

DeepSpeed ZeRO-Offload + with Data Parallelism



MIXTURE OF EXPERTS

The background features a smooth gradient from a deep green on the left to a bright yellow on the right. Overlaid on this is a complex, interconnected network of thin white lines and small dots, resembling a neural network or a data visualization. The network is denser on the right side, where it also appears to form a faint, wireframe-like structure of a face or a similar abstract shape.

MIXTURE OF EXPERTS

Not a new Idea - 1991

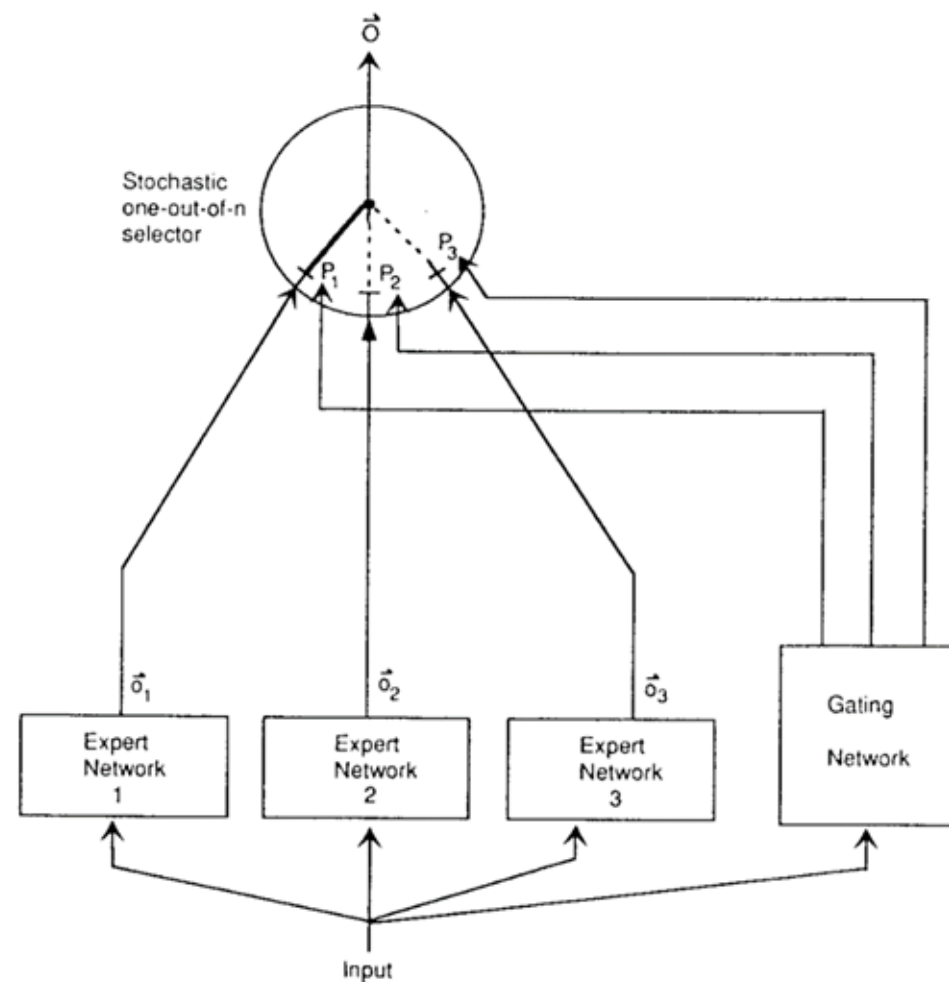
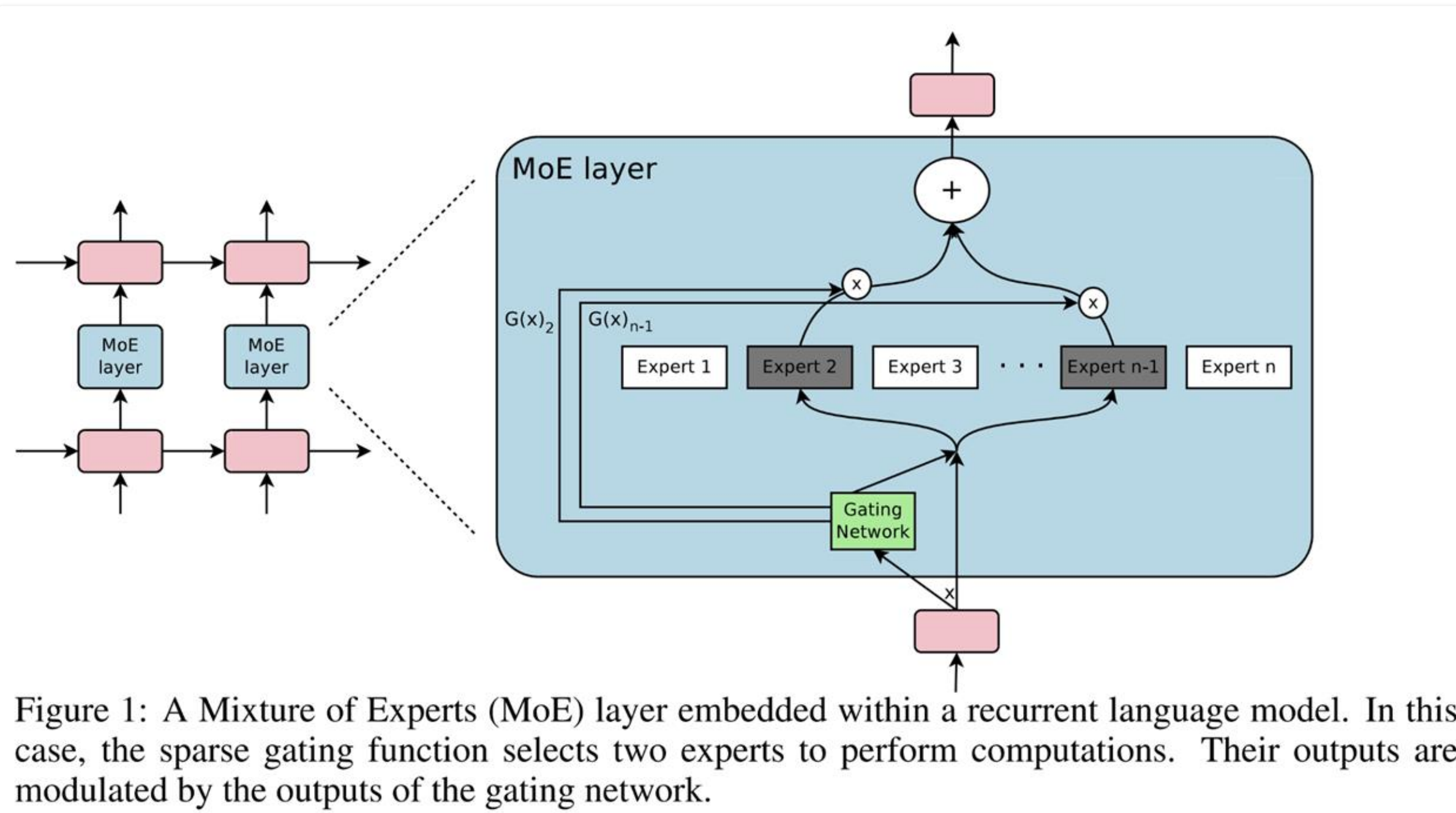


Figure 1: A system of expert and gating networks. Each expert is a feed-forward network and all experts receive the same input and have the same number of outputs. The gating network is also feedforward, and typically receives the same input as the expert networks. It has normalized outputs $p_j = \exp(x_j) / \sum_i \exp(x_i)$, where x_j is the total weighted input received by output unit j of the gating network. The selector acts like a multiple input, single output stochastic switch; the probability that the switch will select the output from expert j is p_j .

- Training a single, multilayer network to perform different subtasks is difficult
- Interaction effects slow learning and lead to poor generalization
- Prior knowledge of task separation is evidence for an expert system
- System is composed of a series of experts with a router (gating network)
- Weights of experts are decoupled
- Choice of objective function is critical - how to combine experts

MIXTURE OF EXPERTS

MoE in Neural Networks (2017)



Applied thousands of mixture of experts to language modelling inside of an RNN

MIXTURE OF EXPERTS

Switch Transformers

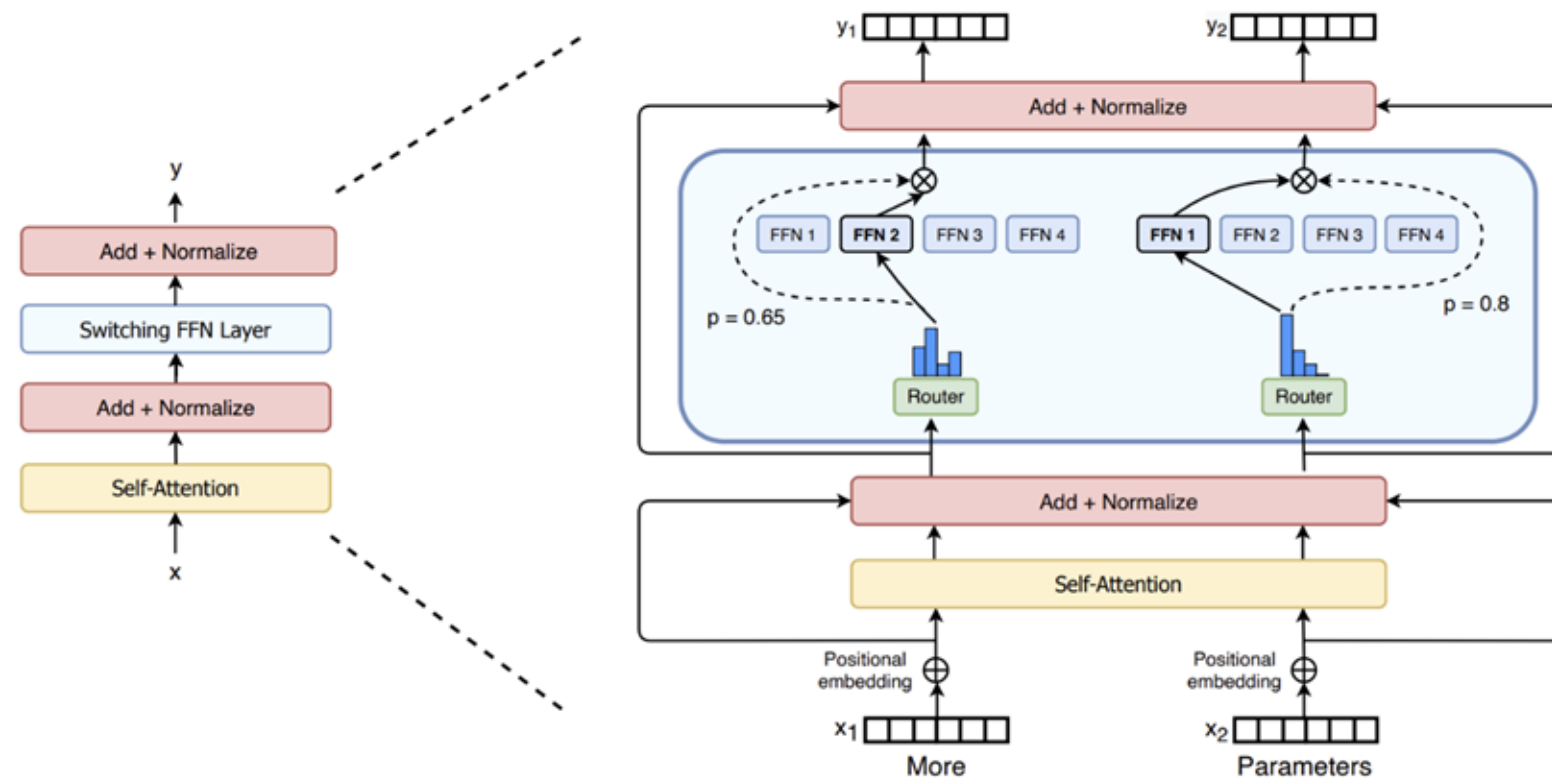


Figure 2: Illustration of a Switch Transformer encoder block. We replace the dense feed forward network (FFN) layer present in the Transformer with a sparse Switch FFN layer (light blue). The layer operates independently on the tokens in the sequence. We diagram two tokens ($x_1 = \text{"More"}$ and $x_2 = \text{"Parameters"}$ below) being routed (solid lines) across four FFN experts, where the router independently routes each token. The switch FFN layer returns the output of the selected FFN multiplied by the router gate value (dotted-line).

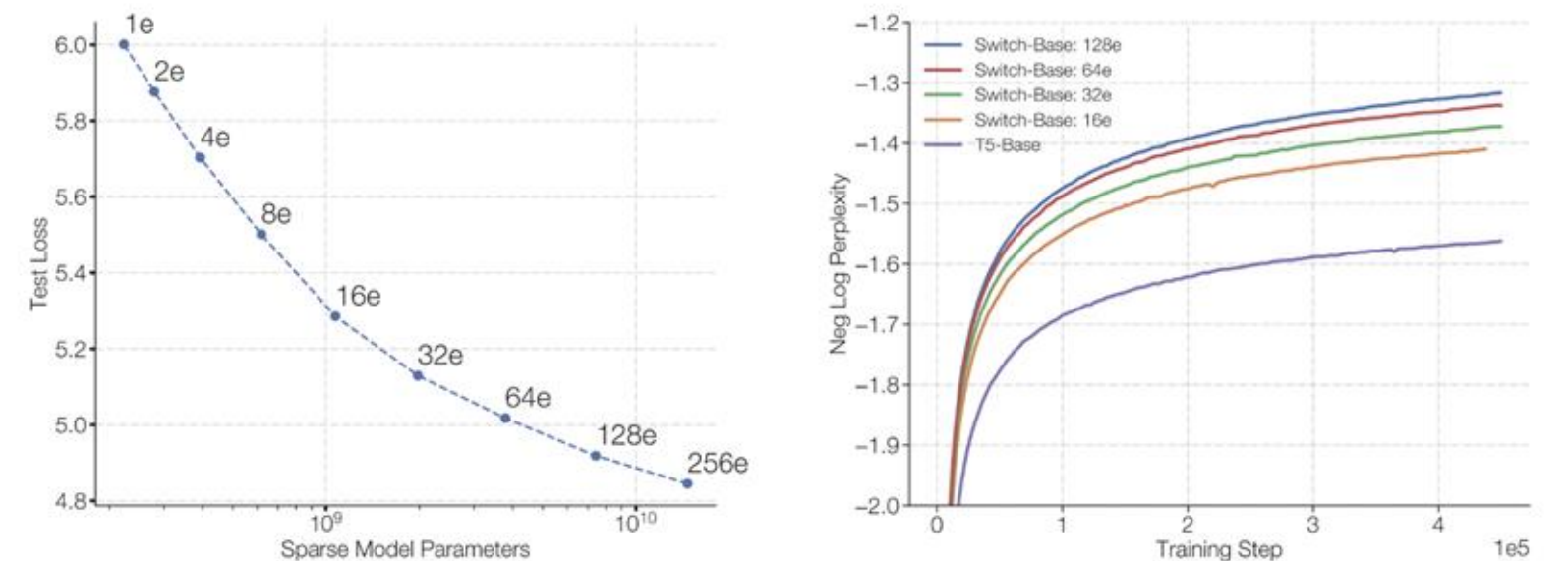


Figure 1: Scaling and sample efficiency of Switch Transformers. Left Plot: Scaling properties for increasingly sparse (more experts) Switch Transformers. Right Plot: Negative log perplexity comparing Switch Transformers to T5 (Raffel et al., 2019) models using the same compute budget.

Increasing the number of experts led to increase in accuracy

Applied mixture of experts to transformer models with sparse expert models

MIXTRAL

Mixtral of Experts

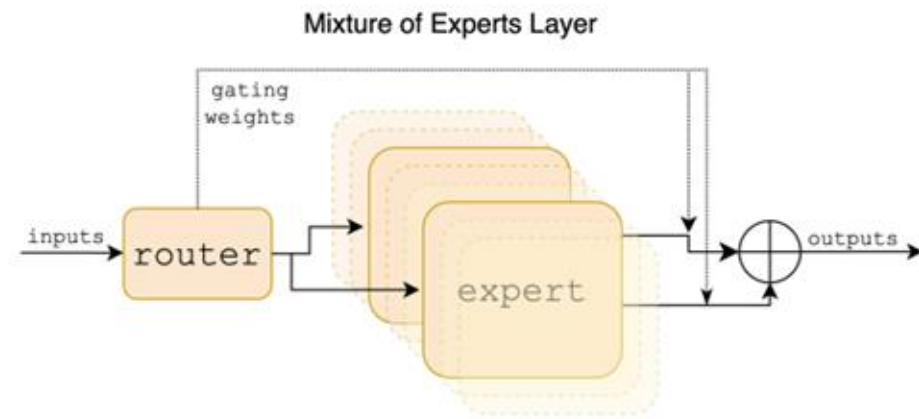


Figure 1: Mixture of Experts Layer. Each input vector is assigned to 2 of the 8 experts by a router. The layer's output is the weighted sum of the outputs of the two selected experts. In Mixtral, an expert is a standard feedforward block as in a vanilla transformer architecture.

- Mixture of expert layer is applied independently per token
- Each token is influenced by 2/8 experts
 - All feed forward sub-blocks are replaced with Mixture of expert layers.
- Apache 2.0 license

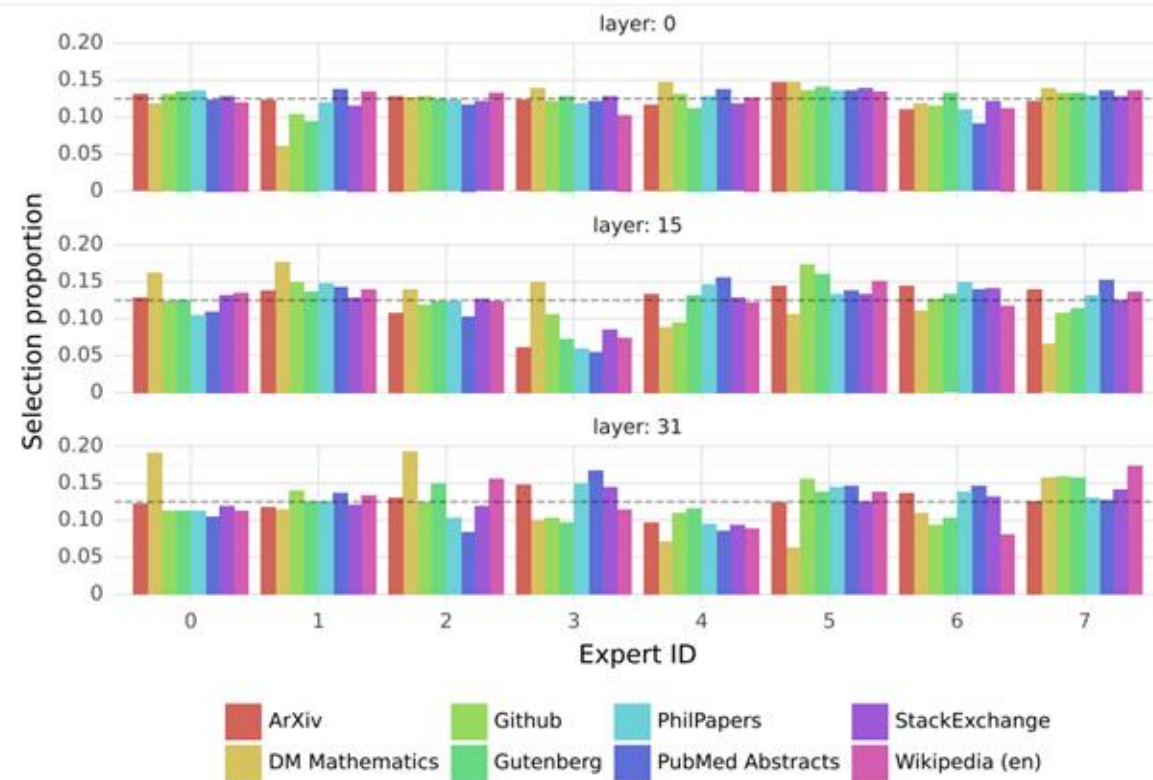


Figure 7: Proportion of tokens assigned to each expert on different domains from The Pile dataset for layers 0, 15, and 31. The gray dashed vertical line marks 1/8, i.e. the proportion expected with uniform sampling. Here, we consider experts that are either selected as a first or second choice by the router. A breakdown of the proportion of assignments done in each case can be seen in Figure 9 in the Appendix.

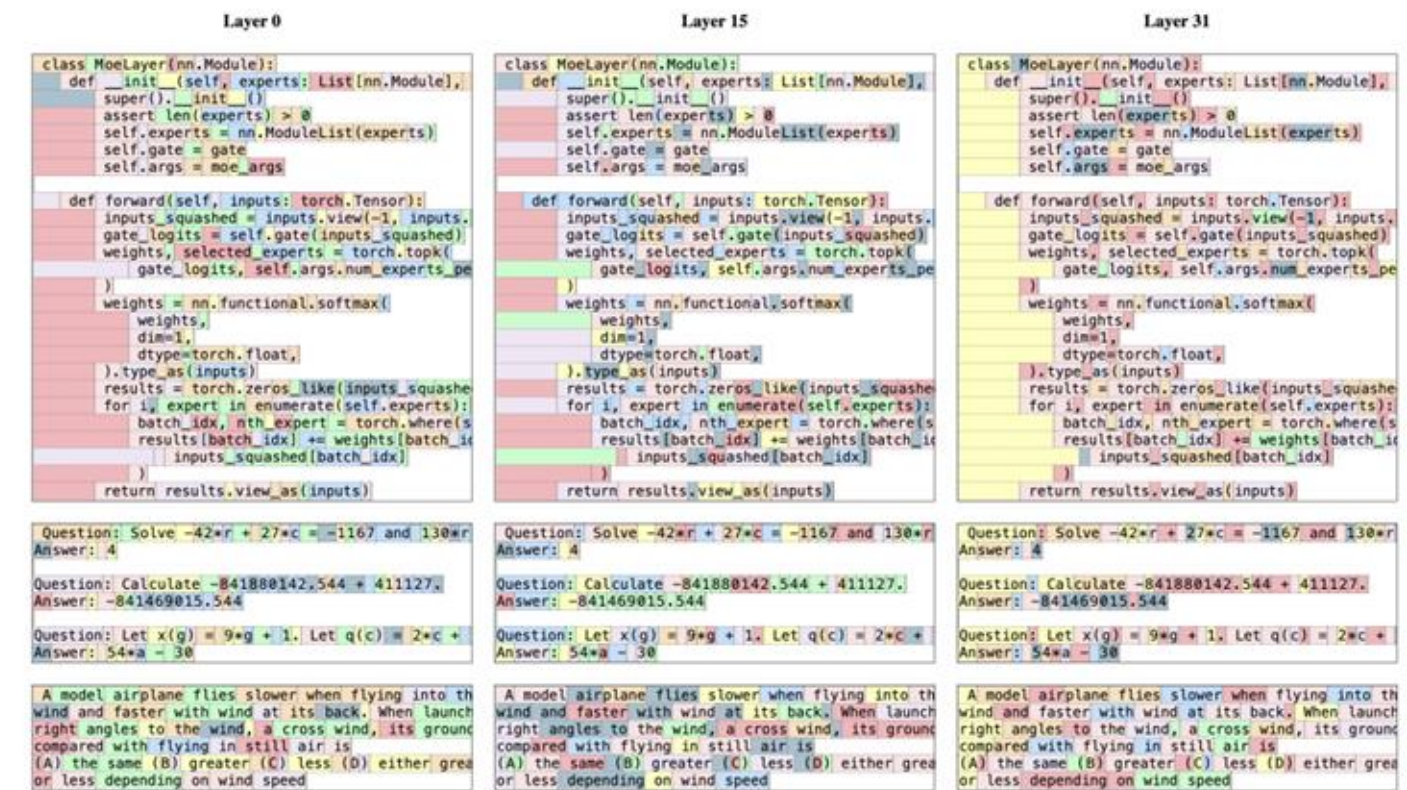


Figure 8: Text samples where each token is colored with the first expert choice. The selection of experts appears to be more aligned with the syntax rather than the domain, especially at the initial and final layers.

IO-AWARE ATTENTION ALGORITHM

FLASH ATTENTION

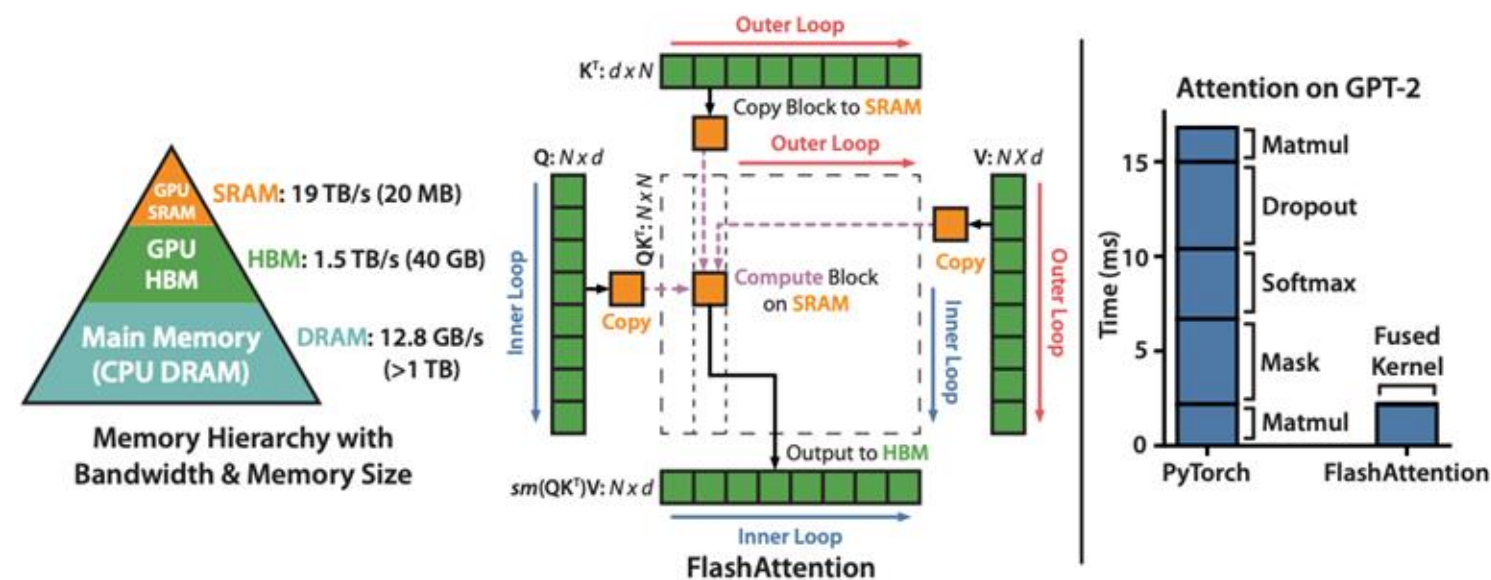


Figure 1: **Left:** FLASHATTENTION uses tiling to prevent materialization of the large $N \times N$ attention matrix (dotted box) on (relatively) slow GPU HBM. In the outer loop (red arrows), FLASHATTENTION loops through blocks of the K and V matrices and loads them to fast on-chip SRAM. In each block, FLASHATTENTION loops over blocks of Q matrix (blue arrows), loading them to SRAM, and writing the output of the attention computation back to HBM. **Right:** Speedup over the PyTorch implementation of attention on GPT-2. FLASHATTENTION does not read and write the large $N \times N$ attention matrix to HBM, resulting in an $7.6\times$ speedup on the attention computation.

- GPU memory access patterns matter to increase training efficiency
- Optimizes communication between on-chip SRAM (fast) and HBM memory (slower) using tiling
- Achieves linear scaling in sequence length vs quadratic for attention

FLASH ATTENTION

Attention	Standard	FLASHATTENTION
GFLOPs	66.6	75.2
HBM R/W (GB)	40.3	4.4
Runtime (ms)	41.7	7.3

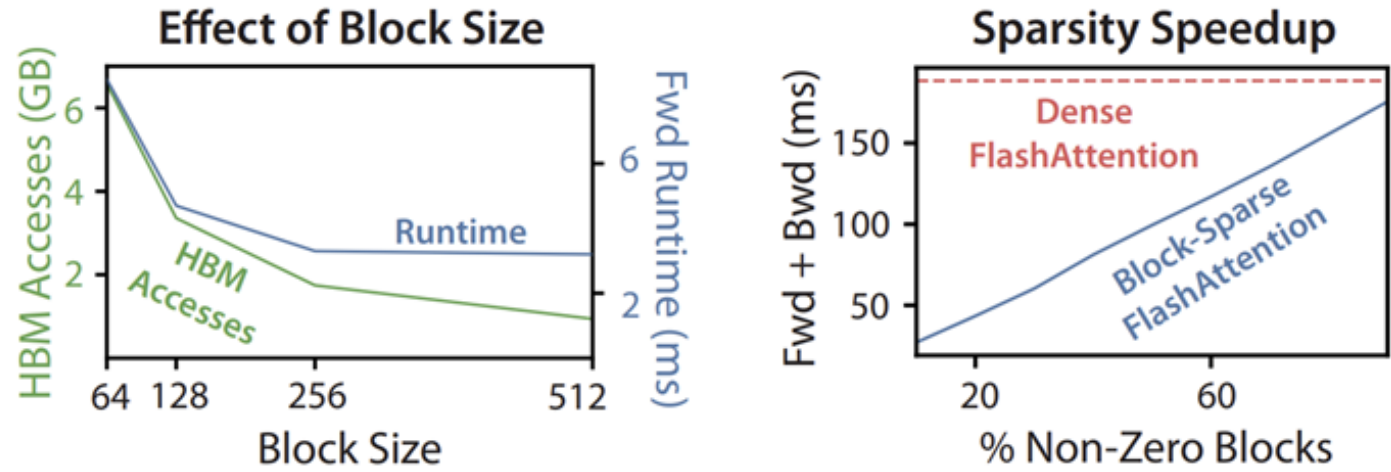


Figure 2: **Left:** Forward + backward runtime of standard attention and FLASHATTENTION for GPT-2 medium (seq. length 1024, head dim. 64, 16 heads, batch size 64) on A100 GPU. HBM access is the primary factor affecting runtime. **Middle:** Forward runtime of FLASHATTENTION (seq. length 1024, head dim. 64, 16 heads, batch size 64) on A100 GPU. Fewer HBM accesses result in faster runtime, up to a point. **Right:** The runtime (for seq. length 4K) of block-sparse FLASHATTENTION is faster than FLASHATTENTION by a factor proportional to the sparsity.

NCCL: NVIDIA COLLECTIVE COMMUNICATION LIBRARY

MULTI-GPU TRAINING

NCCL: NVIDIA Collective Communication Library

NVIDIA Collective Communications Library

1. Automatic topology detection
2. Graph search for the optimal set of rings and trees with the highest bandwidth and lowest latency over PCIe and NVLink high-speed interconnects within a node and over NVIDIA Mellanox Network across nodes
3. Provide routines such as all-gather, all-reduce, broadcast, reduce, reduce-scatter, point-to-point send and receive
4. Integrated within several Deep Learning frameworks such as Caffe2, MxNet, PyTorch
5. Optimized for all platforms, from desktop to DGX Superpod.

PCI



NVLink



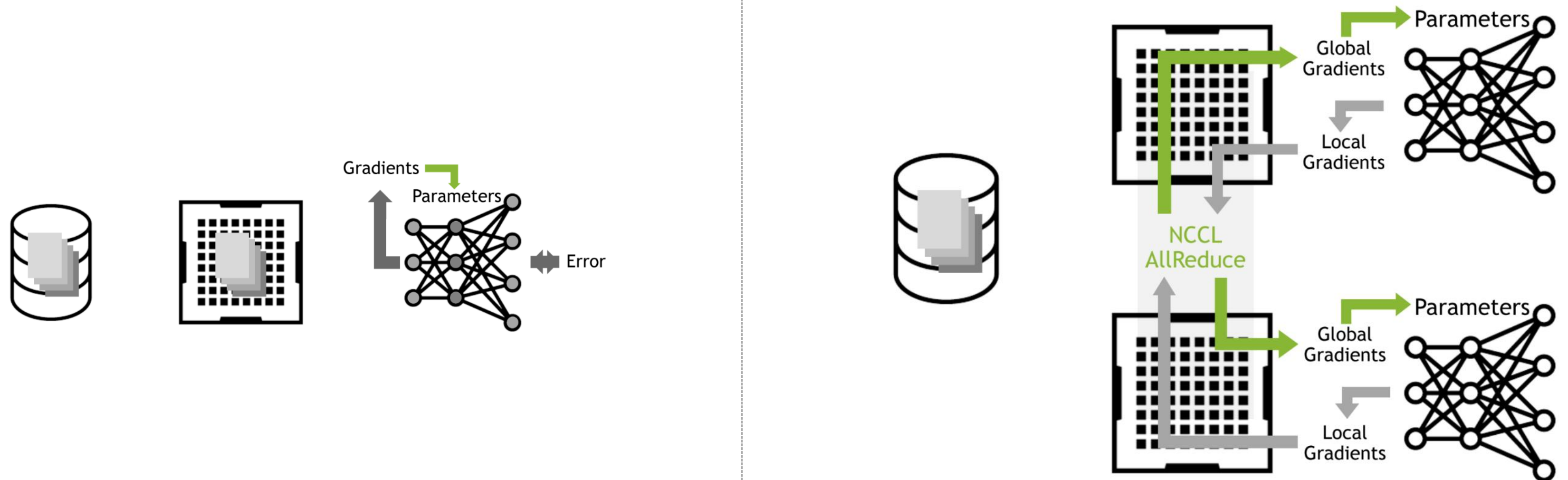
NVLink +
Infiniband



MULTI-GPU TRAINING

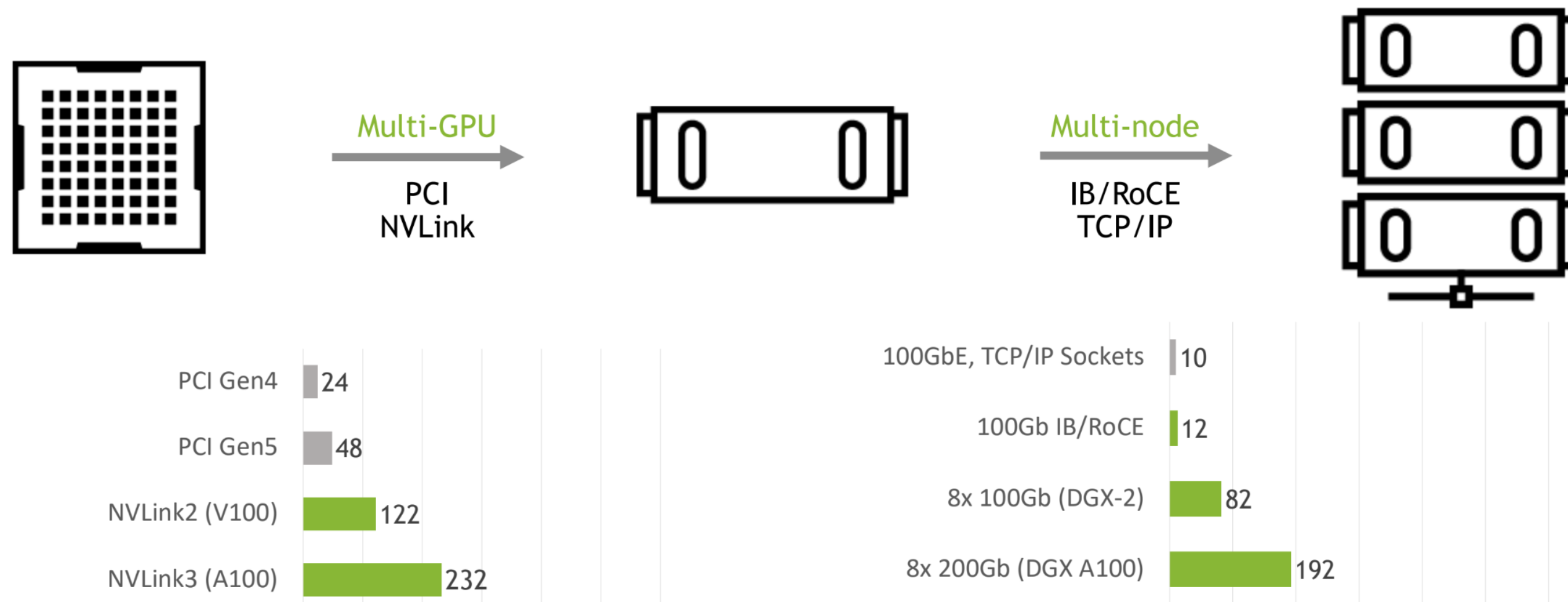
NCCL : NVIDIA Collective Communication Library

In Deep Learning, AllReduce operations used to sum gradients over multiple GPUs



TENSOR PARALLEL TRANSFORMER LAYER

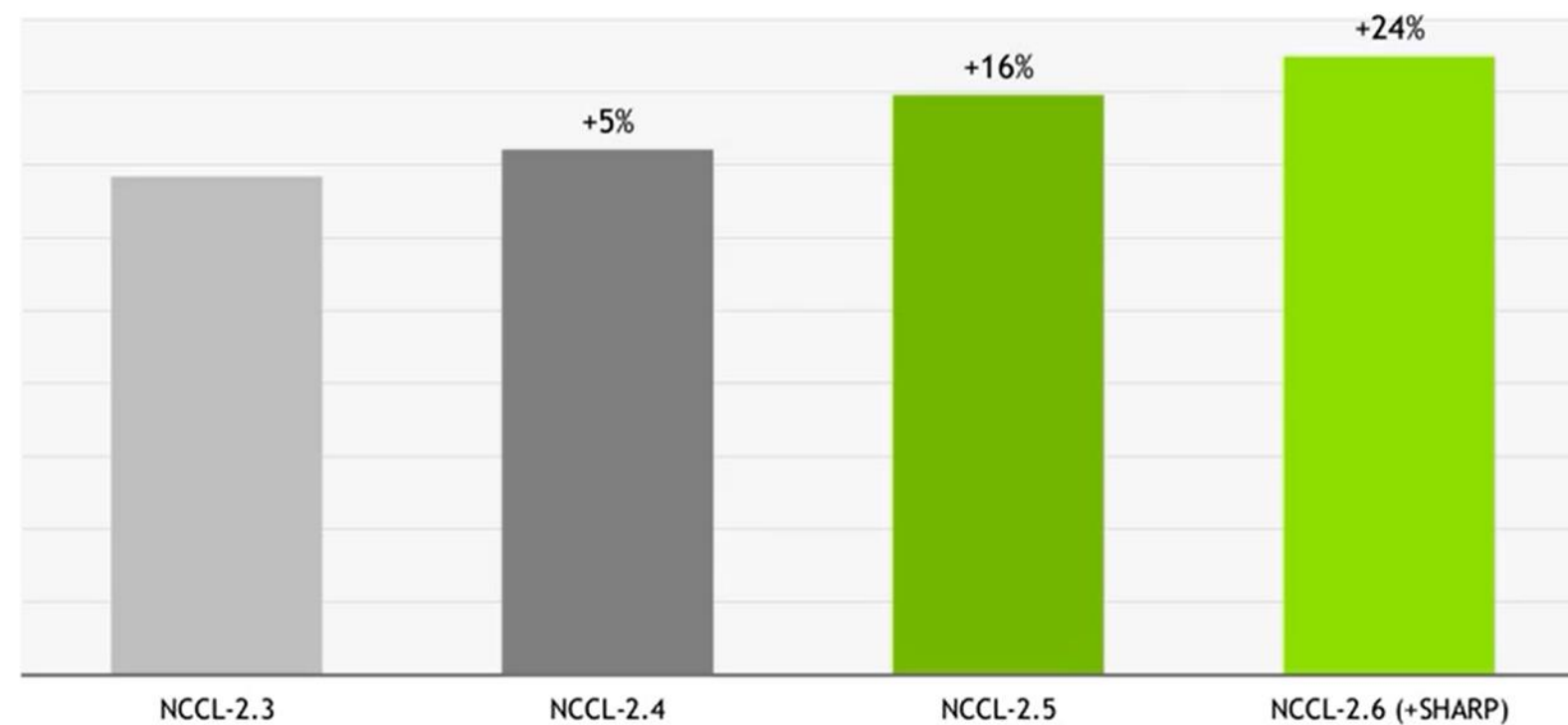
Communication Expensive



MULTI-GPU TRAINING

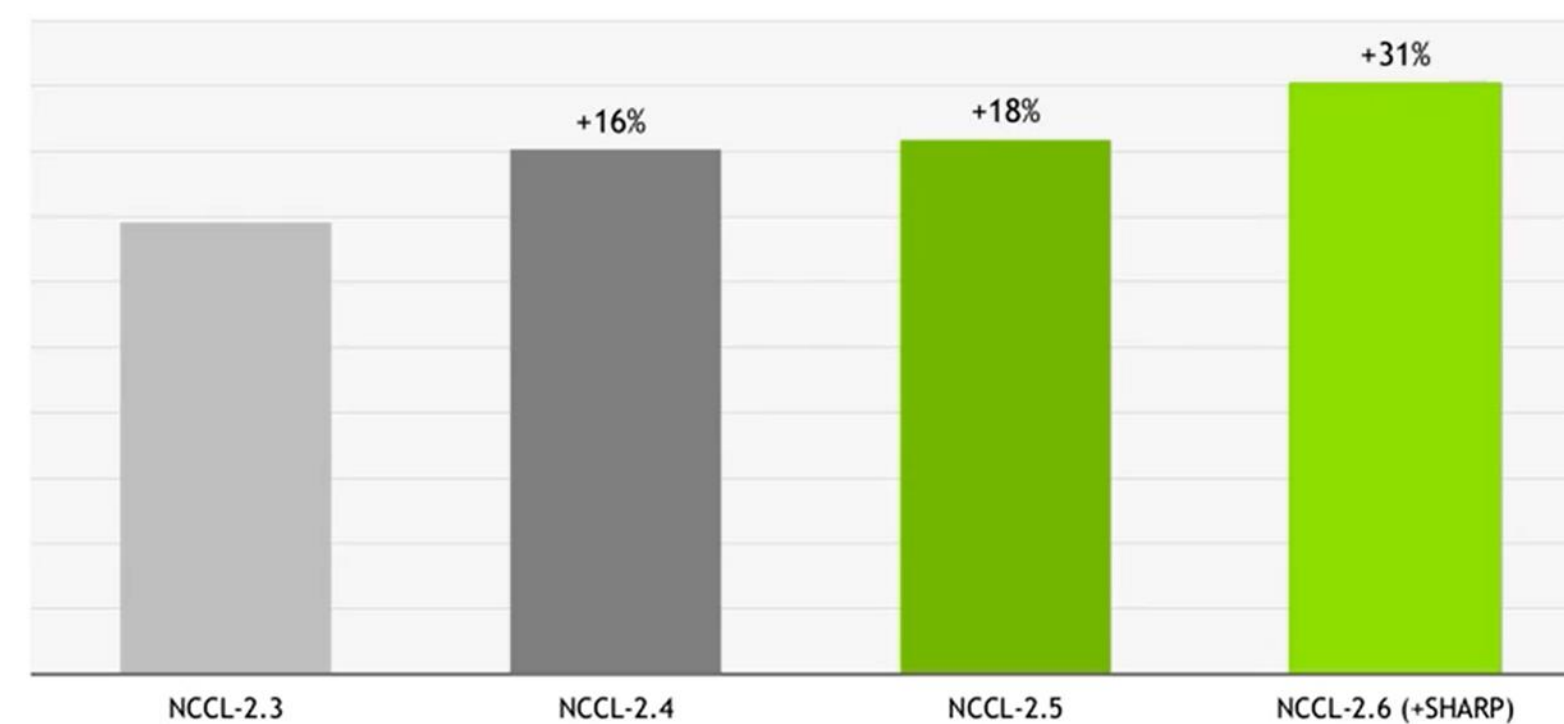
NCCL

GNMT



24xDGX1V + 4xMellanox CX-6, GNMT benchmark: Batch Size=32, Overlap=0.15

Transformer

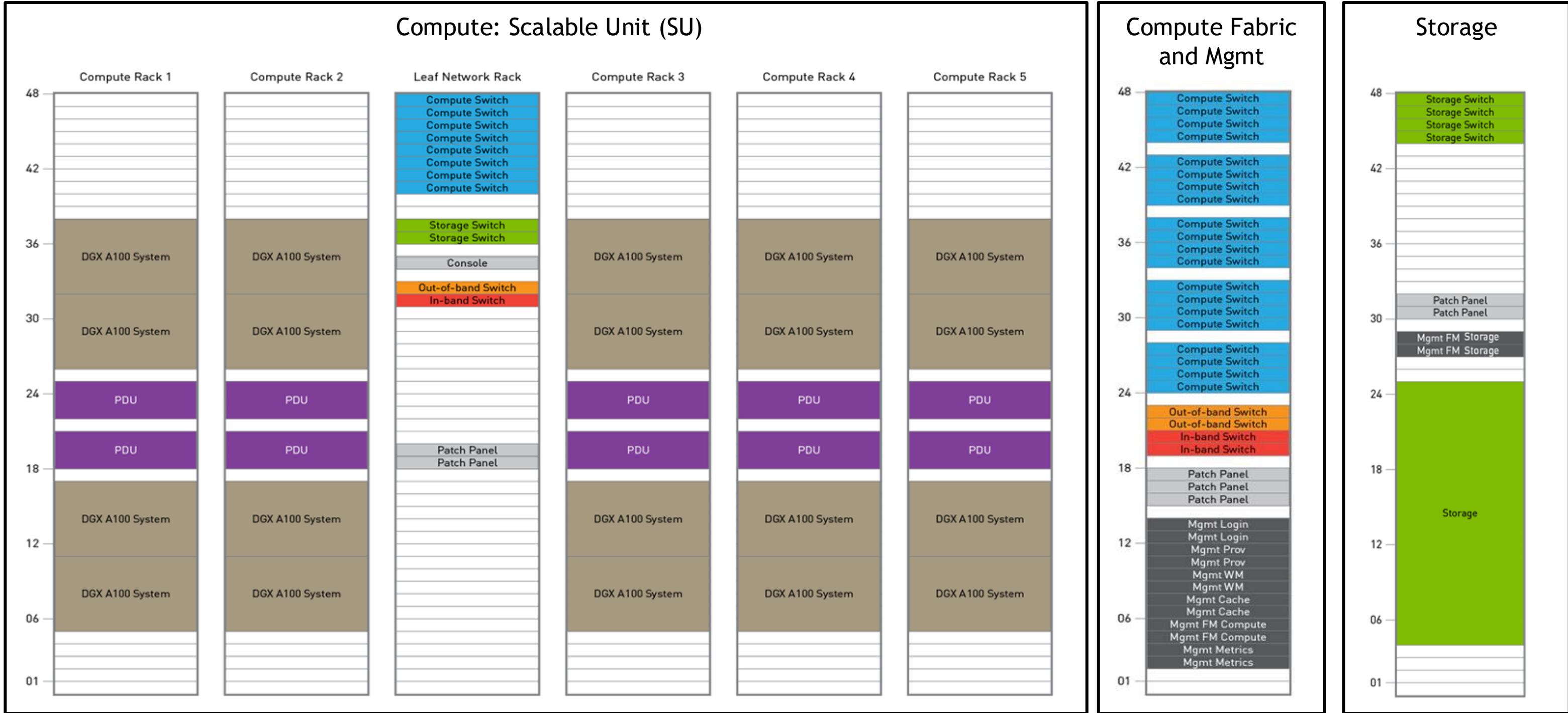


32xDGX1V + 4xMellanox CX-6, Transformer benchmark: Batch Size=640, Overlap=0.20

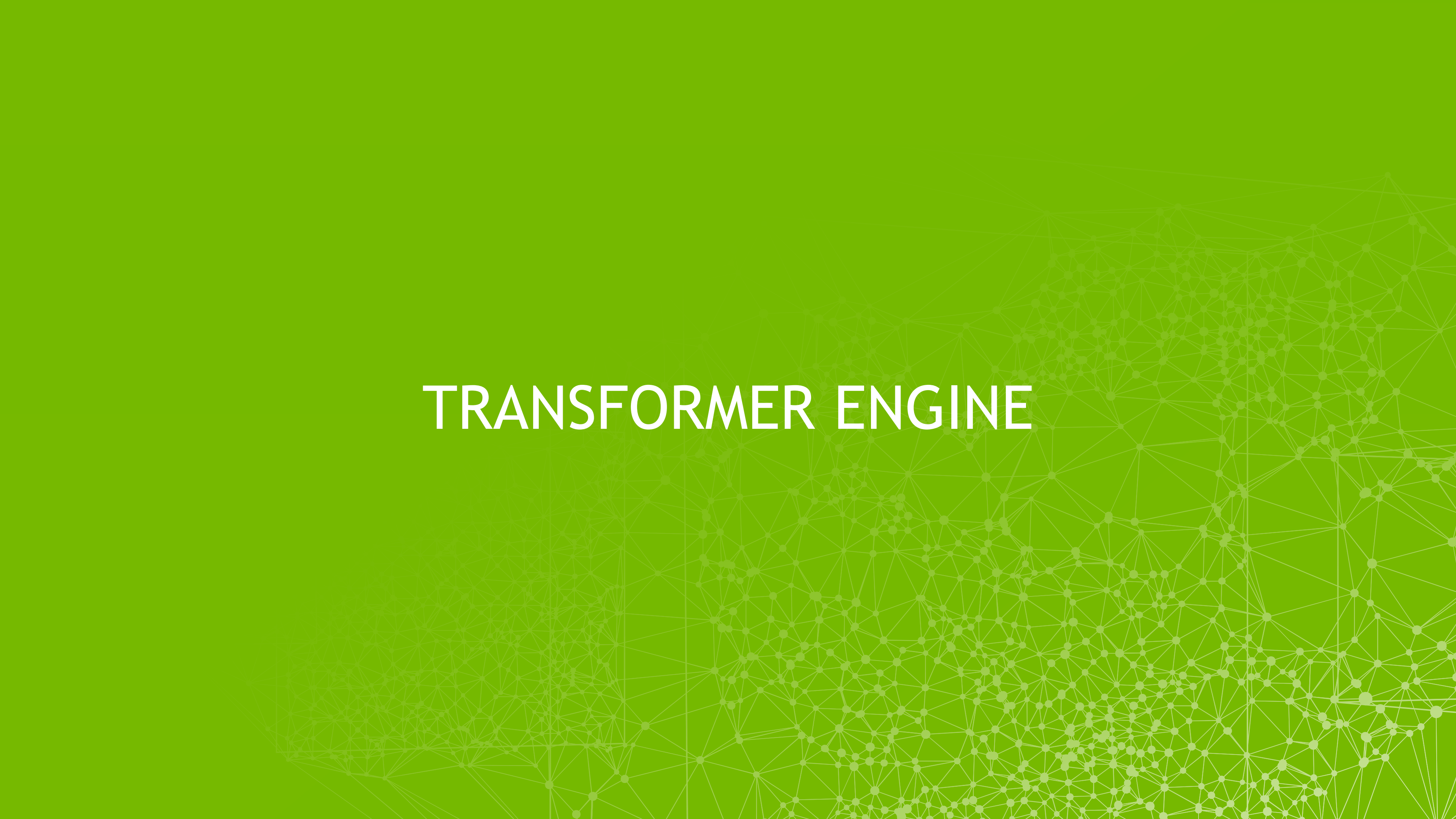
HARDWARE DESIGN

The background features a smooth gradient from a vibrant green on the left to a deep blue on the right. Overlaid on this gradient is a complex, white network of lines and dots, resembling a molecular structure or a digital circuit board. The network is denser on the right side and fades out towards the left.

MODULARITY: RAPID DEPLOYMENT

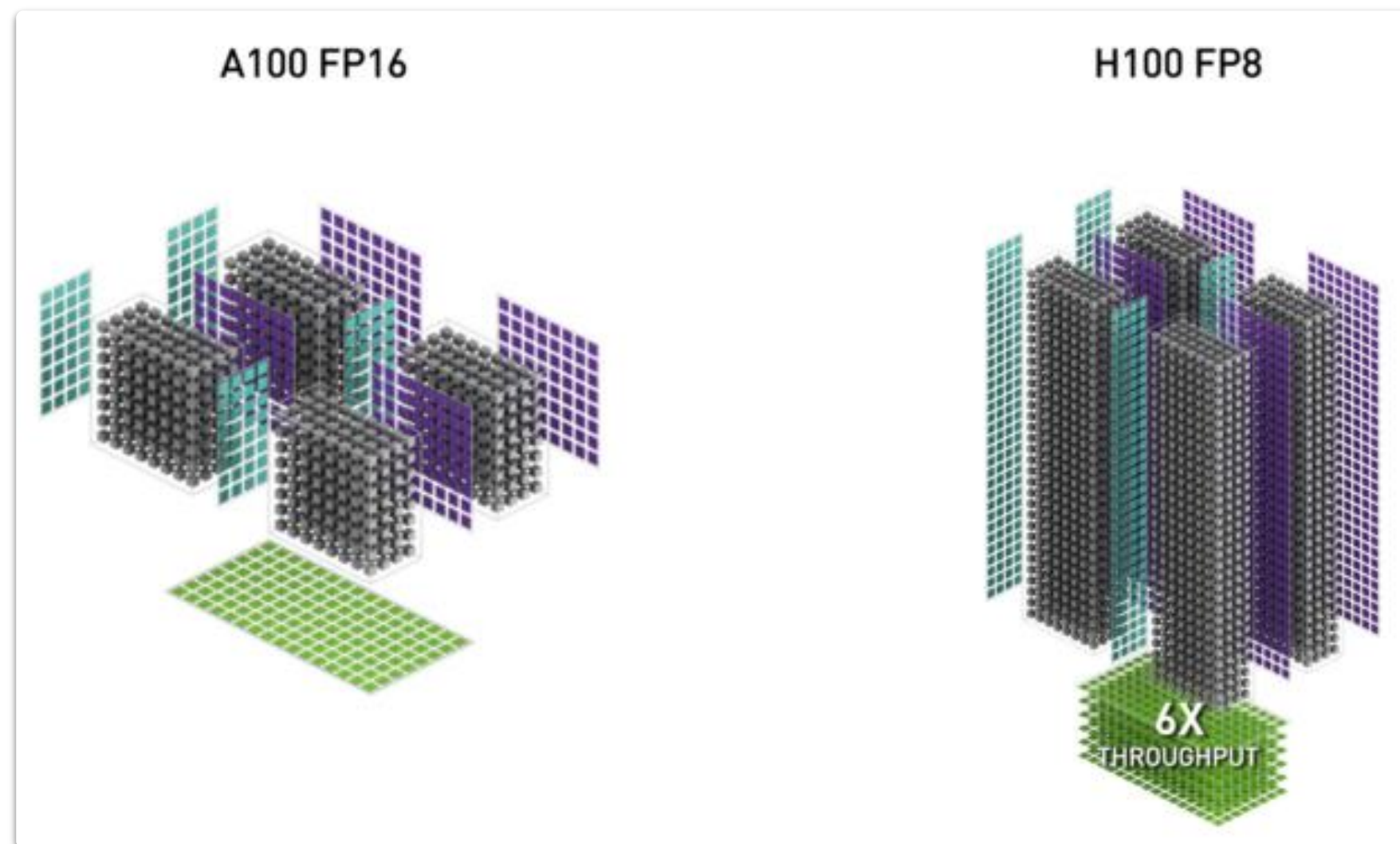


TRANSFORMER ENGINE

The background of the image features a smooth gradient from a vibrant green on the left to a deep blue on the right. Overlaid on this gradient is a complex, abstract network of thin white lines connecting numerous small, light-blue circular nodes. This network is most dense on the right side of the image, creating a sense of depth and connectivity, reminiscent of a neural network or a data visualization.

NVIDIA H100 ARCHITECTURE

TensorCores and Mixed Precision



	sign	exponent							mantissa							FP32 = 0.3952	
FP16	0	0	1	1	0	1	1	0	0	1	0	1	0	0	1	1	= 0.395264
BF16	0	0	1	1	1	1	1	0	1	1	0	0	1	0	1	0	= 0.394531
FP8 E4M3	0	0	1	0	1	1	0	1									= 0.40625
FP8 E5M2	0	0	1	1	0	1	1	0									= 0.375

TRANSFORMER ENGINE

Accelerating Transformer with FP8 precision on Hopper GPUs

- An open-source library implementing the FP8 recipe for Transformer building blocks
- Optimized for FP8 and other datatypes
- Supports pyTorch, with JAX and TF support coming soon
- Composable with the native framework operators
- Supports model and sequence parallelism

```
## Transformer Engine
transformer_engine: True # turn on Transformer Engine
fp8: True # enables fp8 in TransformerLayer forward
fp8_e4m3: False # sets fp8_format = recipe.Format.E4M3
fp8_hybrid: True # sets fp8_format = recipe.Format.HYBRID
fp8_margin: 0 # scaling margin
fp8_interval: 1 # scaling update interval
fp8_amax_history_len: 32 # Number of steps for which amax history is recorded per tensor
fp8_amax_compute_algo: max # 'most_recent' or 'max'. Algorithm for computing amax from history
use_emha: False
```

THE LAB

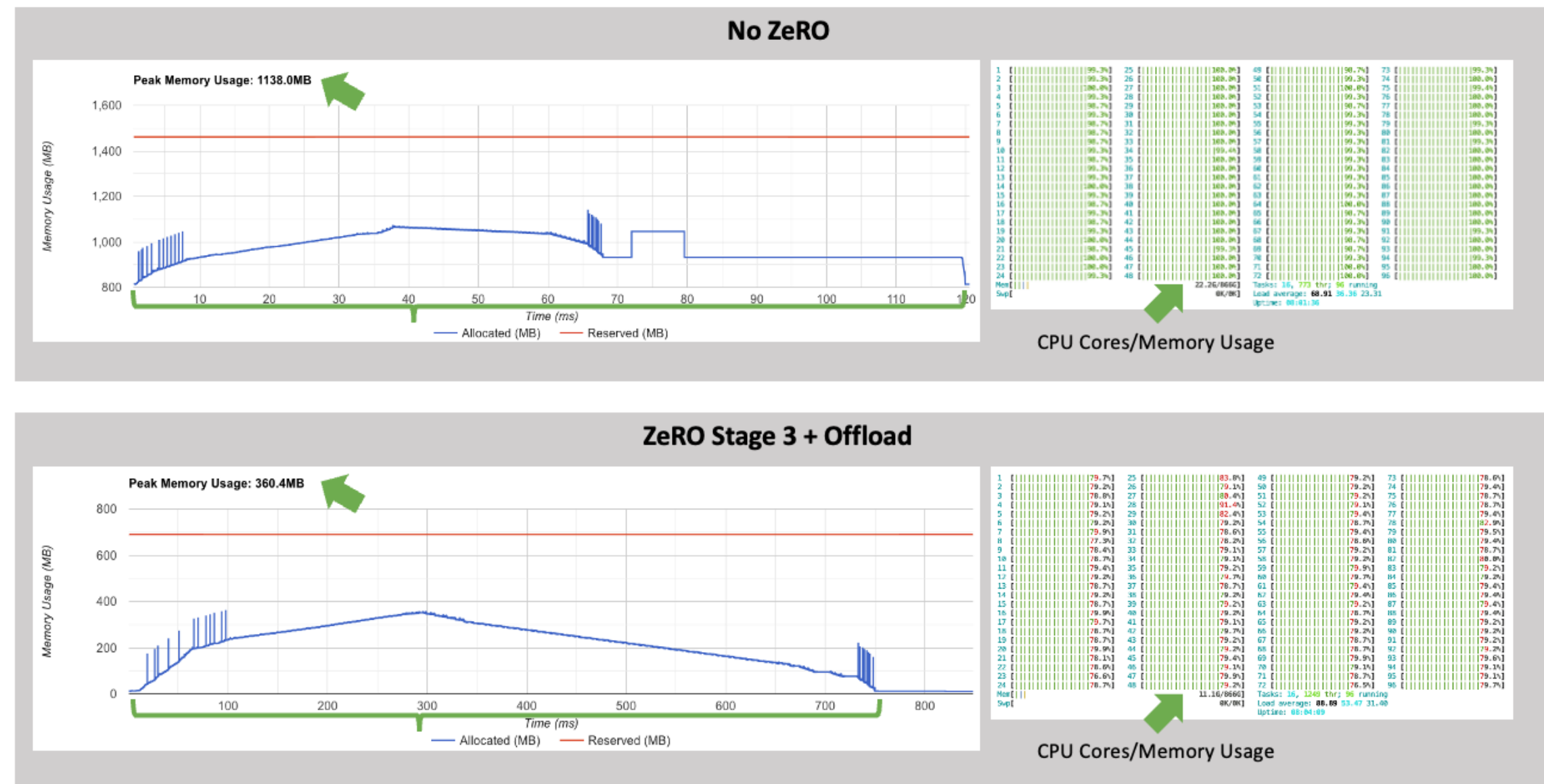
The background of the image is a smooth gradient transitioning from a vibrant green on the left to a deep blue on the right. Overlaid on this gradient is a complex, abstract network of white dots and thin white lines. These dots and lines are distributed across the entire frame, creating a sense of connectivity and digital structure. The density of the network appears to increase towards the right side of the image, where the lines and dots are more closely packed together.

LAB 2

Overview

Distributed Training for Computer Vision

- Learn how to train a simple Image Classifier
- Implement a vanilla pipeline parallel distribution
- Port the code to DeepSpeed library
- Scale training using Data parallel distribution
- Optimize training with DeepSpeed autotuning and Zero Redundancy Optimizer
- Mixture of Experts

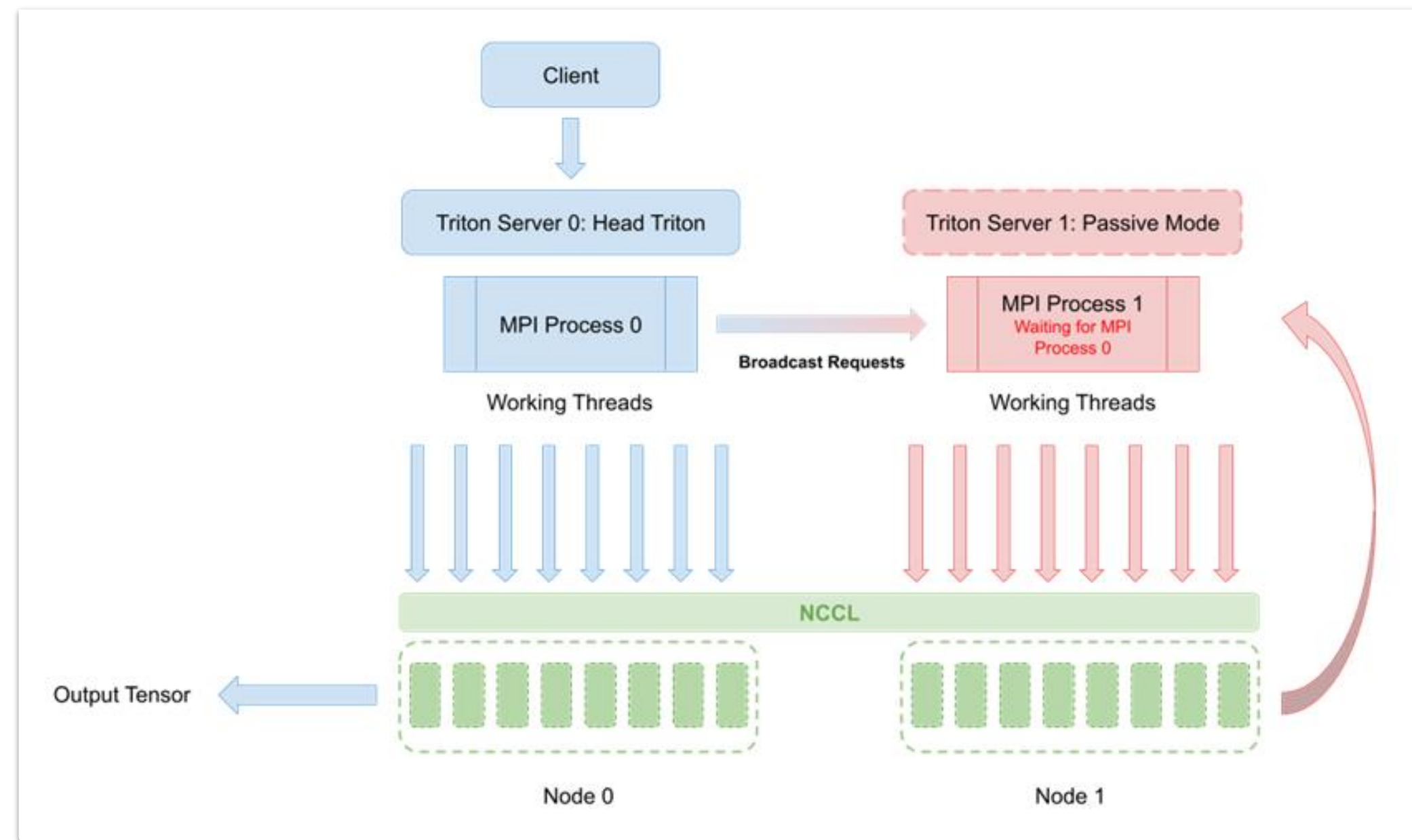




IN THE NEXT CLASS

NEXT CLASS

Discuss large model deployment challenges and solutions



TensorRT-LLM and Triton Inference Server for Multi-Node Inference for large scale Transformer Model